le service ai-service basé sur l'IA , permet la creation et la configuration des packs et des produits via des prompt (une seule prompt pour cree une garantie ou meme pour configurer un pack ou un produit), aussi ce service permet de recommender des produits/packs selon le profil du client ainsi que son besoin .je voulais savoir si le contenu est coherent du service ai-service et la GEstionProduit
Pour un stage PFE ingenieur , au sein de vermeg : ici la gestionUSer complete , où il ya la gestion des comptes user , pour le moment seul l'admin il accede au plateforme , je voulais ajouter la configuration du jwt , pour tous les gestion du projet,pour le projet entier, sans avoir des erreurs dans le reste des gestions !

```java
package tn.vermeg.gestionuser.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import tn.vermeg.gestionuser.utils.JwtUtil;
import java.io.IOException;
import java.util.Collections;

@Component
public class JwtFilter extends OncePerRequestFilter {
   @Override
   protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain filterChain)
           throws ServletException, IOException {

       // 🔥 IMPORTANT: Allow OPTIONS requests to bypass JWT validation
       if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
           filterChain.doFilter(request, response);
           return;
       }

       final String authHeader = request.getHeader("Authorization");
       if (authHeader == null || !authHeader.startsWith("Bearer ")) {
           filterChain.doFilter(request, response);
           return;}

       try {
           String token = authHeader.substring(7);
```

```
            String userName = JwtUtil.extractUsername(token);
            String role = JwtUtil.extractRole(token);
            if (userName != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
                // S'assurer que le rôle a le préfixe ROLE_ pour Spring
Security
                 String formattedRole = role.startsWith("ROLE_") ? role :
"ROLE_" + role;
                SimpleGrantedAuthority authority = new
SimpleGrantedAuthority(formattedRole);
                UsernamePasswordAuthenticationToken authentication = new
UsernamePasswordAuthenticationToken(
                        userName, null, Collections.singleton(authority));
                authentication.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authentication);}
        } catch (Exception e)
{response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("JWT Error: " + e.getMessage());
            return;}
        filterChain.doFilter(request, response);}
}
package tn.vermeg.gestionuser.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.cors.reactive.CorsConfigurationSource;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.util.List;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:4200")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("Authorization", "Content-Type", "Accept",
"X-Requested-With")
                .exposedHeaders("Authorization")
                .allowCredentials(true);
    }
}
//     @Bean
//     public CorsConfigurationSource corsConfigurationSource() {
//         CorsConfiguration config = new CorsConfiguration();
//         config.setAllowCredentials(true);
//         config.setAllowedOrigins(List.of("http://localhost:4200"));
```

```
//
config.setAllowedMethods(List.of("GET","POST","PUT","DELETE","OPTIONS"));
//        config.setAllowedHeaders(List.of("*"));
//
//        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
//        source.registerCorsConfiguration("/**", config);
//        return source;
//    }
package tn.vermeg.gestionuser.config;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

import java.util.Arrays;

@Aspect
@Component
public class LoggingAspect {

   private final Logger logger = LoggerFactory.getLogger(this.getClass());
   @Pointcut("within(tn.vermeg.gestionuser.services..*) ||
within(tn.vermeg.gestionuser.controllers..*)")
   public void applicationPackagePointcut() {}
   @Before("applicationPackagePointcut()")
   public void logBefore(JoinPoint joinPoint) {
       String currentUser =
SecurityContextHolder.getContext().getAuthentication() != null
           ?
SecurityContextHolder.getContext().getAuthentication().getName()
           : "System";
       logger.info("[AUDIT] User: {} | Entering method: {}.{}() with
arguments = {}",
           currentUser,
           joinPoint.getSignature().getDeclaringTypeName(),
           joinPoint.getSignature().getName(),
           Arrays.toString(joinPoint.getArgs()));}

   @AfterReturning(pointcut = "applicationPackagePointcut()", returning =
"result")
   public void logAfterReturning(JoinPoint joinPoint, Object result) {
       String currentUser =
SecurityContextHolder.getContext().getAuthentication() != null
           ?
SecurityContextHolder.getContext().getAuthentication().getName()
           : "System";
```

```
        logger.info("[AUDIT] User: {} | Exiting method: {}.{}() with result =
{}",
            currentUser,
            joinPoint.getSignature().getDeclaringTypeName(),
            joinPoint.getSignature().getName(),
            result);}
}
package tn.vermeg.gestionuser.controllers;

import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionuser.dto.AuthRequest;
import tn.vermeg.gestionuser.dto.RegisterRequest;
import tn.vermeg.gestionuser.entities.Admin;
import tn.vermeg.gestionuser.entities.Client;
import tn.vermeg.gestionuser.repositories.AdminRepository;
import tn.vermeg.gestionuser.repositories.ClientRepository;
import tn.vermeg.gestionuser.services.AdminService;
import tn.vermeg.gestionuser.services.ClientService;
import tn.vermeg.gestionuser.utils.JwtUtil;

import java.util.Map;

@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "http://localhost:4200")
public class AuthController {
   private final AdminService adminService;
   private final PasswordEncoder passwordEncoder;
   private final AdminRepository adminRepository;
   private final ClientRepository clientRepository;

   public AuthController(AdminService adminService, ClientService
clientService,
                         PasswordEncoder passwordEncoder,
                         AdminRepository adminRepository, ClientRepository
clientRepository) {
       this.adminService = adminService;
       this.passwordEncoder = passwordEncoder;
       this.adminRepository = adminRepository;
       this.clientRepository = clientRepository;
   }

   @PostMapping("/login")
   public ResponseEntity<?> login(@RequestBody AuthRequest request) {
       // dY"1 VAcrification ADMIN
       Admin admin = adminRepository.findByUserName(request.getUserName());
       if (admin != null && passwordEncoder.matches(request.getPassword(),
admin.getPassword())) {
           String accessToken =
JwtUtil.generateAccessToken(admin.getUserName(), "ROLE_ADMIN");
```

```
            String refreshToken =
JwtUtil.generateRefreshToken(admin.getUserName());
            return ResponseEntity.ok(new AuthResponse(accessToken,
refreshToken, admin, "ROLE_ADMIN"));
        }

        // dY"1 VAcrification CLIENT
        Client client =
clientRepository.findByUserName(request.getUserName());
        if (client != null && passwordEncoder.matches(request.getPassword(),
client.getPassword())) {
            String accessToken =
JwtUtil.generateAccessToken(client.getUserName(), "ROLE_CLIENT");
            String refreshToken =
JwtUtil.generateRefreshToken(client.getUserName());
            return ResponseEntity.ok(new AuthResponse(accessToken,
refreshToken, client, "ROLE_CLIENT"));
        }

        return ResponseEntity.status(401).body("Identifiants incorrects");
    }

    @PostMapping("/refresh")
    public ResponseEntity<?> refresh(@RequestBody Map<String, String> request)
{
        String refreshToken = request.get("refreshToken");
        if (refreshToken == null) {
            return ResponseEntity.badRequest().body("Refresh token is
missing");
        }

        try {
            String userName = JwtUtil.extractUsername(refreshToken);
            // VAcrifier si l'utilisateur existe encore
            Admin admin = adminRepository.findByUserName(userName);
            Client client = clientRepository.findByUserName(userName);

            String role = (admin != null) ? "ROLE_ADMIN" : (client != null ?
"ROLE_CLIENT" : null);

            if (role == null) {
                return ResponseEntity.status(401).body("User not found");
            }

            if (JwtUtil.validateToken(refreshToken, userName)) {
                String newAccessToken = JwtUtil.generateAccessToken(userName,
role);
                return ResponseEntity.ok(Map.of("accessToken",
newAccessToken));
            }
        } catch (Exception e) {
            return ResponseEntity.status(401).body("Invalid refresh token");
        }
```

```
        return ResponseEntity.status(401).body("Invalid refresh token");
    }

    public static class AuthResponse {
        public String accessToken;
        public String refreshToken;
        public Object user;
        public String role;

        public AuthResponse(String accessToken, String refreshToken, Object
user, String role) {
            this.accessToken = accessToken;
            this.refreshToken = refreshToken;
            this.user = user;
            this.role = role;
        }
    }
}
package tn.vermeg.gestionuser.controllers;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionuser.entities.Client;
import tn.vermeg.gestionuser.dto.ValidationResult;
import tn.vermeg.gestionuser.services.ClientService;
import java.util.List;

@RestController
@RequestMapping("/api/clients")
@CrossOrigin(origins = "http://localhost:4200")
public class ClientController {

    private final ClientService clientService;
    public ClientController(ClientService clientService) {this.clientService =
clientService;}

    @PostMapping("/register")
    public ResponseEntity<String> registerClient(@RequestBody Client client) {
        try {
            clientService.createClient(client);
            return ResponseEntity.ok("Client registered successfully!");
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("Registration failed: " + e.getMessage());
        }
    }

    @GetMapping("/getAllClients")
    public List<Client> getAllClients() {
        return clientService.getAllClients();
    }
```

```
    @GetMapping("getClientById/{idUser}")
    public Client getClientById(@PathVariable String idUser) {
        return clientService.getClientById(idUser);
    }

    @PutMapping("/{idUser}")
    public ResponseEntity<Client> updateClient(@PathVariable String idUser,
@RequestBody Client client) {
        Client updatedClient = clientService.updateClient(idUser, client);
        return ResponseEntity.ok(updatedClient);
    }

    @DeleteMapping("/{idUser}")
    public ResponseEntity<String> deleteClient(@PathVariable String idUser) {
        clientService.deleteClient(idUser);
        return ResponseEntity.ok("Client supprimé avec succès !");
    }

    @PostMapping("/validate")
    public ResponseEntity<ValidationResult> validateClient(@RequestBody Client
client) {
        ValidationResult result = clientService.validateClient(client);
        return ResponseEntity.ok(result);
    }
}
package tn.vermeg.gestionuser.controllers;
import org.springframework.http.ResponseEntity;
import tn.vermeg.gestionuser.entities.Admin;
import tn.vermeg.gestionuser.services.AdminService;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/admins")
@CrossOrigin(origins = "http://localhost:4200")
public class AdminController {
    private final AdminService adminService;
    public AdminController(AdminService adminService) {this.adminService =
adminService;}

    @PostMapping("/createAdmin")
    public Admin createAdmin(@RequestBody Admin admin) {
        return adminService.createAdmin(admin);
    }
    @GetMapping("/getAllAdmins")
    public List<Admin> getAllAdmins() {
        return adminService.getAllAdmins();
    }
    @GetMapping("getAdminById/{idUser}")
    public Admin getAdminById(@PathVariable String idUser) {
        return adminService.getAdminById(idUser);
    }
    @PutMapping("/{idUser}")
```

```
   public ResponseEntity<Admin> updateAdmin(@PathVariable String idUser,
@RequestBody Admin admin) {
       Admin updatedAdmin = adminService.updateAdmin(idUser, admin);
       return ResponseEntity.ok(updatedAdmin)      ;}
   @DeleteMapping("/{idUser}")
   public ResponseEntity<String> deleteAdmin(@PathVariable String idUser) {
       adminService.deleteAdmin(idUser);
       return ResponseEntity.ok("Admin supprimé avec succès !");
   }
}
package tn.vermeg.gestionuser.dto;

public class RegisterRequest {
   private String userName;
   private String password;
   private String email;
   private int phone;
   private String departement;
   public String getUserName() { return userName; }
   public void setUserName(String userName) { this.userName = userName; }
   public String getPassword() { return password; }
   public void setPassword(String password) { this.password = password; }
   public String getEmail() { return email; }
   public void setEmail(String email) { this.email = email; }
   public int getPhone() { return phone; }
   public void setPhone(int phone) { this.phone = phone; }
   public String getDepartement() { return departement; }
   public void setDepartement(String departement) { this.departement =
departement; }
}
package tn.vermeg.gestionuser.dto;
public class ValidationResult {
   private boolean isValid;
   private String message;
   public ValidationResult(boolean isValid, String message){
       this.isValid = isValid;
       this.message = message;}
   public ValidationResult() {}
   public boolean isValid() {
       return isValid;
   }
   public void setValid(boolean valid) {
       isValid = valid;
   }
   public String getMessage() {
       return message;
   }
   public void setMessage(String message) {
       this.message = message;
   }
}
package tn.vermeg.gestionuser.dto;
public class ClientResponseDTO {
```

```java
private String idUser;
private String userName;
private String email;
private Integer phone;
private Integer age;
private String sexe;
private String profession;
private String situationFamiliale;
private Boolean maladieChronique;
private Boolean diabetique;
private Boolean tension;
private Integer nombreBeneficiaires;
private Boolean actif;
private String numeroCIN;
// Constructeurs
public ClientResponseDTO() {}
// Getters et Setters
public String getIdUser() {
    return idUser;
}
public void setIdUser(String idUser) {
    this.idUser = idUser;
}
public String getUserName() {
    return userName;
}
public void setUserName(String userName) {
    this.userName = userName;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}
public Integer getPhone() {
    return phone;
}
public void setPhone(Integer phone) {
    this.phone = phone;
}
public Integer getAge() {
    return age;
}
public void setAge(Integer age) {
    this.age = age;
}
public String getSexe() {
    return sexe;
}
public void setSexe(String sexe) {
    this.sexe = sexe;
}
```

```java
    public String getProfession() {
        return profession;
    }
    public void setProfession(String profession) {
        this.profession = profession;
    }
    public String getSituationFamiliale() {
        return situationFamiliale;
    }
    public void setSituationFamiliale(String situationFamiliale) {
        this.situationFamiliale = situationFamiliale;
    }
    public Boolean isMaladieChronique() {
        return maladieChronique;
    }
    public void setMaladieChronique(Boolean maladieChronique) {
        this.maladieChronique = maladieChronique;
    }
    public Boolean isDiabetique() {
        return diabetique;
    }
    public void setDiabetique(Boolean diabetique) {
        this.diabetique = diabetique;
    }
    public Boolean isTension() {
        return tension;
    }
    public void setTension(Boolean tension) {
        this.tension = tension;
    }
    public Integer getNombreBeneficiaires() {
        return nombreBeneficiaires;
    }
    public void setNombreBeneficiaires(Integer nombreBeneficiaires) {
        this.nombreBeneficiaires = nombreBeneficiaires;
    }
    public Boolean getActif() {
        return actif;
    }
    public void setActif(Boolean actif) {
        this.actif = actif;
    }
    public String getNumeroCIN() {
        return numeroCIN;
    }
    public void setNumeroCIN(String numeroCIN) {
        this.numeroCIN = numeroCIN;
    }
}
package tn.vermeg.gestionuser.dto;

public class AuthRequest {
    private String userName;
```

```
    private String password;

    public String getUserName() {return userName;}
    public void setUserName(String userName) {this.userName = userName;}
    public String getPassword() {return password;}
    public void setPassword(String password) {this.password = password;}
}
package tn.vermeg.gestionuser.entities;
import jakarta.validation.constraints.*;
import org.springframework.data.annotation.TypeAlias;
import org.springframework.data.mongodb.core.mapping.Document;
@Document(collection = "clients")
@TypeAlias("client")
public class Client extends User {

    // Champs spécifiques à Client
    @Min(value = 0, message = "L'âge ne peut pas être négatif")
    @Max(value = 100, message = "L'âge ne peut pas dépasser 100 ans")
    private Integer age;
    @Pattern(regexp = "^(M|F|Homme|Femme)$", message = "Le sexe doit être M,
F, Homme ou Femme")
    private String sexe;
    private String profession;
    private String situationFamiliale;
    private Boolean maladieChronique = false;
    private Boolean diabetique = false;
    private Boolean tension = false;
    @Min(value = 1, message = "Le nombre de bénéficiaires doit être au moins
1")
    private Integer nombreBeneficiaires = 1;

    @Pattern(regexp = "^\\d{8}$", message = "Le CIN doit contenir 8 chiffres")
    private String numeroCIN;

    public Client() {}
    public Client(String idUser, String userName, String email, String
password, Integer phone,
                  Integer age, String sexe, String profession,
                  String situationFamiliale, Boolean maladieChronique,
                  Boolean diabetique, Boolean tension, Integer
nombreBeneficiaires, String numeroCIN) {
        super(idUser, userName, email, password, phone, Role.CLIENT, true,
new java.util.Date());
        this.age = age;
        this.sexe = sexe;
        this.profession = profession;
        this.situationFamiliale = situationFamiliale;
        this.maladieChronique = maladieChronique;
        this.diabetique = diabetique;
        this.tension = tension;
        this.nombreBeneficiaires = nombreBeneficiaires;
        this.numeroCIN = numeroCIN;
    }
```

```
    // Getters et Setters
    public Integer getAge() {
        return age;
    }
    public void setAge(Integer age) {
        this.age = age;
    }
    public String getSexe() {
        return sexe;
    }
    public void setSexe(String sexe) {this.sexe = sexe;}
    public String getProfession() {return profession;}
    public void setProfession(String profession) {this.profession =
profession;}
    public String getSituationFamiliale() {return situationFamiliale;}
    public void setSituationFamiliale(String situationFamiliale)
{this.situationFamiliale = situationFamiliale;}
    public Boolean isMaladieChronique() {return maladieChronique;}
    public void setMaladieChronique(Boolean maladieChronique)
{this.maladieChronique = maladieChronique;}
    public Boolean isDiabetique() {return diabetique;}
    public void setDiabetique(Boolean diabetique) {this.diabetique =
diabetique;}
    public Boolean isTension() {return tension;}
    public void setTension(Boolean tension) {this.tension = tension;}
    public Integer getNombreBeneficiaires() {return nombreBeneficiaires;}
    public void setNombreBeneficiaires(Integer nombreBeneficiaires)
{this.nombreBeneficiaires = nombreBeneficiaires;}
    public String getNumeroCIN() {return numeroCIN;}
    public void setNumeroCIN(String numeroCIN) {this.numeroCIN = numeroCIN;}
}
package tn.vermeg.gestionuser.entities;
import org.springframework.data.annotation.TypeAlias;
import org.springframework.data.mongodb.core.mapping.Document;

import java.util.Date;

@Document(collection = "admins")
@TypeAlias("admin")
public class Admin extends User {
    private String departement;

    public String getDepartement() {return departement;}

    public void setDepartement(String departement) {this.departement =
departement;}

    public Admin(String departement) {this.departement = departement;}

    public Admin() {}
```

```java
   public Admin(String idUser, String userName, String email, String
password, Integer phone, Role role, Boolean actif, Date dateCreation, String
departement) {
       super(idUser, userName, email, password, phone, role, actif,
dateCreation);
       this.departement = departement;
   }
}
package tn.vermeg.gestionuser.entities;

import jakarta.validation.constraints.*;
import org.springframework.data.annotation.TypeAlias;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "agents")
@TypeAlias("agent")
public class Agent extends User {

   @NotBlank(message = "Le service ne peut pas être vide")
   private String service;
   @NotNull(message = "Le grade ne peut pas être vide")
   private String grade;
   @Pattern(regexp = "^(M|F|Homme|Femme)$",message = "Le sexe doit être M, F,
Homme ou Femme")
   private String sexe;
   private Integer age;
   public Agent() {}
   public Agent(String idUser, String userName, String email, String
password, Integer phone,
                String service, String grade, String sexe, Integer age) {
       super(idUser, userName, email, password, phone, Role.AGENT, true, new
java.util.Date());
       this.service = service;
       this.grade = grade;
       this.sexe = sexe;
       this.age = age;}

   // Getters et Setters
   public String getService() { return service; }
   public void setService(String service) { this.service = service; }
   public String getGrade() { return grade; }
   public void setGrade(String grade) { this.grade = grade; }
   public String getSexe() { return sexe; }
   public void setSexe(String sexe) { this.sexe = sexe; }
   public Integer getAge() { return age; }
   public void setAge(Integer age) { this.age = age; }
}
package tn.vermeg.gestionuser.entities;

import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.annotation.TypeAlias;
```

```java
import org.springframework.data.mongodb.core.mapping.Document;

import java.util.Date;

@Document(collection = "users")
@TypeAlias("user")
public abstract class User {

   @Id
   private String idUser;
   private String userName;
   private String email;
   private String password;
   private Integer phone;
   private Role role;
   private Boolean actif;
   @CreatedDate
   private Date dateCreation;
   @LastModifiedDate
   private Date dateModification;

   public String getIdUser() { return idUser; }
   public void setIdUser(String idUser) { this.idUser = idUser; }
   public String getUserName() { return userName; }
   public void setUserName(String userName) { this.userName = userName; }
   public String getEmail() { return email; }
   public void setEmail(String email) { this.email = email; }
   public String getPassword() { return password; }
   public void setPassword(String password) { this.password = password; }
   public Integer getPhone() { return phone; }
   public void setPhone(Integer phone) { this.phone = phone; }
   public Role getRole() { return role; }
   public void setRole(Role role) { this.role = role; }
   public Boolean getActif() { return actif; }
   public void setActif(Boolean actif) { this.actif = actif; }
   public Date getDateCreation() { return dateCreation; }
   public void setDateCreation(Date dateCreation) { this.dateCreation =
dateCreation; }
   public Date getDateModification() { return dateModification; }
   public void setDateModification(Date dateModification) {
this.dateModification = dateModification; }

   public User() {}
   public User(String idUser, String userName, String email,
               String password, Integer phone, Role role,
               Boolean actif, Date dateCreation) {
       this.idUser = idUser;
       this.userName = userName;
       this.email = email;
       this.password = password;
       this.phone = phone;
       this.role = role;
       this.actif = actif;
```

```
        this.dateCreation = dateCreation;}
}
package tn.vermeg.gestionuser.entities;

public enum Role {ADMIN, CLIENT, AGENT}
package tn.vermeg.gestionuser.repositories;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionuser.entities.Client;
import java.util.List;
import java.util.Optional;

@Repository
public interface ClientRepository extends MongoRepository<Client, String> {
   Client findByUserName(String userName);
   List<Client> findByEmail(String email);
   List<Client>
findByUserNameContainingIgnoreCaseOrEmailContainingIgnoreCase(String
username, String email);
}
package tn.vermeg.gestionuser.repositories;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionuser.entities.Agent;
@Repository
public interface AgentRepository extends MongoRepository<Agent, String> {
   Agent findByUserName(String userName);}
package tn.vermeg.gestionuser.repositories;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionuser.entities.Admin;
@Repository
public interface AdminRepository extends MongoRepository<Admin, String> {
   Admin findByUserName(String userName); }
package tn.vermeg.gestionuser.security;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticatio
nFilter;
import org.springframework.security.config.Customizer;
import org.springframework.http.HttpMethod;
import tn.vermeg.gestionuser.config.JwtFilter;
```

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {
   private final JwtFilter jwtFilter;
   public SecurityConfig(JwtFilter jwtFilter) {
       this.jwtFilter = jwtFilter;
   }
   @Bean
   public PasswordEncoder passwordEncoder() {
       return new BCryptPasswordEncoder();
   }
   @Bean
   public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
       http
               .cors(Customizer.withDefaults())
               .csrf(csrf -> csrf.disable())
               .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
               .authorizeHttpRequests(auth -> auth
                       .requestMatchers(HttpMethod.OPTIONS,
"/**").permitAll()
                       .requestMatchers("/api/auth/**").permitAll()

.requestMatchers("/api/admins/createAdmin").permitAll()
                       .requestMatchers("/api/admins/**").hasRole("ADMIN")
                       .requestMatchers("/api/client/**").hasAnyRole("ADMIN",
"CLIENT")
                       .anyRequest().authenticated())
               .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);
       return http.build();}
}
package tn.vermeg.gestionuser.security;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;
@Service
public class PasswordService {

   private final BCryptPasswordEncoder passwordEncoder = new
BCryptPasswordEncoder();
    // Hash un mot de passe avec BCrypt
    // @param plainPassword Le mot de passe en clair + @return Le mot de
passe hashé
   public String hashPassword(String plainPassword) {
       return passwordEncoder.encode(plainPassword);}
   /**
    * Vérifie si un mot de passe correspond au hash
    * @param plainPassword Le mot de passe en clair + @param hashedPassword
Le mot de passe hashé
    * @return true si le mot de passe correspond
    */
```

```java
   public boolean verifyPassword(String plainPassword, String hashedPassword)
{
       return passwordEncoder.matches(plainPassword, hashedPassword);}
    // Génère un mot de passe temporaire + @return Un mot de passe
temporaire de 8 caractères
   public String generateTemporaryPassword() {
       String chars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
       StringBuilder password = new StringBuilder();
       for (int i = 0; i < 8; i++) {
           int index = (int) (Math.random() * chars.length());
           password.append(chars.charAt(index));}
       return password.toString();}
}
package tn.vermeg.gestionuser.services;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import tn.vermeg.gestionuser.dto.ValidationResult;
import tn.vermeg.gestionuser.entities.Client;
import tn.vermeg.gestionuser.repositories.ClientRepository;
import tn.vermeg.gestionuser.security.PasswordService;
import tn.vermeg.gestionuser.validation.UserValidator;
import java.util.List;
@Service
@Transactional
public class ClientService {
   private final ClientRepository clientRepository;
//    private final ClientMapper clientMapper;
   private final MailService mailService;
   private final PasswordService passwordService;

   public ClientService(ClientRepository clientRepository,
                        MailService mailService, PasswordService
passwordService) {
       this.clientRepository = clientRepository;
      // this.clientMapper = clientMapper;
       this.mailService = mailService;
       this.passwordService = passwordService;}

   public List<Client> getAllClients() {return clientRepository.findAll();}
   public Client getClientById(String idUser) {
       return clientRepository.findById(idUser)
               .orElseThrow(() -> new RuntimeException("Client non
trouvé"));}

   @Transactional
   public Client createClient(Client client) {
       // Valider les données
       validateClientData(client, true);
       // Vérifier les doublons
       checkForDuplicates(client, null);
       // Sauvegarder le mot de passe original pour l'email
```

```
        String originalPassword = client.getPassword();
        // Hashage du mot de passe

client.setPassword(passwordService.hashPassword(client.getPassword()));
        Client savedClient = clientRepository.save(client);
        // Envoyer l'email de bienvenue avec le mot de passe original
        sendWelcomeEmail(savedClient, originalPassword);
        return savedClient;}

    @Transactional
    public Client updateClient(String idUser, Client client) {
        // Valider les données
        validateClientData(client, false);
        Client existingClient = clientRepository.findById(idUser)
                .orElseThrow(() -> new RuntimeException("Client non trouvé"));
        // Vérifier les doublons (en excluant le client actuel)
        checkForDuplicates(client, idUser);

        // Sauvegarder le mot de passe pour l'email
        String passwordForEmail = null;
        if (client.getPassword() != null &&
!client.getPassword().trim().isEmpty()) {
            passwordForEmail = client.getPassword();
        }
        // Mise à jour des champs
        existingClient.setUserName(client.getUserName());
        existingClient.setEmail(client.getEmail());
        existingClient.setPhone(client.getPhone());
        existingClient.setAge(client.getAge());
        existingClient.setSexe(client.getSexe());
        existingClient.setProfession(client.getProfession());
        existingClient.setSituationFamiliale(client.getSituationFamiliale());
        existingClient.setMaladieChronique(client.isMaladieChronique());
        existingClient.setDiabetique(client.isDiabetique());
        existingClient.setTension(client.isTension());

existingClient.setNombreBeneficiaires(client.getNombreBeneficiaires());
        // Hasher le nouveau mot de passe si fourni
        if (client.getPassword() != null &&
!client.getPassword().trim().isEmpty()) {

existingClient.setPassword(passwordService.hashPassword(client.getPassword())
);}
        Client updatedClient = clientRepository.save(existingClient);
        // Envoyer l'email de mise à jour avec le mot de passe
        sendUpdateEmail(updatedClient, passwordForEmail);
        return updatedClient;}
    @Transactional
    public void deleteClient(String idUser) {
        Client client = clientRepository.findById(idUser)
                .orElseThrow(() -> new RuntimeException("Client non trouvé"));
        clientRepository.delete(client);
        // Envoyer l'email de suppression
```

```
        sendDeletionEmail(client);}

    public ValidationResult validateUsername(String username) {
        String error = UserValidator.validateUsername(username);
        if (error != null) {
            return new ValidationResult(false, error);}

        // Vérifier si le username est déjà utilisé
        Client existingClient = clientRepository.findByUserName(username);
        if (existingClient != null) {
            return new ValidationResult(false, "Ce username est déjà
utilisé");}
        return new ValidationResult(true, "Username valide");}

    public ValidationResult validateEmail(String email) {
        String error = UserValidator.validateEmail(email);
        if (error != null) {
            return new ValidationResult(false, error);}

        // Plus de vérification d'unicité - les emails peuvent être
dupliqués
        return new ValidationResult(true, "Email valide");}

    public List<Client> searchClients(String query) {
        return
clientRepository.findByUserNameContainingIgnoreCaseOrEmailContainingIgnoreCas
e(query, query);}

    private void validateClientData(Client client, boolean isCreation) {
        // Validation du username
        if (client.getUserName() == null ||
client.getUserName().trim().isEmpty()) {
            throw new IllegalArgumentException("Le username ne peut pas être
vide");
        }
        if (!client.getUserName().matches("^[a-zA-Z]+(-[a-zA-Z]+)*$")) {
            throw new IllegalArgumentException("Le username doit être un seul
mot ou des mots séparés uniquement par des tirets (ex: jean, jean-marc)");
        }
        if (client.getUserName().length() < 3 || client.getUserName().length()
> 30) {
            throw new IllegalArgumentException("Le username doit contenir
entre 3 et 30 caractères");
        }

        // Validation du mot de passe (uniquement en création ou si modifié)
        if (isCreation || (client.getPassword() != null &&
!client.getPassword().trim().isEmpty())) {
            if (client.getPassword() == null || client.getPassword().length()
< 6) {
                throw new IllegalArgumentException("Le mot de passe doit
contenir au moins 6 caractères");}
```

```
            if
(!client.getPassword().matches("^(?=.*[a-zA-Z])(?=.*[0-9]).+$")) {
                throw new IllegalArgumentException("Le mot de passe doit
contenir au moins une lettre et un chiffre");}
        }
        // Validation de l'email
        if (client.getEmail() == null || client.getEmail().trim().isEmpty())
{
            throw new IllegalArgumentException("L'email ne peut pas être
vide");}
        if (!client.getEmail().matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            throw new IllegalArgumentException("L'email doit avoir un format
valide");}

        // Validation du téléphone
        if (client.getPhone() == null) {
            throw new IllegalArgumentException("Le téléphone ne peut pas être
vide");}
        if (client.getPhone() < 20000000 || client.getPhone() > 99999999) {
            throw new IllegalArgumentException("Le numéro de téléphone doit
être valide");}
        // Validation de l'âge
        if (client.getAge() != null && (client.getAge() < 0 ||
client.getAge() > 150)) {
            throw new IllegalArgumentException("L'âge doit être entre 0 et 150
ans");}
        // Validation du sexe
        if (client.getSexe() != null &&
!client.getSexe().matches("^(M|F|Homme|Femme)$")) {
            throw new IllegalArgumentException("Le sexe doit être M, F, Homme
ou Femme");}
        // Validation du nombre de bénéficiaires
        if (client.getNombreBeneficiaires() != null &&
client.getNombreBeneficiaires() < 1) {
            throw new IllegalArgumentException("Le nombre de bénéficiaires
doit être au moins 1");}
    }
    private void checkForDuplicates(Client client, String excludeId) {
        // Vérifier uniquement le username (l'email peut être dupliqué)
        Client existingUsernameClient =
clientRepository.findByUserName(client.getUserName());
        if (existingUsernameClient != null &&
!existingUsernameClient.getIdUser().equals(excludeId)) {
            throw new RuntimeException("Ce username est déjà utilisé");}}
    private void sendWelcomeEmail(Client client, String originalPassword) {
        String subject = "VOS COORDONNÉES DE CONNEXION - Compte Client";
        String body = "Bonjour " + client.getUserName() + ",\n\n" +
                "Votre compte client a été créé avec succès.\n\n" +
            //    "=== VOS COORDONNÉES DE CONNEXION ===\n" +
                "Type de compte : CLIENT\n" +
                "Nom d'utilisateur : " + client.getUserName() + "\n" +
                "Email : " + client.getEmail() + "\n" +
                "Mot de passe : " + originalPassword + "\n\n" +
```

```
                "Téléphone : " + client.getPhone() + "\n" +
            //    "=== VOS INFORMATIONS PERSONNELLES ===\n" +
//                  "Âge : " + (client.getAge() != null ? client.getAge() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (client.getSexe() != null ? client.getSexe()
: "Non spécifié") + "\n" +
//                  "Profession : " + (client.getProfession() != null ?
client.getProfession() : "Non spécifié") + "\n" +
//                  "Situation familiale : " +
(client.getSituationFamiliale() != null ? client.getSituationFamiliale() :
"Non spécifié") + "\n" +
//                  "Nombre de bénéficiaires : " +
client.getNombreBeneficiaires() + "\n" +
//                  "Maladie chronique : " + (client.isMaladieChronique() ?
"Oui" : "Non") + "\n" +
//                  "Diabétique : " + (client.isDiabetique() ? "Oui" : "Non")
+ "\n" +
//                  "Tension : " + (client.isTension() ? "Oui" : "Non") +
"\n\n" +
//                  "=== INSTRUCTIONS DE CONNEXION ===\n" +
                "1. Connectez-vous avec votre nom d'utilisateur et mot de
passe\n" +
                "2. Gardez ces informations confidentielles\n" +
                "3. Changez votre mot de passe lors de la première
connexion\n\n" +
                "Cordialement,\nL'équipe Vermeg.";

        mailService.sendEmail(client.getEmail(), subject, body);
    }
    private void sendUpdateEmail(Client client, String passwordForEmail) {
        String subject = "MISE À JOUR - Vos coordonnées de connexion";
        String body = "Bonjour " + client.getUserName() + ",\n\n" +
                "Votre compte client a été mis à jour avec succès.\n\n" +
               // "=== VOS COORDONNÉES ACTUELLES ===\n" +
                "Type de compte : CLIENT\n" +
                "Nom d'utilisateur : " + client.getUserName() + "\n" +
                "Email : " + client.getEmail() + "\n" +
                "Téléphone : " + client.getPhone() + "\n" +
                (passwordForEmail != null ? "Mot de passe : " +
passwordForEmail + "\n" : "Mot de passe : Inchangé\n") +
            //    "\n=== VOS INFORMATIONS PERSONNELLES ===\n" +
//                  "Âge : " + (client.getAge() != null ? client.getAge() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (client.getSexe() != null ? client.getSexe()
: "Non spécifié") + "\n" +
//                  "Profession : " + (client.getProfession() != null ?
client.getProfession() : "Non spécifié") + "\n" +
//                  "Situation familiale : " +
(client.getSituationFamiliale() != null ? client.getSituationFamiliale() :
"Non spécifié") + "\n" +
//                  "Nombre de bénéficiaires : " +
client.getNombreBeneficiaires() + "\n" +
```

```
//                  "Maladie chronique : " + (client.isMaladieChronique() ?
"Oui" : "Non") + "\n" +
//                  "Diabétique : " + (client.isDiabetique() ? "Oui" : "Non")
+ "\n" +
//                  "Tension : " + (client.isTension() ? "Oui" : "Non") +
"\n\n" +
                "Si vous n'êtes pas à l'origine de cette modification,
contactez-nous immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";

        mailService.sendEmail(client.getEmail(), subject, body);
    }
    private void sendDeletionEmail(Client client) {
        String subject = "SUPPRESSION - Compte Client Vermeg";
        String body = "Bonjour " + client.getUserName() + ",\n\n" +
                "Votre compte client a été supprimé avec succès.\n\n" +
               //  "=== COORDONNÉES DU COMPTE SUPPRIMÉ ===\n" +
                "Type de compte : CLIENT\n" +
                "Nom d'utilisateur : " + client.getUserName() + "\n" +
                "Email : " + client.getEmail() + "\n" +
                "Mot de passe : [Non disponible - compte supprimé]\n\n" +
                "Téléphone : " + client.getPhone() + "\n" +
//                  "=== INFORMATIONS PERSONNELLES ===\n" +
//                  "Âge : " + (client.getAge() != null ? client.getAge() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (client.getSexe() != null ? client.getSexe()
: "Non spécifié") + "\n" +
//                  "Profession : " + (client.getProfession() != null ?
client.getProfession() : "Non spécifié") + "\n" +
//                  "Situation familiale : " +
(client.getSituationFamiliale() != null ? client.getSituationFamiliale() :
"Non spécifié") + "\n" +
              //  "Nombre de bénéficiaires : " +
client.getNombreBeneficiaires() + "\n\n" +
                "Si vous n'êtes pas à l'origine de cette suppression,
veuillez nous contacter immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";

        mailService.sendEmail(client.getEmail(), subject, body);
    }

    public ValidationResult validateClient(Client client) {
        try {
            validateClientData(client, false);
            return new ValidationResult(true, "Client valide");
        } catch (IllegalArgumentException e) {
            return new ValidationResult(false, e.getMessage());
        }
    }
}
package tn.vermeg.gestionuser.services;
import org.springframework.stereotype.Service;
import tn.vermeg.gestionuser.entities.Admin;
```

```java
import tn.vermeg.gestionuser.repositories.AdminRepository;
import tn.vermeg.gestionuser.security.PasswordService;
import java.util.List;

@Service
public class AdminService {
   private final AdminRepository adminRepository;
   private final MailService mailService;
   private final PasswordService passwordService;

   public AdminService(AdminRepository adminRepository, MailService
mailService, PasswordService passwordService) {
       this.adminRepository = adminRepository;
       this.mailService = mailService;
       this.passwordService = passwordService;}

   // Création d'un admin
  // create Admin+ envoi par mail
   public Admin createAdmin(Admin admin) {
       // Sauvegarder le mot de passe original pour l'email
       String originalPassword = admin.getPassword();
       // Hasher le mot de passe avant de sauvegarder
       admin.setPassword(passwordService.hashPassword(admin.getPassword()));
       Admin savedAdmin = adminRepository.save(admin);
       // Envoyer l'email avec toutes les coordonnées
       sendWelcomeEmail(savedAdmin, originalPassword);
       return savedAdmin;}

   // Récupération de tous les admins
   public List<Admin> getAllAdmins() {return adminRepository.findAll();}
   // Récupération par ID
   public Admin getAdminById(String idUser) {
       return adminRepository.findById(idUser).orElseThrow(() -> new
RuntimeException("Admin not found"));}

   // Mise à jour d'un admin
   public Admin updateAdmin(String idUser, Admin updatedAdmin) {
       Admin admin = getAdminById(idUser);

       // Sauvegarder le mot de passe pour l'email
       String passwordForEmail = null;
       if (updatedAdmin.getPassword() != null &&
!updatedAdmin.getPassword().trim().isEmpty()) {
           passwordForEmail = updatedAdmin.getPassword();}
       admin.setUserName(updatedAdmin.getUserName());
       admin.setEmail(updatedAdmin.getEmail());
       admin.setPassword(updatedAdmin.getPassword());
       admin.setPhone(updatedAdmin.getPhone());
       //admin.setDepartment(updatedAdmin.getDepartment());
       //admin.setActive(updatedAdmin.getActive());
       Admin savedAdmin = adminRepository.save(admin);

       // Envoyer l'email de mise à jour avec le mot de passe
```

```
        sendUpdateEmail(savedAdmin, passwordForEmail);
        return savedAdmin;
    }
    // Suppression
    public void deleteAdmin(String idUser) {
        Admin admin = getAdminById(idUser);
        adminRepository.delete(admin);
        // Envoyer l'email de suppression
        sendDeletionEmail(admin);
    }

    private void sendWelcomeEmail(Admin admin, String originalPassword) {
        String subject = "VOS COORDONNÉES DE CONNEXION - Compte
Administrateur";
        String body = "Bonjour " + admin.getUserName() + ",\n\n" +
                "Votre compte administrateur a été créé avec succès.\n\n" +
               //  "=== VOS COORDONNÉES DE CONNEXION ===\n" +
                "Type de compte : ADMINISTRATEUR\n" +
                "Nom d'utilisateur : " + admin.getUserName() + "\n" +
                "Email : " + admin.getEmail() + "\n" +
                "Téléphone : " + admin.getPhone() + "\n" +
                "Mot de passe : " + originalPassword + "\n" +
            //    "=== VOS INFORMATIONS PROFESSIONNELLES ===\n" +
                "Département : " + (admin.getDepartement() != null ?
admin.getDepartement().toString() : "Non spécifié") + "\n\n" +
               //  "=== INSTRUCTIONS DE CONNEXION ===\n" +
                "1. Connectez-vous avec votre nom d'utilisateur et mot de
passe\n" +
                "2. Gardez ces informations confidentielles\n" +
                "3. Changez votre mot de passe lors de la première
connexion\n" +
                "4. Vous avez accès aux fonctionnalités d'administration\n\n"
+
                "Cordialement,\nL'équipe Vermeg.";

        mailService.sendEmail(admin.getEmail(), subject, body);
    }

    private void sendUpdateEmail(Admin admin, String passwordForEmail) {
        String subject = "MISE À JOUR - Vos coordonnées Administrateur";
        String body = "Bonjour " + admin.getUserName() + ",\n\n" +
                "Votre compte administrateur a été mis à jour avec
succès.\n\n" +
            //   "=== VOS COORDONNÉES ACTUELLES ===\n" +
                "Type de compte : ADMINISTRATEUR\n" +
                "Nom d'utilisateur : " + admin.getUserName() + "\n" +
                "Téléphone : " + admin.getPhone() + "\n" +
                "Email : " + admin.getEmail() + "\n" +
                "Mot de passe : " + admin.getPassword() + "\n" +
                (passwordForEmail != null ? "Mot de passe : " +
passwordForEmail + "\n" : "Mot de passe : Inchangé\n") +
           //     "\n=== VOS INFORMATIONS PROFESSIONNELLES ===\n" +
```

```
                "Département : " + (admin.getDepartement() != null ?
admin.getDepartement().toString() : "Non spécifié") + "\n\n" +
                "Si vous n'êtes pas à l'origine de cette modification,
contactez-nous immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";
        mailService.sendEmail(admin.getEmail(), subject, body);}

    private void sendDeletionEmail(Admin admin) {
        String subject = "SUPPRESSION - Compte Administrateur Vermeg";
        String body = "Bonjour " + admin.getUserName() + ",\n\n" +
                "Votre compte administrateur a été supprimé avec succès.\n\n"
+
               //  "=== COORDONNÉES DU COMPTE SUPPRIMÉ ===\n" +
                "Type de compte : ADMINISTRATEUR\n" +
                "Nom d'utilisateur : " + admin.getUserName() + "\n" +
                "Email : " + admin.getEmail() + "\n" +
                "Téléphone : " + admin.getPhone() + "\n" +
                "Mot de passe : [Non disponible - compte supprimé]\n" +
            //    "=== INFORMATIONS PROFESSIONNELLES ===\n" +
                "Département : " + (admin.getDepartement() != null ?
admin.getDepartement().toString() : "Non spécifié") + "\n\n" +
                "Si vous n'êtes pas à l'origine de cette suppression, veuillez
nous contacter immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";
        mailService.sendEmail(admin.getEmail(), subject, body);}
}
package tn.vermeg.gestionuser.services;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;
@Service
public class MailService {
    private final JavaMailSender mailSender;
    @Value("${spring.mail.username}")
    private String fromEmail;
    public MailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    public void sendEmail(String to, String subject, String body) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom(fromEmail);
        message.setTo(to);
        message.setSubject(subject);
        message.setText(body);
        try {
            mailSender.send(message);
            System.out.println("Email envoyé avec succès à " + to);} catch
(Exception e) {
            System.err.println("Erreur lors de l'envoi de l'email : " +
e.getMessage());}}
    public void sendCredentials(String to, String username, String password) {
```

```java
        String subject = "Vos identifiants de connexion - Assurance Vermeg";
        String text = "Bonjour,\n\n" +
                "Votre compte a été créé par l'administrateur.\n" +
                "Voici vos identifiants :\n" +
                "Nom d'utilisateur : " + username + "\n" +
                "Mot de passe : " + password + "\n" +
                "Cordialement,\n" +
                "L'équipe Vermeg.";
        sendEmail(to, subject, text);}
}
package tn.vermeg.gestionuser.services;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import tn.vermeg.gestionuser.entities.Agent;
import tn.vermeg.gestionuser.repositories.AgentRepository;
import tn.vermeg.gestionuser.security.PasswordService;

import java.util.List;

@Service
@Transactional
public class AgentService {
   private final AgentRepository agentRepository;
   private final MailService mailService;
   private final PasswordService passwordService;

   public AgentService(AgentRepository agentRepository, MailService
mailService, PasswordService passwordService) {
        this.agentRepository = agentRepository;
        this.mailService = mailService;
        this.passwordService = passwordService;
   }

   public List<Agent> getAllAgents() {
        return agentRepository.findAll();
   }

   public Agent getAgentById(String idUser) {
        return agentRepository.findById(idUser)
                .orElseThrow(() -> new RuntimeException("Agent non trouvé"));
   }

   @Transactional
   public Agent createAgent(Agent agent) {
        // Sauvegarder le mot de passe original pour l'email
        String originalPassword = agent.getPassword();
        // Hashage du mot de passe
        agent.setPassword(passwordService.hashPassword(agent.getPassword()));
        Agent savedAgent = agentRepository.save(agent);
        // Envoyer l'email de bienvenue avec le mot de passe original
        sendWelcomeEmail(savedAgent, originalPassword);
        return savedAgent;
```

```
    }

    @Transactional
    public Agent updateAgent(String idUser, Agent agent) {
        Agent existingAgent = agentRepository.findById(idUser)
                .orElseThrow(() -> new RuntimeException("Agent non trouvé"));

        // Sauvegarder le mot de passe pour l'email
        String passwordForEmail = null;
        if (agent.getPassword() != null &&
!agent.getPassword().trim().isEmpty()) {
            passwordForEmail = agent.getPassword();
        }

        // Mise à jour des champs
        existingAgent.setUserName(agent.getUserName());
        existingAgent.setEmail(agent.getEmail());
        existingAgent.setPhone(agent.getPhone());
        existingAgent.setService(agent.getService());
        existingAgent.setGrade(agent.getGrade());
        existingAgent.setSexe(agent.getSexe());
        existingAgent.setAge(agent.getAge());

        // Hasher le nouveau mot de passe si fourni
        if (agent.getPassword() != null &&
!agent.getPassword().trim().isEmpty()) {

existingAgent.setPassword(passwordService.hashPassword(agent.getPassword()));
        }

        Agent updatedAgent = agentRepository.save(existingAgent);
        // Envoyer l'email de mise à jour avec le mot de passe
        sendUpdateEmail(updatedAgent, passwordForEmail);
        return updatedAgent;
    }

    @Transactional
    public void deleteAgent(String idUser) {
        Agent agent = agentRepository.findById(idUser)
                .orElseThrow(() -> new RuntimeException("Agent non trouvé"));

        agentRepository.delete(agent);
        // Envoyer l'email de suppression
        sendDeletionEmail(agent);
    }

    private void sendWelcomeEmail(Agent agent, String originalPassword) {
        String subject = "VOS COORDONNÉES DE CONNEXION - Compte Agent";
        String body = "Bonjour " + agent.getUserName() + ",\n\n" +
                "Votre compte agent a été créé avec succès.\n\n" +
               //  "=== VOS COORDONNÉES DE CONNEXION ===\n" +
                "Type de compte : AGENT\n" +
                "Nom d'utilisateur : " + agent.getUserName() + "\n" +
```

```
                "Email : " + agent.getEmail() + "\n" +
                "Téléphone : " + agent.getPhone() + "\n" +
                "Mot de passe : " + originalPassword + "\n" +
            //     "=== VOS INFORMATIONS PROFESSIONNELLES ===\n" +
                "Service : " + (agent.getService() != null ?
agent.getService() : "Non spécifié") + "\n" +
                "Grade : " + (agent.getGrade() != null ? agent.getGrade() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (agent.getSexe() != null ? agent.getSexe() :
"Non spécifié") + "\n" +
//                  "Âge : " + (agent.getAge() != null ? agent.getAge() :
"Non spécifié") + "\n\n" +
//                  "=== INSTRUCTIONS DE CONNEXION ===\n" +
                "1. Connectez-vous avec votre nom d'utilisateur et mot de
passe\n" +
                "2. Gardez ces informations confidentielles\n" +
                "3. Changez votre mot de passe lors de la première
connexion\n" +
                "4. Vous avez accès aux fonctionnalités d'agent\n\n" +
                "Cordialement,\nL'équipe Vermeg.";
        mailService.sendEmail(agent.getEmail(), subject, body);
    }

    private void sendUpdateEmail(Agent agent, String passwordForEmail) {
        String subject = "MISE À JOUR - Vos coordonnées Agent";
        String body = "Bonjour " + agent.getUserName() + ",\n\n" +
                "Votre compte agent a été mis à jour avec succès.\n\n" +
               //  "=== VOS COORDONNÉES ACTUELLES ===\n" +
                "Type de compte : AGENT\n" +
                "Nom d'utilisateur : " + agent.getUserName() + "\n" +
                "Email : " + agent.getEmail() + "\n" +
                "Téléphone : " + agent.getPhone() + "\n" +
                (passwordForEmail != null ? "Mot de passe : " +
passwordForEmail + "\n" : "Mot de passe : Inchangé\n") +
            //   "\n=== VOS INFORMATIONS PROFESSIONNELLES ===\n" +
                "Service : " + (agent.getService() != null ?
agent.getService() : "Non spécifié") + "\n" +
                "Grade : " + (agent.getGrade() != null ? agent.getGrade() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (agent.getSexe() != null ? agent.getSexe() :
"Non spécifié") + "\n" +
//                  "Âge : " + (agent.getAge() != null ? agent.getAge() :
"Non spécifié") + "\n\n" +
                "Si vous n'êtes pas à l'origine de cette modification,
contactez-nous immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";
        mailService.sendEmail(agent.getEmail(), subject, body);
    }
    private void sendDeletionEmail(Agent agent) {
        String subject = "SUPPRESSION - Compte Agent Vermeg";
        String body = "Bonjour " + agent.getUserName() + ",\n\n" +
                "Votre compte agent a été supprimé avec succès.\n\n" +
               //  "=== COORDONNÉES DU COMPTE SUPPRIMÉ ===\n" +
```

```
                "Type de compte : AGENT\n" +
                "Nom d'utilisateur : " + agent.getUserName() + "\n" +
                "Email : " + agent.getEmail() + "\n" +
                "Téléphone : " + agent.getPhone() + "\n" +
                "Mot de passe : [Non disponible - compte supprimé]\n" +
              //  "=== INFORMATIONS PROFESSIONNELLES ===\n" +
                "Service : " + (agent.getService() != null ?
agent.getService() : "Non spécifié") + "\n" +
                "Grade : " + (agent.getGrade() != null ? agent.getGrade() :
"Non spécifié") + "\n" +
//                  "Sexe : " + (agent.getSexe() != null ? agent.getSexe() :
"Non spécifié") + "\n" +
//                  "Âge : " + (agent.getAge() != null ? agent.getAge() :
"Non spécifié") + "\n\n" +
                "Si vous n'êtes pas à l'origine de cette suppression,
veuillez nous contacter immédiatement.\n\n" +
                "Cordialement,\nL'équipe Vermeg.";
        mailService.sendEmail(agent.getEmail(), subject, body);
    }
}
package tn.vermeg.gestionuser.utils;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import java.security.Key;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;
import java.nio.charset.StandardCharsets;

public class JwtUtil {
    // Clé secrète robuste (Min 64 caractères recommandés pour HS512)
    private static final String SECRET =
"InsuranceProjectVermegSecretKeyForJWTAuth2026SecurityOverhaulStandard";
    private static final Key SECRET_KEY =
Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));

    // Durée de vie des tokens
    private static final long ACCESS_TOKEN_VALIDITY = 1000 * 60 * 60; // 1
heure
    private static final long REFRESH_TOKEN_VALIDITY = 1000 * 60 * 60 * 24 *
7; // 7 jours

    public static String generateAccessToken(String userName, String role) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
        return createToken(claims, userName, ACCESS_TOKEN_VALIDITY);}

    public static String generateRefreshToken(String userName) {
```

```
        return createToken(new HashMap<>(), userName,
REFRESH_TOKEN_VALIDITY);}

    private static String createToken(Map<String, Object> claims, String
subject, long validity) {
        return Jwts.builder()
                .setClaims(claims)
                .setSubject(subject)
                .setIssuedAt(new Date(System.currentTimeMillis()))
                .setExpiration(new Date(System.currentTimeMillis() +
validity))
                .signWith(SECRET_KEY, SignatureAlgorithm.HS256)
                .compact();}

    public static Boolean validateToken(String token, String userName) {
        final String username = extractUsername(token);
        return (username.equals(userName) && !isTokenExpired(token));}

    public static String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
    public static String extractRole(String token) {
        final Claims claims = extractAllClaims(token);
        return (String) claims.get("role");}
    public static Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public static <T> T extractClaim(String token, Function<Claims, T>
claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);}
    private static Claims extractAllClaims(String token) {
        return
Jwts.parserBuilder().setSigningKey(SECRET_KEY).build().parseClaimsJws(token)
.getBody();}
    private static Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }
}
package tn.vermeg.gestionuser.validation;
import java.util.regex.Pattern;

public class UserValidator {
    // Pattern pour username: un seul mot ou mots séparés par des tirets
uniquement
    private static final Pattern USERNAME_PATTERN =
Pattern.compile("^[a-zA-Z]+(-[a-zA-Z]+)*$");
    // Pattern pour mot de passe: minimum 6 caractères
    private static final int MIN_PASSWORD_LENGTH = 6;
     // Valide le format du username
     //@param username Le username à valider + @return message d'erreur si
invalide, null si valide
```

```java
   public static String validateUsername(String username) {
       if (username == null || username.trim().isEmpty()) {
           return "Le username ne peut pas être vide";}
       String trimmedUsername = username.trim();
       // Vérifier la longueur
       if (trimmedUsername.length() < 3) {
           return "Le username doit contenir au moins 3 caractères";}
       if (trimmedUsername.length() > 30) {
           return "Le username ne peut pas dépasser 20 caractères";}
       // Vérifier le format (un seul mot ou mots séparés par des tirets)
       if (!USERNAME_PATTERN.matcher(trimmedUsername).matches()) {
           return "Le username doit être un seul mot ou des mots séparés
uniquement par des tirets (ex: jean, jean-marc)";}
       return null; // Valide
        }
    // Valide le format du mot de passe
    //@param password Le mot de passe à valider + @return message d'erreur
si invalide, null si valide
   public static String validatePassword(String password) {
       if (password == null || password.trim().isEmpty()) {
           return "Le mot de passe ne peut pas être vide";}

       // Vérifier la longueur minimale
       if (password.length() < MIN_PASSWORD_LENGTH) {
           return "Le mot de passe doit contenir au moins " +
MIN_PASSWORD_LENGTH + " caractères";}
       // Vérifier qu'il ne contient pas d'espaces
       if (password.contains(" ")) {
           return "Le mot de passe ne peut pas contenir d'espaces";}
       // Validation supplémentaire (optionnelle)
       if (!password.matches(".*[a-zA-Z].*")) {
           return "Le mot de passe doit contenir au moins une lettre";}
       if (!password.matches(".*[0-9].*")) {
           return "Le mot de passe doit contenir au moins un chiffre";}
       return null; // Valide
   }
    // Valide le format de l'email ,
    // @param email L'email à valider + @return message d'erreur si
invalide, null si valide
   public static String validateEmail(String email) {
       if (email == null || email.trim().isEmpty()) {
           return "L'email ne peut pas être vide";}
       String trimmedEmail = email.trim();
       // Pattern email simple
       if
(!trimmedEmail.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
           return "L'email n'a pas un format valide";}
       return null; // Valide
   }

   // Valide le numéro de téléphone
    // @param phone Le numéro de téléphone à valider + @return message
d'erreur si invalide, null si valide
```

```
    public static String validatePhone(Integer phone) {
        if (phone == null) {return "Le numéro de téléphone ne peut pas être
vide";}
        String phoneStr = phone.toString();
        // Vérifier que c'est un nombre positif
        if (phone < 0) {return "Le numéro de téléphone doit être positif";}
        // Vérifier la longueur (Tunisie: 8 chiffres)
        if (phoneStr.length() != 8) {return "Le numéro de téléphone doit
contenir 8 chiffres";}
        // Vérifier qu'il commence par les bons préfixes tunisiens
        if (!phoneStr.matches("^[2-9]\\d{7}$")) {return "Le numéro de
téléphone doit commencer par 2, 4 , 5 ou 9";}
        return null; // Valide
    }
}
spring.application.name=GestionUser
server.port=9092
logging.level.tn.vermeg=TRACE
logging.level.org.springframework.data.mongodb=DEBUG
spring.web.error.include-stacktrace=always
spring.web.error.include-message=always
# JWT properties
jwt.secret=4b38e62a241cb93514e8b1069cdfe1d8297c838f196388b535c06433b7ca5652
#jwt.secret=ec2b47213a74800db3e2c456a6dc9e3edd05f2f22dae84e3e9bcbd9c5e78132
ce98de943d8d5e65105d091ac0553be9635238a6d2330fe1fa95b570ba72cd08c373278eeeb
f49c2513701b984ae529f56d4712b85ec52378e39122ece6a12875355a3948f5737785757ae
c8ebc2b952ded15afa18c7cc913deb10383281e26538322777ab7bc8e489a992244955cc603
0f0ccf15bf977c0b01d180ea5296815faa2b81b1c1c2b932928144e7c27f2d0abcab31a8fc8
f3579de0a00c0c8e7afdaecad38b9b8724ccdf619953f34ea1e935a96fbbfb2b697fdc4e78a
684c1a8656160e50cf4c38321fa990f19dc8f0db0ecabc84c74d1051f0e6c781fa1f714ca18
207f947aa491b24a33f52979ffc2726ffe3c2aac6fb87c30a383e0cd07b394da15bb3ae7b24
34dc648afd7105c73c401daad05f56f9f9cca8f878d96161f5c205c445cc8f90435988a14b2
5d8555899aea49c306c6fc4046c848220f4b94714e36e10dc4c720d2988d93bcdaa3f1b68c8
c2ee4b3e5234180bf139cfa5aade3614a260521d4ca4b38b766d17b47d8a42b8535950fa052
c5a4a643a6166f4004ac5a1b97151423924524595 4ed1d8dfe24735a7bc954bc1f5bf5144c0
cc54f2fbdf70e6a2fb59d1c9bcd87d1c4b2276c88970243d5f78a589a5d6685e1e1e5160fa4
ceb2f50573dec2a7541a24a977420c59b99f8403115b2980f47b6dd9fdd39
jwt.expiration=86400000
# CORS configuration for Angular
spring.web.cors.allowed-origins=http://localhost:4200
spring.web.cors.allowed-methods=GET,POST,DELETE,PUT,OPTIONS
spring.web.cors.allowed-headers=*
spring.web.cors.max-age=3600
spring.web.cors.allow-credentials=true
# Logging
logging.level.org.springframework.security=DEBUG
logging.level.org.springframework=DEBUG
logging.level.org.hibernate=DEBUG
logging.level.com.zaxxer.hikari=DEBUG
# Eureka registration
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.register-with-eureka=true
# Mail configuration
```

```
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=benabdeljamyla@gmail.com
spring.mail.password=nkrv szvw uvyk yrqk
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
# Disable Spring Cloud Config (important!)
spring.cloud.config.enabled=false
# MongoDB configuration
#spring.mongodb.database=gestionuser
#spring.mongodb.host=localhost
#spring.mongodb.port=27017
```

```java
package tn.vermeg.gestionproduit.entities;

import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;

@Document(collection = "produits")
public class Produit {

   @Id
   private String idProduit;

   private String nomProduit;
   private String description;

   private TypeProduit typeProduit;
   private Statut statut;

   @CreatedDate
   private Instant dateCreation;

   @LastModifiedDate
   private Instant dateModification;

   // CONSTRUCTEURS

   public Produit() {}

   public Produit(String idProduit, String nomProduit, String description,
                  TypeProduit typeProduit, Statut statut,
                  Instant dateCreation, Instant dateModification) {

       this.idProduit = idProduit;
       this.nomProduit = nomProduit;
       this.description = description;
```

```java
        this.typeProduit = typeProduit;
        this.statut = statut;
        this.dateCreation = dateCreation;
        this.dateModification = dateModification;
    }

    // MÉTHODES MÉTIER

    public boolean estActif() {
        return statut == Statut.ACTIF;
    }

    public boolean estValide() {
        return nomProduit != null && !nomProduit.isBlank()
                && typeProduit != null
                && statut != null;
    }

    // GETTERS & SETTERS

    public String getIdProduit() { return idProduit; }
    public void setIdProduit(String idProduit) { this.idProduit = idProduit; }

    public String getNomProduit() { return nomProduit; }
    public void setNomProduit(String nomProduit) { this.nomProduit =
nomProduit; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }

    public TypeProduit getTypeProduit() { return typeProduit; }
    public void setTypeProduit(TypeProduit typeProduit) { this.typeProduit =
typeProduit; }

    public Statut getStatut() { return statut; }
    public void setStatut(Statut statut) { this.statut = statut; }

    public Instant getDateCreation() { return dateCreation; }
    public void setDateCreation(Instant dateCreation) { this.dateCreation =
dateCreation; }

    public Instant getDateModification() { return dateModification; }
    public void setDateModification(Instant dateModification) {
this.dateModification = dateModification; }
}
package tn.vermeg.gestionproduit.entities;

import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.mongodb.core.mapping.Document;
```

```java
import java.time.Instant;

@Document(collection = "garanties")
public class Garantie {

   @Id
   private String idGarantie;

   private String nomGarantie;
   private String description;

   private Statut statut;
   private TypeGarantie typeGarantie;

   // PARAMÈTRES FINANCIERS

   private double tauxRemboursement; // 0 → 1 recommandé
   private TypeMontant typeMontant;
   private TypePlafond typePlafond;

   private double plafondAnnuel;
   private double plafondMensuel;
   private double plafondParActe;

   private double franchise;

   private double coutMoyenParSinistre;

   // PARAMÈTRES CONTRACTUELS

   private int dureeMinContrat;
   private int dureeMaxContrat;

   private boolean resiliableAnnuellement;

   // AUDIT
   private String creePar;

   @CreatedDate
   private Instant dateCreation;

   @LastModifiedDate
   private Instant dateModification;

   private Instant dateDesactivation;

   // MÉTHODES MÉTIER

   public boolean estValide() {
       return tauxRemboursement >= 0
               && tauxRemboursement <= 1
               && plafondAnnuel >= 0
               && franchise >= 0;
```

```java
    }

    public boolean estActive() {
        return statut == Statut.ACTIF && dateDesactivation == null;
    }

    // CONSTRUCTEURS

    public Garantie() {}

    public Garantie(String idGarantie, String nomGarantie, String description,
                    Statut statut, TypeGarantie typeGarantie,
                    double tauxRemboursement, TypeMontant typeMontant,
                    TypePlafond typePlafond,
                    double plafondAnnuel, double plafondMensuel,
                    double plafondParActe, double franchise,
                    double coutMoyenParSinistre,
                    int dureeMinContrat, int dureeMaxContrat,
                    boolean resiliableAnnuellement,
                    String creePar,
                    Instant dateCreation,
                    Instant dateModification,
                    Instant dateDesactivation) {

        this.idGarantie = idGarantie;
        this.nomGarantie = nomGarantie;
        this.description = description;
        this.statut = statut;
        this.typeGarantie = typeGarantie;
        this.tauxRemboursement = tauxRemboursement;
        this.typeMontant = typeMontant;
        this.typePlafond = typePlafond;
        this.plafondAnnuel = plafondAnnuel;
        this.plafondMensuel = plafondMensuel;
        this.plafondParActe = plafondParActe;
        this.franchise = franchise;
        this.coutMoyenParSinistre = coutMoyenParSinistre;
        this.dureeMinContrat = dureeMinContrat;
        this.dureeMaxContrat = dureeMaxContrat;
        this.resiliableAnnuellement = resiliableAnnuellement;
        this.creePar = creePar;
        this.dateCreation = dateCreation;
        this.dateModification = dateModification;
        this.dateDesactivation = dateDesactivation;
    }

    // GETTERS & SETTERS

    public String getIdGarantie() { return idGarantie; }
    public void setIdGarantie(String idGarantie) { this.idGarantie =
idGarantie; }

    public String getNomGarantie() { return nomGarantie; }
```

```
   public void setNomGarantie(String nomGarantie) { this.nomGarantie =
nomGarantie; }

   public String getDescription() { return description; }
   public void setDescription(String description) { this.description =
description; }

   public Statut getStatut() { return statut; }
   public void setStatut(Statut statut) { this.statut = statut; }

   public TypeGarantie getTypeGarantie() { return typeGarantie; }
   public void setTypeGarantie(TypeGarantie typeGarantie) { this.typeGarantie
= typeGarantie; }

   public double getTauxRemboursement() { return tauxRemboursement; }
   public void setTauxRemboursement(double tauxRemboursement) {
this.tauxRemboursement = tauxRemboursement; }

   public TypeMontant getTypeMontant() { return typeMontant; }
   public void setTypeMontant(TypeMontant typeMontant) { this.typeMontant =
typeMontant; }

   public TypePlafond getTypePlafond() { return typePlafond; }
   public void setTypePlafond(TypePlafond typePlafond) { this.typePlafond =
typePlafond; }

   public double getPlafondAnnuel() { return plafondAnnuel; }
   public void setPlafondAnnuel(double plafondAnnuel) { this.plafondAnnuel =
plafondAnnuel; }

   public double getPlafondMensuel() { return plafondMensuel; }
   public void setPlafondMensuel(double plafondMensuel) { this.plafondMensuel
= plafondMensuel; }

   public double getPlafondParActe() { return plafondParActe; }
   public void setPlafondParActe(double plafondParActe) { this.plafondParActe
= plafondParActe; }

   public double getFranchise() { return franchise; }
   public void setFranchise(double franchise) { this.franchise = franchise; }

   public double getCoutMoyenParSinistre() { return coutMoyenParSinistre; }
   public void setCoutMoyenParSinistre(double coutMoyenParSinistre) {
this.coutMoyenParSinistre = coutMoyenParSinistre; }

   public int getDureeMinContrat() { return dureeMinContrat; }
   public void setDureeMinContrat(int dureeMinContrat) { this.dureeMinContrat
= dureeMinContrat; }

   public int getDureeMaxContrat() { return dureeMaxContrat; }
   public void setDureeMaxContrat(int dureeMaxContrat) { this.dureeMaxContrat
= dureeMaxContrat; }
```

```java
   public boolean isResiliableAnnuellement() { return resiliableAnnuellement;
}
   public void setResiliableAnnuellement(boolean resiliableAnnuellement) {
this.resiliableAnnuellement = resiliableAnnuellement; }

   public String getCreePar() { return creePar; }
   public void setCreePar(String creePar) { this.creePar = creePar; }

   public Instant getDateCreation() { return dateCreation; }
   public void setDateCreation(Instant dateCreation) { this.dateCreation =
dateCreation; }

   public Instant getDateModification() { return dateModification; }
   public void setDateModification(Instant dateModification) {
this.dateModification = dateModification; }

   public Instant getDateDesactivation() { return dateDesactivation; }
   public void setDateDesactivation(Instant dateDesactivation) {
this.dateDesactivation = dateDesactivation; }
}
package tn.vermeg.gestionproduit.entities;

import org.springframework.data.annotation.Id;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;
import java.util.List;

@Document(collection = "packs")
public class Pack {

   @Id
   private String idPack;

   private String nomPack;
   private String description;
   private String produitId;
   private String nomProduit;

   // Conditions d'éligibilité
   private Integer ageMinimum;
   private Integer ageMaximum;
   private List<TypeClient> typeClients;
   private int ancienneteContratMois;
   private CouvertureGeographique couvertureGeographique;

   private double prixMensuel;
   private int dureeMinContrat;
   private int dureeMaxContrat;

   private NiveauCouverture niveauCouverture;
```

```java
    private Statut statut;

    @CreatedDate
    private Instant dateCreation;

    @LastModifiedDate
    private Instant dateModification;

    public Pack() {}
    public Pack(String idPack, String nomPack, String description, String
produitId, String nomProduit,
                Integer ageMinimum, Integer ageMaximum, List<TypeClient>
typeClients,
                int ancienneteContratMois, CouvertureGeographique
couvertureGeographique,
                double prixMensuel, int dureeMinContrat, int dureeMaxContrat,
                NiveauCouverture niveauCouverture, Statut statut,
                Instant dateCreation, Instant dateModification) {

        this.idPack = idPack;
        this.nomPack = nomPack;
        this.description = description;
        this.produitId = produitId;
        this.nomProduit = nomProduit;
        this.ageMinimum = ageMinimum;
        this.ageMaximum = ageMaximum;
        this.typeClients = typeClients;
        this.ancienneteContratMois = ancienneteContratMois;
        this.couvertureGeographique = couvertureGeographique;
        this.prixMensuel = prixMensuel;
        this.dureeMinContrat = dureeMinContrat;
        this.dureeMaxContrat = dureeMaxContrat;
        this.niveauCouverture = niveauCouverture;
        this.statut = statut;
        this.dateCreation = dateCreation;
        this.dateModification = dateModification;
    }

    // ✅ Getters & Setters

    public String getIdPack() { return idPack; }
    public void setIdPack(String idPack) { this.idPack = idPack; }

    public String getNomPack() { return nomPack; }
    public void setNomPack(String nomPack) { this.nomPack = nomPack; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }

    public String getProduitId() { return produitId; }
    public void setProduitId(String produitId) { this.produitId = produitId; }
```

```
   public String getNomProduit() { return nomProduit; }
   public void setNomProduit(String nomProduit) { this.nomProduit =
nomProduit; }

   public Integer getAgeMinimum() { return ageMinimum; }
   public void setAgeMinimum(Integer ageMinimum) { this.ageMinimum =
ageMinimum; }

   public Integer getAgeMaximum() { return ageMaximum; }
   public void setAgeMaximum(Integer ageMaximum) { this.ageMaximum =
ageMaximum; }

   public List<TypeClient> getTypeClients() { return typeClients; }
   public void setTypeClients(List<TypeClient> typeClients) {
this.typeClients = typeClients; }

   public int getAncienneteContratMois() { return ancienneteContratMois; }
   public void setAncienneteContratMois(int ancienneteContratMois) {
this.ancienneteContratMois = ancienneteContratMois; }

   public CouvertureGeographique getCouvertureGeographique() { return
couvertureGeographique; }
   public void setCouvertureGeographique(CouvertureGeographique
couvertureGeographique) { this.couvertureGeographique =
couvertureGeographique; }

   public double getPrixMensuel() { return prixMensuel; }
   public void setPrixMensuel(double prixMensuel) { this.prixMensuel =
prixMensuel; }

   public int getDureeMinContrat() { return dureeMinContrat; }
   public void setDureeMinContrat(int dureeMinContrat) { this.dureeMinContrat
= dureeMinContrat; }

   public int getDureeMaxContrat() { return dureeMaxContrat; }
   public void setDureeMaxContrat(int dureeMaxContrat) { this.dureeMaxContrat
= dureeMaxContrat; }

   public NiveauCouverture getNiveauCouverture() { return niveauCouverture; }
   public void setNiveauCouverture(NiveauCouverture niveauCouverture) {
this.niveauCouverture = niveauCouverture; }

   public Statut getStatut() { return statut; }
   public void setStatut(Statut statut) { this.statut = statut; }

   public Instant getDateCreation() { return dateCreation; }
   public void setDateCreation(Instant dateCreation) { this.dateCreation =
dateCreation; }

   public Instant getDateModification() { return dateModification; }
   public void setDateModification(Instant dateModification) {
this.dateModification = dateModification; }
}
```

```java
package tn.vermeg.gestionproduit.entities;
public enum TypeProduit {SANTE,HABITATION,AUTO,EPARGNE,VIE}
package tn.vermeg.gestionproduit.entities;

public enum TypeGarantie {
   HOSPITALISATION,
   DENTAIRE,
   OPTIQUE,
   CONSULTATION
}
package tn.vermeg.gestionproduit.entities;

import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;

@Document(collection = "pack_garanties")
public class PackGarantie {

   @Id
   private String idPackGarantie;

   private String packId;
   private String garantieId;
   private String nomGarantie;

   private double tauxRemboursement;
   private double plafond;
   private double franchise;

   private TypeMontant typeMontant;

   private int delaiCarence; // en jours/mois
   private int priorite;

   private boolean actif;

   @CreatedDate
   private Instant dateActivation;

   private Instant dateDesactivation;

   // ✅ renommé pour clean code
   private boolean optionnelle;

   private double supplementPrix;

   // =======================
   // MÉTHODES MÉTIER
   // =======================
```

```java
    public boolean estValide() {
        return plafond >= 0 && tauxRemboursement >= 0;
    }

    public double calculerRemboursement(double montant) {
        double base = Math.max(0, montant - franchise); // ✅ correction
        double remboursement = base * tauxRemboursement;
        return Math.min(remboursement, plafond);
    }

    // =======================
    // CONSTRUCTEURS
    // =======================

    public PackGarantie() {}

    public PackGarantie(String idPackGarantie, String packId, String
garantieId,
                        String nomGarantie, double tauxRemboursement, double
plafond,
                        double franchise, TypeMontant typeMontant,
                        int delaiCarence, int priorite, boolean actif,
                        Instant dateActivation, Instant dateDesactivation,
                        boolean optionnelle, double supplementPrix) {

        this.idPackGarantie = idPackGarantie;
        this.packId = packId;
        this.garantieId = garantieId;
        this.nomGarantie = nomGarantie;
        this.tauxRemboursement = tauxRemboursement;
        this.plafond = plafond;
        this.franchise = franchise;
        this.typeMontant = typeMontant;
        this.delaiCarence = delaiCarence;
        this.priorite = priorite;
        this.actif = actif;
        this.dateActivation = dateActivation;
        this.dateDesactivation = dateDesactivation;
        this.optionnelle = optionnelle;
        this.supplementPrix = supplementPrix;
    }

    // =======================
    // GETTERS & SETTERS
    // =======================

    public String getIdPackGarantie() {
        return idPackGarantie;
    }

    public void setIdPackGarantie(String idPackGarantie) {
        this.idPackGarantie = idPackGarantie;
    }
```

```java
public String getPackId() {
    return packId;
}

public void setPackId(String packId) {
    this.packId = packId;
}

public String getGarantieId() {
    return garantieId;
}

public void setGarantieId(String garantieId) {
    this.garantieId = garantieId;
}

public String getNomGarantie() {
    return nomGarantie;
}

public void setNomGarantie(String nomGarantie) {
    this.nomGarantie = nomGarantie;
}

public double getTauxRemboursement() {
    return tauxRemboursement;
}

public void setTauxRemboursement(double tauxRemboursement) {
    this.tauxRemboursement = tauxRemboursement;
}

public double getPlafond() {
    return plafond;
}

public void setPlafond(double plafond) {
    this.plafond = plafond;
}

public double getFranchise() {
    return franchise;
}

public void setFranchise(double franchise) {
    this.franchise = franchise;
}

public TypeMontant getTypeMontant() {
    return typeMontant;
}
```

```java
public void setTypeMontant(TypeMontant typeMontant) {
    this.typeMontant = typeMontant;
}

public int getDelaiCarence() {
    return delaiCarence;
}

public void setDelaiCarence(int delaiCarence) {
    this.delaiCarence = delaiCarence;
}

public int getPriorite() {
    return priorite;
}

public void setPriorite(int priorite) {
    this.priorite = priorite;
}

public boolean isActif() {
    return actif;
}

public void setActif(boolean actif) {
    this.actif = actif;
}

public Instant getDateActivation() {
    return dateActivation;
}

public void setDateActivation(Instant dateActivation) {
    this.dateActivation = dateActivation;
}

public Instant getDateDesactivation() {
    return dateDesactivation;
}

public void setDateDesactivation(Instant dateDesactivation) {
    this.dateDesactivation = dateDesactivation;
}

public boolean isOptionnelle() {
    return optionnelle;
}

public void setOptionnelle(boolean optionnelle) {
    this.optionnelle = optionnelle;
}

public double getSupplementPrix() {
```

```java
        return supplementPrix;
    }

    public void setSupplementPrix(double supplementPrix) {
        this.supplementPrix = supplementPrix;
    }
}
package tn.vermeg.gestionproduit.entities;
public enum NiveauCouverture {BASIC, PREMIUM, GOLD}
package tn.vermeg.gestionproduit.entities;

public enum TypePlafond {
        PAR_ACTE,ANNUEL, MENSUEL, GLOBAL,PAR_SOINS}
package tn.vermeg.gestionproduit.entities;
public enum Statut {ACTIF, INACTIF, EN_ATTENTE, SUSPENDU, EXPIRE,RESLIE}
package tn.vermeg.gestionproduit.entities;
public enum CouvertureGeographique {LOCAL, NATIONAL, INTERNATIONAL, UE,
MAGHREB}
package tn.vermeg.gestionproduit.entities;
public enum TypeClient {INDIVIDUEL, FAMILLE, ENFANT, SENIOR, ENTREPRISE,
ETUDIANT}
package tn.vermeg.gestionproduit.entities;
public enum TypeMontant {FORFAIT, FRAIS_REELS, TARIF_CONVENTIONNE}
package tn.vermeg.gestionproduit.entities;

public enum BaseCalcul {
    FRAIS_REELS, TARIF_CONVENTIONNE,
    FORFAIT_FIXE, POURCENTAGE_PLAFOND,
    BASE_REMBOURSEMENT_SECU
}
package tn.vermeg.gestionproduit.repositories;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionproduit.entities.Garantie;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeGarantie;

import java.util.List;

@Repository
public interface GarantieRepository extends MongoRepository<Garantie, String>
{

    List<Garantie> findByTypeGarantie(TypeGarantie typeGarantie);
    List<Garantie> findByStatut(Statut statut);
    List<Garantie> findByNomGarantieContainingIgnoreCase(String nomGarantie);
    boolean existsByNomGarantieIgnoreCase(String nomGarantie);
    List<Garantie> findByTauxRemboursementGreaterThanEqual(double tauxMin);
    List<Garantie> findByPlafondAnnuelGreaterThanEqual(double plafondMin);
}
package tn.vermeg.gestionproduit.repositories;
```

```java
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionproduit.entities.PackGarantie;

import java.util.List;

@Repository
public interface PackGarantieRepository extends MongoRepository<PackGarantie,
String> {

   List<PackGarantie> findByPackId(String packId);
   List<PackGarantie> findByGarantieId(String garantieId);
   List<PackGarantie> findByPackIdAndOptionnelle(String packId, boolean
optionnelle);
   boolean existsByPackIdAndGarantieId(String packId, String garantieId);
   void deleteByPackId(String packId);
   void deleteByGarantieId(String garantieId);
}
package tn.vermeg.gestionproduit.repositories;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionproduit.entities.NiveauCouverture;
import tn.vermeg.gestionproduit.entities.Pack;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeClient;

import java.util.List;

@Repository
public interface PackRepository extends MongoRepository<Pack, String> {

   List<Pack> findByProduitId(String produitId);
   List<Pack> findByStatut(Statut statut);
   List<Pack> findByNomPackContainingIgnoreCase(String nom);
   boolean existsByNomPackIgnoreCase(String nom);
   List<Pack> findByPrixMensuelBetween(double prixMin, double prixMax);
   List<Pack> findByNiveauCouverture(NiveauCouverture niveauCouverture);
   List<Pack> findByTypeClientsContaining(TypeClient typeClient);
   List<Pack> findByProduitIdAndStatut(String produitId, Statut statut);
}

package tn.vermeg.gestionproduit.repositories;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import tn.vermeg.gestionproduit.entities.Produit;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeProduit;

import java.util.List;

@Repository
```

```java
public interface ProduitRepository extends MongoRepository<Produit, String> {

   List<Produit> findByTypeProduit(TypeProduit typeProduit);
   List<Produit> findByStatut(Statut statut);
   List<Produit> findByNomProduitContainingIgnoreCase(String nomProduit);
   boolean existsByNomProduitIgnoreCase(String nomProduit);
}
package tn.vermeg.gestionproduit.services;

import org.springframework.stereotype.Service;
import tn.vermeg.gestionproduit.entities.Garantie;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeGarantie;
import tn.vermeg.gestionproduit.repositories.GarantieRepository;
import java.time.Instant;
import java.util.List;

@Service
public class GarantieService {

   private final GarantieRepository garantieRepository;

   public GarantieService(GarantieRepository garantieRepository) {
       this.garantieRepository = garantieRepository;
   }

   // READ

   public List<Garantie> getAllGaranties() {
       return garantieRepository.findAll();
   }

   public Garantie getGarantieById(String idGarantie) {
       return garantieRepository.findById(idGarantie)
               .orElseThrow(() ->
                       new IllegalArgumentException("Garantie non trouvée
avec l'ID: " + idGarantie));
   }

   public List<Garantie> getGarantiesByStatut(Statut statut) {
       return garantieRepository.findByStatut(statut);
   }

   public List<Garantie> getGarantiesByType(TypeGarantie typeGarantie) {
       return garantieRepository.findByTypeGarantie(typeGarantie);
   }

   public List<Garantie> searchGaranties(String nom) {
       return garantieRepository.findByNomGarantieContainingIgnoreCase(nom);
   }

   public List<Garantie> getGarantiesByTauxRemboursementMin(double tauxMin) {
```

```
        if (tauxMin < 0 || tauxMin > 1) {
            throw new IllegalArgumentException("Le taux doit être entre 0 et
1.");
        }

        return
garantieRepository.findByTauxRemboursementGreaterThanEqual(tauxMin);
    }

    public List<Garantie> getGarantiesByPlafondMin(double plafondMin) {

        if (plafondMin < 0) {
            throw new IllegalArgumentException("Le plafond ne peut pas être
négatif.");
        }

        return
garantieRepository.findByPlafondAnnuelGreaterThanEqual(plafondMin);
    }

    // CREATE

    public Garantie createGarantie(Garantie garantie) {

        validateGarantie(garantie);

        if
(garantieRepository.existsByNomGarantieIgnoreCase(garantie.getNomGarantie()))
{
            throw new IllegalArgumentException("Une garantie avec ce nom
existe déjà.");
        }

        garantie.setStatut(Statut.ACTIF);
        return garantieRepository.save(garantie);
    }

    // UPDATE

    public Garantie updateGarantie(String idGarantie, Garantie details) {

        Garantie garantie = getGarantieById(idGarantie);

        validateGarantie(details);

        boolean nomChange = !garantie.getNomGarantie()
                .equalsIgnoreCase(details.getNomGarantie());

        if (nomChange &&
garantieRepository.existsByNomGarantieIgnoreCase(details.getNomGarantie())) {
            throw new IllegalArgumentException("Une garantie avec ce nom
existe déjà.");
        }
```

```
        garantie.setNomGarantie(details.getNomGarantie());
        garantie.setDescription(details.getDescription());
        garantie.setTypeGarantie(details.getTypeGarantie());
        garantie.setTauxRemboursement(details.getTauxRemboursement());
        garantie.setTypeMontant(details.getTypeMontant());
        garantie.setTypePlafond(details.getTypePlafond());
        garantie.setPlafondAnnuel(details.getPlafondAnnuel());
        garantie.setPlafondMensuel(details.getPlafondMensuel());
        garantie.setPlafondParActe(details.getPlafondParActe());
        garantie.setFranchise(details.getFranchise());
        garantie.setCoutMoyenParSinistre(details.getCoutMoyenParSinistre());
        garantie.setDureeMinContrat(details.getDureeMinContrat());
        garantie.setDureeMaxContrat(details.getDureeMaxContrat());

garantie.setResiliableAnnuellement(details.isResiliableAnnuellement());
        garantie.setCreePar(details.getCreePar());

        return garantieRepository.save(garantie);
    }

    // DELETE / DESACTIVATION

    public void deleteGarantie(String idGarantie) {

        if (!garantieRepository.existsById(idGarantie)) {
            throw new IllegalArgumentException("Garantie non trouvée avec
l'ID: " + idGarantie);
        }

        garantieRepository.deleteById(idGarantie);
    }

    public Garantie desactiverGarantie(String idGarantie) {

        Garantie garantie = getGarantieById(idGarantie);

        if (garantie.getStatut() == Statut.INACTIF) {
            throw new IllegalStateException("La garantie est déjà inactive.");
        }

        garantie.setStatut(Statut.INACTIF);
        garantie.setDateDesactivation(Instant.now());

        return garantieRepository.save(garantie);
    }

    // VALIDATION

    private void validateGarantie(Garantie garantie) {

        if (garantie.getNomGarantie() == null ||
garantie.getNomGarantie().isBlank()) {
```

```
            throw new IllegalArgumentException("Nom obligatoire.");
        }

        if (garantie.getTypeGarantie() == null) {
            throw new IllegalArgumentException("Type de garantie
obligatoire.");
        }

        if (!garantie.estValide()) {
            throw new IllegalArgumentException("Paramètres financiers
invalides.");
        }
    }
}
package tn.vermeg.gestionproduit.services;

import org.springframework.stereotype.Service;
import tn.vermeg.gestionproduit.entities.*;
import tn.vermeg.gestionproduit.repositories.GarantieRepository;
import tn.vermeg.gestionproduit.repositories.PackGarantieRepository;
import tn.vermeg.gestionproduit.repositories.PackRepository;

import java.util.List;

@Service
public class PackConfigurationService {

    private final PackRepository packRepository;
    private final PackGarantieRepository packGarantieRepository;
    private final GarantieRepository garantieRepository;

    public PackConfigurationService(PackRepository packRepository,
                                    PackGarantieRepository
packGarantieRepository,
                                    GarantieRepository garantieRepository) {
        this.packRepository = packRepository;
        this.packGarantieRepository = packGarantieRepository;
        this.garantieRepository = garantieRepository;
    }

    // =======================
    // AJOUT GARANTIE AU PACK
    // =======================

    public PackGarantie ajouterGarantieAuPack(String packId, String
garantieId, PackGarantie packGarantie) {

        Pack pack = packRepository.findById(packId)
                .orElseThrow(() -> new IllegalArgumentException("Pack non
trouvé"));

        Garantie garantie = garantieRepository.findById(garantieId)
```

```
                .orElseThrow(() -> new IllegalArgumentException("Garantie non
trouvée"));

        // 🔥 Vérifier duplication
        if (packGarantieRepository.existsByPackIdAndGarantieId(packId,
garantieId)) {
            throw new IllegalArgumentException("Garantie déjà ajoutée à ce
pack");
        }

        // 🔥 Initialisation propre
        packGarantie.setPackId(packId);
        packGarantie.setGarantieId(garantieId);
        packGarantie.setNomGarantie(garantie.getNomGarantie());

        packGarantie.setPriorite(packGarantie.getPriorite() <= 0 ? 1 :
packGarantie.getPriorite());
        packGarantie.setActif(true);
        packGarantie.setDelaiCarence(Math.max(packGarantie.getDelaiCarence(),
0));

        return packGarantieRepository.save(packGarantie);
    }

    // =======================
    // SUPPRESSION
    // =======================

    public void supprimerGarantieDuPack(String packId, String packGarantieId)
{

        packRepository.findById(packId)
                .orElseThrow(() -> new IllegalArgumentException("Pack non
trouvé"));

        PackGarantie pg = packGarantieRepository.findById(packGarantieId)
                .orElseThrow(() -> new IllegalArgumentException("PackGarantie
non trouvée"));

        // 🔥 Vérifier cohérence
        if (!pg.getPackId().equals(packId)) {
            throw new IllegalArgumentException("Cette garantie n'appartient
pas à ce pack");
        }

        packGarantieRepository.deleteById(packGarantieId);
    }

    // =======================
    // MODIFICATION
    // =======================
```

```java
    public PackGarantie modifierGarantieDansPack(String packId, String
packGarantieId, PackGarantie details) {

        packRepository.findById(packId)
                .orElseThrow(() -> new IllegalArgumentException("Pack non
trouvé"));

        PackGarantie pg = packGarantieRepository.findById(packGarantieId)
                .orElseThrow(() -> new IllegalArgumentException("PackGarantie
non trouvée"));

        // 🔥 Vérifier cohérence
        if (!pg.getPackId().equals(packId)) {
            throw new IllegalArgumentException("Cette garantie n'appartient
pas à ce pack");
        }

        // 🔥 Update contrôlé
        pg.setTauxRemboursement(details.getTauxRemboursement());
        pg.setPlafond(details.getPlafond());
        pg.setFranchise(details.getFranchise());
        pg.setTypeMontant(details.getTypeMontant());
        pg.setOptionnelle(details.isOptionnelle());
        pg.setSupplementPrix(details.getSupplementPrix());

        return packGarantieRepository.save(pg);
    }

    // =======================
    // LECTURE
    // =======================

    public List<PackGarantie> getGarantiesDuPack(String packId) {
        return packGarantieRepository.findByPackId(packId);
    }

    public List<PackGarantie> getGarantiesInclusesDuPack(String packId) {
        return packGarantieRepository.findByPackIdAndOptionnelle(packId,
false);
    }

    public List<PackGarantie> getGarantiesOptionnellesDuPack(String packId) {
        return packGarantieRepository.findByPackIdAndOptionnelle(packId,
true);
    }

    // =======================
    // PRIX TOTAL PACK
    // =======================

    public double calculerPrixTotalPack(String packId) {

        Pack pack = packRepository.findById(packId)
```

```
                .orElseThrow(() -> new IllegalArgumentException("Pack non
trouvé"));

        List<PackGarantie> garanties =
packGarantieRepository.findByPackId(packId);

        double supplement = garanties.stream()
                .filter(PackGarantie::isOptionnelle)
                .mapToDouble(PackGarantie::getSupplementPrix)
                .sum();

        return pack.getPrixMensuel() + supplement;
    }

    // =======================
    // CREATION PACK + GARANTIES
    // =======================

    public Pack creerPackAvecGaranties(Pack pack, List<String> garantieIds) {

        Pack savedPack = packRepository.save(pack);

        for (String garantieId : garantieIds) {

            Garantie garantie = garantieRepository.findById(garantieId)
                    .orElseThrow(() -> new IllegalArgumentException("Garantie
non trouvée"));

            // 🔥 éviter duplication
            if
(packGarantieRepository.existsByPackIdAndGarantieId(savedPack.getIdPack(),
garantieId)) {
                continue;
            }

            PackGarantie pg = new PackGarantie();
            pg.setPackId(savedPack.getIdPack());
            pg.setGarantieId(garantieId);
            pg.setNomGarantie(garantie.getNomGarantie());

            pg.setTauxRemboursement(garantie.getTauxRemboursement());
            pg.setPlafond(garantie.getPlafondAnnuel());
            pg.setFranchise(garantie.getFranchise());
            pg.setTypeMontant(garantie.getTypeMontant());

            pg.setOptionnelle(false);
            pg.setSupplementPrix(0.0);
            pg.setActif(true);
            pg.setDelaiCarence(0);
            pg.setPriorite(1);

            packGarantieRepository.save(pg);
        }
```

```
        return savedPack;
    }

    // GARANTIES DISPONIBLES

    public List<Garantie> getGarantiesDisponiblesPourPack(String packId) {

        List<Garantie> toutes = garantieRepository.findByStatut(Statut.ACTIF);
        List<PackGarantie> existantes =
packGarantieRepository.findByPackId(packId);

        return toutes.stream()
                .filter(g -> existantes.stream()
                        .noneMatch(pg ->
pg.getGarantieId().equals(g.getIdGarantie())))
                .toList();
    }
}
package tn.vermeg.gestionproduit.services;

import org.springframework.stereotype.Service;
import tn.vermeg.gestionproduit.entities.NiveauCouverture;
import tn.vermeg.gestionproduit.entities.Pack;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeClient;
import tn.vermeg.gestionproduit.repositories.PackRepository;

import java.util.List;

@Service
public class PackService {

    private final PackRepository packRepository;

    public PackService(PackRepository packRepository) {
        this.packRepository = packRepository;
    }

    // READ

    public List<Pack> getAllPacks() {
        return packRepository.findAll();
    }

    public Pack getPackById(String idPack) {
        return packRepository.findById(idPack)
                .orElseThrow(() ->
                        new IllegalArgumentException("Pack non trouvé avec
l'ID: " + idPack));
    }

    // CREATE
```

```java
    public Pack createPack(Pack pack) {

        validatePack(pack);

        if (packRepository.existsByNomPackIgnoreCase(pack.getNomPack())) {
            throw new IllegalArgumentException("Un pack avec ce nom existe
déjà.");
        }

        pack.setStatut(Statut.ACTIF);

        return packRepository.save(pack);
    }

    // UPDATE

    public Pack updatePack(String idPack, Pack details) {

        Pack pack = getPackById(idPack);

        validatePack(details);

        boolean nomChange = !pack.getNomPack()
                .equalsIgnoreCase(details.getNomPack());

        if (nomChange &&
packRepository.existsByNomPackIgnoreCase(details.getNomPack())) {
            throw new IllegalArgumentException("Un pack avec ce nom existe
déjà.");
        }

        pack.setNomPack(details.getNomPack());
        pack.setDescription(details.getDescription());
        pack.setProduitId(details.getProduitId());
        pack.setNomProduit(details.getNomProduit());
        pack.setPrixMensuel(details.getPrixMensuel());
        pack.setDureeMinContrat(details.getDureeMinContrat());
        pack.setDureeMaxContrat(details.getDureeMaxContrat());
        pack.setStatut(details.getStatut());

        return packRepository.save(pack);
    }

    // DELETE

    public void deletePack(String idPack) {

        if (!packRepository.existsById(idPack)) {
            throw new IllegalArgumentException("Pack non trouvé avec l'ID: " +
idPack);
        }
```

```
        packRepository.deleteById(idPack);
    }

    // DESACTIVER

    public Pack desactiverPack(String idPack) {

        Pack pack = getPackById(idPack);

        pack.setStatut(Statut.INACTIF);

        return packRepository.save(pack);
    }

    // FILTRES

    public List<Pack> getPacksByProduit(String produitId) {
        return packRepository.findByProduitId(produitId);
    }

    public List<Pack> getPacksByStatut(Statut statut) {
        return packRepository.findByStatut(statut);
    }

    public List<Pack> searchPacks(String nom) {
        return packRepository.findByNomPackContainingIgnoreCase(nom);
    }

    public List<Pack> getPacksByPrixRange(double prixMin, double prixMax) {

        if (prixMin < 0 || prixMax < 0 || prixMin > prixMax) {
            throw new IllegalArgumentException("Intervalle de prix
invalide.");
        }

        return packRepository.findByPrixMensuelBetween(prixMin, prixMax);
    }

    public List<Pack> getPacksByNiveauCouverture(NiveauCouverture
niveauCouverture) {
        return packRepository.findByNiveauCouverture(niveauCouverture);
    }

    public List<Pack> getPacksByTypeClient(TypeClient typeClient) {
        return packRepository.findByTypeClientsContaining(typeClient);
    }

    // VALIDATION

    private void validatePack(Pack pack) {

        if (pack.getNomPack() == null || pack.getNomPack().isBlank()) {
            throw new IllegalArgumentException("Nom du pack obligatoire.");
```

```
        }

        if (pack.getPrixMensuel() < 0) {
            throw new IllegalArgumentException("Prix invalide.");
        }
    }
}
package tn.vermeg.gestionproduit.services;

import org.springframework.stereotype.Service;
import tn.vermeg.gestionproduit.entities.Produit;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeProduit;
import tn.vermeg.gestionproduit.repositories.ProduitRepository;

import java.util.List;

@Service
public class ProduitService {

    private final ProduitRepository produitRepository;

    public ProduitService(ProduitRepository produitRepository) {
        this.produitRepository = produitRepository;
    }

    // READ

    public List<Produit> getAllProduits() {
        return produitRepository.findAll();
    }

    public Produit getProduitById(String idProduit) {
        return produitRepository.findById(idProduit)
                .orElseThrow(() ->
                        new IllegalArgumentException("Produit non trouvé avec
l'ID: " + idProduit));
    }

    public List<Produit> getProduitsByType(TypeProduit typeProduit) {
        return produitRepository.findByTypeProduit(typeProduit);
    }

    public List<Produit> getProduitsByStatut(Statut statut) {
        return produitRepository.findByStatut(statut);
    }

    public List<Produit> searchProduits(String nomProduit) {
        return
produitRepository.findByNomProduitContainingIgnoreCase(nomProduit);
    }

    // CREATE
```

```java
    public Produit createProduit(Produit produit) {

        validateProduit(produit);

        if
(produitRepository.existsByNomProduitIgnoreCase(produit.getNomProduit())) {
            throw new IllegalArgumentException("Un produit avec ce nom existe
déjà.");
        }

        produit.setStatut(Statut.ACTIF);

        return produitRepository.save(produit);
    }

    // UPDATE

    public Produit updateProduit(String idProduit, Produit produitDetails) {

        Produit produit = getProduitById(idProduit);

        validateProduit(produitDetails);

        boolean nomChange = !produit.getNomProduit()
                .equalsIgnoreCase(produitDetails.getNomProduit());

        if (nomChange &&

produitRepository.existsByNomProduitIgnoreCase(produitDetails.getNomProduit()
)) {
            throw new IllegalArgumentException("Un produit avec ce nom existe
déjà.");
        }

        produit.setNomProduit(produitDetails.getNomProduit());
        produit.setDescription(produitDetails.getDescription());
        produit.setTypeProduit(produitDetails.getTypeProduit());
        produit.setStatut(produitDetails.getStatut());

        return produitRepository.save(produit);
    }

    // DELETE

    public void deleteProduit(String idProduit) {

        if (!produitRepository.existsById(idProduit)) {
            throw new IllegalArgumentException("Produit non trouvé avec l'ID:
" + idProduit);
        }

        produitRepository.deleteById(idProduit);
```

```
    }

    // DÉSACTIVATION

    public Produit desactiverProduit(String idProduit) {

        Produit produit = getProduitById(idProduit);

        if (produit.getStatut() == Statut.INACTIF) {
            throw new IllegalStateException("Le produit est déjà inactif.");
        }

        produit.setStatut(Statut.INACTIF);

        return produitRepository.save(produit);
    }

    // VALIDATION MÉTIER

    private void validateProduit(Produit produit) {

        if (produit.getNomProduit() == null ||
produit.getNomProduit().isBlank()) {
            throw new IllegalArgumentException("Le nom du produit est
obligatoire.");
        }

        if (produit.getTypeProduit() == null) {
            throw new IllegalArgumentException("Le type du produit est
obligatoire.");
        }
    }
}
package tn.vermeg.gestionproduit.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionproduit.entities.Produit;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeProduit;
import tn.vermeg.gestionproduit.services.ProduitService;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/produits")
@CrossOrigin(origins = "http://localhost:4200")
public class ProduitController {

    @Autowired
```

```
    private ProduitService produitService;

    // READ

    @GetMapping
    public ResponseEntity<List<Produit>> getAllProduits() {
        return ResponseEntity.ok(produitService.getAllProduits());
    }

    @GetMapping("/{idProduit}")
    public ResponseEntity<Produit> getProduitById(@PathVariable String
idProduit) {
        try {
            return
ResponseEntity.ok(produitService.getProduitById(idProduit));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @GetMapping("/type/{typeProduit}")
    public ResponseEntity<List<Produit>> getProduitsByType(@PathVariable
TypeProduit typeProduit) {
        return
ResponseEntity.ok(produitService.getProduitsByType(typeProduit));
    }

    @GetMapping("/statut/{statut}")
    public ResponseEntity<List<Produit>> getProduitsByStatut(@PathVariable
Statut statut) {
        return ResponseEntity.ok(produitService.getProduitsByStatut(statut));
    }

    @GetMapping("/search")
    public ResponseEntity<List<Produit>> searchProduits(@RequestParam String
nom) {
        return ResponseEntity.ok(produitService.searchProduits(nom));
    }

    // CREATE

    @PostMapping
    public ResponseEntity<Produit> createProduit(@RequestBody Produit produit)
{
        try {
            Produit newProduit = produitService.createProduit(produit);
            return
ResponseEntity.status(HttpStatus.CREATED).body(newProduit);
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }
```

```
    // UPDATE

    @PutMapping("/{idProduit}")
    public ResponseEntity<Produit> updateProduit(
            @PathVariable String idProduit,
            @RequestBody Produit produitDetails) {
        try {
            return ResponseEntity.ok(produitService.updateProduit(idProduit,
produitDetails));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @PatchMapping("/{idProduit}/desactiver")
    public ResponseEntity<Produit> desactiverProduit(@PathVariable String
idProduit) {
        try {
            return
ResponseEntity.ok(produitService.desactiverProduit(idProduit));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().build();
        }
    }


    // DELETE
    @DeleteMapping("/{idProduit}")
    public ResponseEntity<Void> deleteProduit(@PathVariable String idProduit)
{
        try {
            produitService.deleteProduit(idProduit);
            return ResponseEntity.noContent().build();
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }
}
package tn.vermeg.gestionproduit.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionproduit.entities.NiveauCouverture;
import tn.vermeg.gestionproduit.entities.Pack;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeClient;
import tn.vermeg.gestionproduit.services.PackService;

import java.util.List;
```

```java
@RestController
@RequestMapping("/api/packs")
@CrossOrigin(origins = "http://localhost:4200")
public class PackController {

   @Autowired
   private PackService packService;

   // READ

   @GetMapping
   public ResponseEntity<List<Pack>> getAllPacks() {
       return ResponseEntity.ok(packService.getAllPacks());
   }

   @GetMapping("/{idPack}")
   public ResponseEntity<Pack> getPackById(@PathVariable String idPack) {
       try {
           return ResponseEntity.ok(packService.getPackById(idPack));
       } catch (IllegalArgumentException e) {
           return ResponseEntity.notFound().build();
       }
   }

   @GetMapping("/produit/{produitId}")
   public ResponseEntity<List<Pack>> getPacksByProduit(@PathVariable String
produitId) {
       return ResponseEntity.ok(packService.getPacksByProduit(produitId));
   }

   @GetMapping("/statut/{statut}")
   public ResponseEntity<List<Pack>> getPacksByStatut(@PathVariable Statut
statut) {
       return ResponseEntity.ok(packService.getPacksByStatut(statut));
   }

   @GetMapping("/niveau/{niveauCouverture}")
   public ResponseEntity<List<Pack>> getPacksByNiveau(
           @PathVariable NiveauCouverture niveauCouverture) {
       return
ResponseEntity.ok(packService.getPacksByNiveauCouverture(niveauCouverture));
   }

   @GetMapping("/type-client/{typeClient}")
   public ResponseEntity<List<Pack>> getPacksByTypeClient(
           @PathVariable TypeClient typeClient) {
       return
ResponseEntity.ok(packService.getPacksByTypeClient(typeClient));
   }

   @GetMapping("/search")
```

```
    public ResponseEntity<List<Pack>> searchPacks(@RequestParam String
nomPack) {
        return ResponseEntity.ok(packService.searchPacks(nomPack));
    }

    @GetMapping("/prix-range")
    public ResponseEntity<List<Pack>> getPacksByPrixRange(
            @RequestParam double prixMin,
            @RequestParam double prixMax) {
        try {
            return ResponseEntity.ok(packService.getPacksByPrixRange(prixMin,
prixMax));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    // CREATE

    @PostMapping
    public ResponseEntity<Pack> createPack(@RequestBody Pack pack) {
        try {
            return ResponseEntity.status(HttpStatus.CREATED)
                    .body(packService.createPack(pack));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    // UPDATE

    @PutMapping("/{idPack}")
    public ResponseEntity<Pack> updatePack(
            @PathVariable String idPack,
            @RequestBody Pack packDetails) {
        try {
            return ResponseEntity.ok(packService.updatePack(idPack,
packDetails));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @PatchMapping("/{idPack}/desactiver")
    public ResponseEntity<Pack> desactiverPack(@PathVariable String idPack) {
        try {
            return ResponseEntity.ok(packService.desactiverPack(idPack));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().build();
        }
    }
```

```
    // DELETE

    @DeleteMapping("/{idPack}")
    public ResponseEntity<Void> deletePack(@PathVariable String idPack) {
        try {
            packService.deletePack(idPack);
            return ResponseEntity.noContent().build();
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }
}
package tn.vermeg.gestionproduit.controllers;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionproduit.entities.Garantie;
import tn.vermeg.gestionproduit.entities.Pack;
import tn.vermeg.gestionproduit.entities.PackGarantie;
import tn.vermeg.gestionproduit.services.PackConfigurationService;

import java.util.List;

@RestController
@RequestMapping("/api/pack-configuration")
@CrossOrigin(origins = "http://localhost:4200")
public class PackConfigurationController {

    private final PackConfigurationService packConfigurationService;

    public PackConfigurationController(PackConfigurationService
packConfigurationService) {
        this.packConfigurationService = packConfigurationService;
    }

    // READ

    @GetMapping("/{packId}/garanties")
    public ResponseEntity<?> getGarantiesDuPack(@PathVariable String packId) {
        try {
            return
ResponseEntity.ok(packConfigurationService.getGarantiesDuPack(packId));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(e.getMessage());
        }
    }

    @GetMapping("/{packId}/garanties/incluses")
    public ResponseEntity<?> getGarantiesInclusesDuPack(@PathVariable String
packId) {
```

```
        try {
            return
ResponseEntity.ok(packConfigurationService.getGarantiesInclusesDuPack(packId)
);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(e.getMessage());
        }
    }

    @GetMapping("/{packId}/garanties/optionnelles")
    public ResponseEntity<?> getGarantiesOptionnellesDuPack(@PathVariable
String packId) {
        try {
            return
ResponseEntity.ok(packConfigurationService.getGarantiesOptionnellesDuPack(pac
kId));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(e.getMessage());
        }
    }

    @GetMapping("/{packId}/garanties-disponibles")
    public ResponseEntity<?> getGarantiesDisponiblesPourPack(@PathVariable
String packId) {
        try {
            return
ResponseEntity.ok(packConfigurationService.getGarantiesDisponiblesPourPack(pa
ckId));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(e.getMessage());
        }
    }

    @GetMapping("/{packId}/prix-total")
    public ResponseEntity<?> calculerPrixTotalPack(@PathVariable String
packId) {
        try {
            return
ResponseEntity.ok(packConfigurationService.calculerPrixTotalPack(packId));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                    .body(e.getMessage());
        }
    }

    // CREATE

    @PostMapping("/{packId}/garanties/{garantieId}")
    public ResponseEntity<?> ajouterGarantieAuPack(
            @PathVariable String packId,
```

```
            @PathVariable String garantieId,
            @RequestBody PackGarantie packGarantie) {

        try {
            return ResponseEntity.status(HttpStatus.CREATED)

.body(packConfigurationService.ajouterGarantieAuPack(packId, garantieId,
packGarantie));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body(e.getMessage());
        }
    }

    @PostMapping("/avec-garanties-complet")
    public ResponseEntity<?> creerPackAvecGaranties(
            @RequestBody CreatePackWithGarantiesRequest request) {

        try {
            return ResponseEntity.status(HttpStatus.CREATED)
                    .body(packConfigurationService.creerPackAvecGaranties(
                            request.getPack(), request.getGarantieIds()));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body(e.getMessage());
        }
    }

    // UPDATE

    @PutMapping("/{packId}/garanties/{packGarantieId}")
    public ResponseEntity<?> modifierGarantieDansPack(
            @PathVariable String packId,
            @PathVariable String packGarantieId,
            @RequestBody PackGarantie details) {

        try {
            return ResponseEntity.ok(
                    packConfigurationService.modifierGarantieDansPack(packId,
packGarantieId, details));
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                    .body(e.getMessage());
        }
    }

    // DELETE

    @DeleteMapping("/{packId}/garanties/{packGarantieId}")
    public ResponseEntity<Void> supprimerGarantieDuPack(
            @PathVariable String packId,
            @PathVariable String packGarantieId) {
```

```
        try {
            packConfigurationService.supprimerGarantieDuPack(packId,
packGarantieId);
            return ResponseEntity.noContent().build(); // ✅ 204
        } catch (Exception e) {
            return ResponseEntity.badRequest().build();
        }
    }

    // DTO

    public static class CreatePackWithGarantiesRequest {

        private Pack pack;
        private List<String> garantieIds;

        public Pack getPack() {
            return pack;
        }

        public void setPack(Pack pack) {
            this.pack = pack;
        }

        public List<String> getGarantieIds() {
            return garantieIds;
        }

        public void setGarantieIds(List<String> garantieIds) {
            this.garantieIds = garantieIds;
        }
    }
}
package tn.vermeg.gestionproduit.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import tn.vermeg.gestionproduit.entities.Garantie;
import tn.vermeg.gestionproduit.entities.Statut;
import tn.vermeg.gestionproduit.entities.TypeGarantie;
import tn.vermeg.gestionproduit.services.GarantieService;

import java.util.List;

@RestController
@RequestMapping("/api/garanties")
@CrossOrigin(origins = "http://localhost:4200")
public class GarantieController {

    @Autowired
    private GarantieService garantieService;
```

```
    // READ
    @GetMapping
    public ResponseEntity<List<Garantie>> getAllGaranties() {
        return ResponseEntity.ok(garantieService.getAllGaranties());
    }

    @GetMapping("/{idGarantie}")
    public ResponseEntity<Garantie> getGarantieById(@PathVariable String
idGarantie) {
        try {
            return
ResponseEntity.ok(garantieService.getGarantieById(idGarantie));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @GetMapping("/type/{typeGarantie}")
    public ResponseEntity<List<Garantie>> getGarantiesByType(
            @PathVariable TypeGarantie typeGarantie) {
        return
ResponseEntity.ok(garantieService.getGarantiesByType(typeGarantie));
    }

    @GetMapping("/statut/{statut}")
    public ResponseEntity<List<Garantie>> getGarantiesByStatut(@PathVariable
Statut statut) {
        return
ResponseEntity.ok(garantieService.getGarantiesByStatut(statut));
    }

    @GetMapping("/search")
    public ResponseEntity<List<Garantie>> searchGaranties(@RequestParam String
nomGarantie) {
        return
ResponseEntity.ok(garantieService.searchGaranties(nomGarantie));
    }

    @GetMapping("/taux-min/{tauxMin}")
    public ResponseEntity<List<Garantie>> getGarantiesByTauxMin(
            @PathVariable double tauxMin) {
        try {
            return ResponseEntity.ok(

garantieService.getGarantiesByTauxRemboursementMin(tauxMin));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    @GetMapping("/plafond-min/{plafondMin}")
    public ResponseEntity<List<Garantie>> getGarantiesByPlafondMin(
```

```
            @PathVariable double plafondMin) {
        try {
            return ResponseEntity.ok(
                    garantieService.getGarantiesByPlafondMin(plafondMin));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }
    // CREATE

    @PostMapping
    public ResponseEntity<Garantie> createGarantie(@RequestBody Garantie
garantie) {
        try {
            return ResponseEntity.status(HttpStatus.CREATED)
                    .body(garantieService.createGarantie(garantie));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    // UPDATE

    @PutMapping("/{idGarantie}")
    public ResponseEntity<Garantie> updateGarantie(
            @PathVariable String idGarantie,
            @RequestBody Garantie garantieDetails) {
        try {
            return ResponseEntity.ok(
                    garantieService.updateGarantie(idGarantie,
garantieDetails));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @PatchMapping("/{idGarantie}/desactiver")
    public ResponseEntity<Garantie> desactiverGarantie(@PathVariable String
idGarantie) {
        try {
            return
ResponseEntity.ok(garantieService.desactiverGarantie(idGarantie));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.notFound().build();
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    // DELETE

    @DeleteMapping("/{idGarantie}")
```

```
   public ResponseEntity<Void> deleteGarantie(@PathVariable String
idGarantie) {
       try {
           garantieService.deleteGarantie(idGarantie);
           return ResponseEntity.noContent().build();
       } catch (IllegalArgumentException e) {
           return ResponseEntity.notFound().build();
       }
   }
}
```

//ai-service

```
import os
import requests
from typing import Dict, List, Any
from .base_chatbot import BaseChatbot

class UnifiedPackChatbot(BaseChatbot):
    """
    Chatbot unifié pour la création ET configuration de packs d'assurance.
    Permet de créer un pack et de le configurer en une seule conversation.
    """

    def __init__(self):
        # Définir les champs pour la création et la configuration
        fields = [
            {
                "key": "action_type",
                "question": "Que souhaitez-vous faire ? (creer / configurer)",
                "type": "choice",
                "choices": ["creer", "configurer"],
                "required": True,
                "validation": lambda x: x in ["creer", "configurer"],
                "error_message": "Veuillez choisir entre 'creer' ou
'configurer'."
            },
            {
                "key": "pack_id",
                "question": "Quel est l'ID du pack ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID du pack doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "nom_pack",
                "question": "Quel est le nom du pack d'assurance ?",
                "type": "text",
                "required": True,
```

```
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "Le nom du pack doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "description",
                "question": "Quelle est la description du pack ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 10,
                "error_message": "La description doit contenir au moins 10
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "produit_id",
                "question": "Quel est l'ID du produit auquel appartient ce
pack ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID du produit doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "prix_mensuel",
                "question": "Quel est le prix mensuel du pack (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x > 0,
                "error_message": "Le prix mensuel doit être un nombre
positif.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "duree_min_contrat",
                "question": "Quelle est la durée minimale du contrat (en mois)
?",
                "type": "int",
                "required": True,
                "validation": lambda x: 1 <= x <= 120,
                "error_message": "La durée minimale doit être entre 1 et 120
mois.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "duree_max_contrat",
                "question": "Quelle est la durée maximale du contrat (en mois)
?",
                "type": "int",
                "required": True,
```

```
                "validation": lambda x, data: x >
data.get("duree_min_contrat", 0) and x <= 120,
                "error_message": "La durée maximale doit être supérieure à la
durée minimale et <= 120 mois.",
                "depends_on": "duree_min_contrat",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "niveau_couverture",
                "question": "Quel est le niveau de couverture ? (basic /
premium / gold)",
                "type": "choice",
                "choices": ["basic", "premium", "gold"],
                "required": True,
                "validation": lambda x: x in ["basic", "premium", "gold"],
                "error_message": "Le niveau doit être: basic, premium ou
gold.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "statut",
                "question": "Quel est le statut du pack ? (ACTIF / INACTIF /
EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "configuration_type",
                "question": "Quel type de configuration ? (statut / garanties
/ prix / conditions_eligibilite / complet)",
                "type": "choice",
                "choices": ["statut", "garanties", "prix",
"conditions_eligibilite", "complet"],
                "required": True,
                "validation": lambda x: x in ["statut", "garanties", "prix",
"conditions_eligibilite", "complet"],
                "error_message": "Veuillez choisir parmi: statut, garanties,
prix, conditions_eligibilite, complet.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "new_statut",
                "question": "Quel est le nouveau statut ? (ACTIF / INACTIF /
EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
```

```
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE.",
                "condition": lambda data: data.get("configuration_type") ==
"statut"
            },
            {
                "key": "garantie_action",
                "question": "Quel action sur les garanties ? (ajouter /
supprimer / modifier / lister)",
                "type": "choice",
                "choices": ["ajouter", "supprimer", "modifier", "lister"],
                "required": True,
                "validation": lambda x: x in ["ajouter", "supprimer",
"modifier", "lister"],
                "error_message": "Veuillez choisir parmi: ajouter, supprimer,
modifier, lister.",
                "condition": lambda data: data.get("configuration_type") in
["garanties", "complet"]
            },
            {
                "key": "garantie_id",
                "question": "Quel est l'ID de la garantie à
ajouter/supprimer/modifier ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID de la garantie doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("garantie_action") in
["ajouter", "supprimer", "modifier"]
            },
            {
                "key": "pack_garantie_id",
                "question": "Quel est l'ID du pack-garantie à
supprimer/modifier ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID du pack-garantie doit contenir au moins
3 caractères.",
                "condition": lambda data: data.get("garantie_action") in
["supprimer", "modifier"]
            },
            {
                "key": "new_prix",
                "question": "Quel est le nouveau prix mensuel (en DT) ?",
                "type": "float",
                "required": True,
```

```
                "validation": lambda x: x > 0,
                "error_message": "Le prix doit être un nombre positif.",
                "condition": lambda data: data.get("configuration_type") in
["prix", "complet"]
            },
            {
                "key": "add_condition_eligibilite",
                "question": "Voulez-vous ajouter une condition d'éligibilité
spécifique pour ce pack ? (Oui/Non)",
                "type": "boolean",
                "required": True,
                "validation": lambda x: isinstance(x, bool),
                "error_message": "Veuillez répondre par 'Oui' ou 'Non'.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["conditions_eligibilite", "complet"]
            }
        ]

        # Champs pour les conditions d'éligibilité
        condition_fields = [
            {
                "key": "age_minimum",
                "question": "Quel est l'âge minimum autorisé pour ce pack ?",
                "type": "int",
                "required": True,
                "validation": lambda x: 0 <= x <= 120,
                "error_message": "L'âge minimum doit être entre 0 et 120."
            },
            {
                "key": "age_maximum",
                "question": "Quel est l'âge maximum autorisé pour ce pack ?",
                "type": "int",
                "required": True,
                "validation": lambda x, data: x > data.get("age_minimum", 0),
                "error_message": "L'âge maximum doit être supérieur à l'âge
minimum.",
                "depends_on": "age_minimum"
            },
            {
                "key": "type_clients",
                "question": "Quels types de clients sont acceptés ?
(INDIVIDUEL / FAMILLE / ENFANT / SENIOR / ENTREPRISE / ETUDIANT) - Séparez
par des virgules",
                "type": "list",
                "required": True,
                "validation": lambda x: all(client in ["INDIVIDUEL",
"FAMILLE", "ENFANT", "SENIOR", "ENTREPRISE", "ETUDIANT"] for client in x),
                "error_message": "Types valides: INDIVIDUEL, FAMILLE, ENFANT,
SENIOR, ENTREPRISE, ETUDIANT."
            },
            {
                "key": "anciennete_contrat_mois",
```

```
                "question": "Quelle est l'ancienneté de contrat minimale
requise (en mois) ?",
                "type": "int",
                "required": True,
                "validation": lambda x: x >= 0,
                "error_message": "L'ancienneté doit être un nombre positif."
            },
            {
                "key": "couverture_geographique",
                "question": "Quelle est la couverture géographique ? (LOCAL /
NATIONAL / INTERNATIONAL / UE / MAGHREB)",
                "type": "choice",
                "choices": ["LOCAL", "NATIONAL", "INTERNATIONAL", "UE",
"MAGHREB"],
                "required": True,
                "validation": lambda x: x in ["LOCAL", "NATIONAL",
"INTERNATIONAL", "UE", "MAGHREB"],
                "error_message": "Veuillez choisir parmi: LOCAL, NATIONAL,
INTERNATIONAL, UE, MAGHREB."
            }
        ]

        # Combiner tous les champs
        all_fields = fields + condition_fields

        service_url = os.getenv("PRODUCT_SERVICE_URL",
"http://localhost:9093")
        super().__init__(all_fields, service_url, "Pack")

        # Stocker les champs conditionnels séparément
        self.condition_fields = condition_fields

    def _should_ask_field(self, field_key: str, data: Dict) -> bool:
        """Détermine si un champ doit être demandé selon les conditions"""
        field = next((f for f in self.fields if f["key"] == field_key), None)
        if not field:
            return False

        # Si le champ a une condition, l'évaluer
        if "condition" in field:
            return field["condition"](data)

        return True

    def _get_next_field(self) -> Dict:
        """Obtient le prochain champ à demander selon les conditions"""
        data = self.collected_data

        # Parcourir les champs dans l'ordre
        for i, field in enumerate(self.fields):
            if i <= self.current_field_index:
                continue
```

```
        # Vérifier si ce champ doit être demandé
        if self._should_ask_field(field["key"], data):
            self.current_field_index = i
            return field

    return None

def _handle_collecting(self, message: str) -> Dict[str, Any]:
    """Gère la collecte des données avec champs conditionnels"""
    data = self.collected_data
    current_field = self._get_current_field()

    if not current_field:
        return self._handle_welcome(message)

    field_key = current_field["key"]

    # Vérifier si ce champ doit être traité
    if not self._should_ask_field(field_key, data):
        # Passer au champ suivant
        self.current_field_index += 1
        return self._handle_collecting(message)

    # Parser et valider le message
    parsed_value = self._smart_parse(message, current_field)

    if parsed_value is None:
        return {
            "response": current_field["error_message"],
            "is_complete": False,
            "field": field_key
        }

    # Valider la valeur
    if not self._validate_field(parsed_value, current_field):
        return {
            "response": current_field["error_message"],
            "is_complete": False,
            "field": field_key
        }

    # Stocker la valeur
    data[field_key] = parsed_value

    # Passer au champ suivant
    self.current_field_index += 1

    # Vérifier si la collecte est terminée
    next_field = self._get_next_field()
    if not next_field:
        return self._handle_validation("")

    return {
```

```
            "response": next_field["question"],
            "is_complete": False,
            "field": next_field["key"]
        }

    def _handle_validation(self, message: str) -> Dict[str, Any]:
        """Gère la validation et la création/configuration"""
        data = self.collected_data
        action_type = data.get("action_type")

        try:
            if action_type == "creer":
                return self._create_pack(data)
            elif action_type == "configurer":
                return self._configure_pack(data)
            else:
                return {
                    "response": "Type d'action non valide.",
                    "is_complete": False
                }
        except Exception as e:
            return {
                "response": f"Erreur lors de l'opération: {str(e)}",
                "is_complete": False
            }

    def _create_pack(self, data: Dict) -> Dict[str, Any]:
        """Crée un nouveau pack"""
        pack_data = {
            "nomPack": data["nom_pack"],
            "description": data["description"],
            "produitId": data["produit_id"],
            "prixMensuel": data["prix_mensuel"],
            "dureeMinContrat": data["duree_min_contrat"],
            "dureeMaxContrat": data["duree_max_contrat"],
            "niveauCouverture": data["niveau_couverture"],
            "statut": data["statut"]
        }

        # Ajouter les conditions d'éligibilité si présentes
        if data.get("add_condition_eligibilite"):
            pack_data.update({
                "ageMinimum": data.get("age_minimum"),
                "ageMaximum": data.get("age_maximum"),
                "typeClients": data.get("type_clients", []),
                "ancienneteContratMois": data.get("anciennete_contrat_mois"),
                "couvertureGeographique": data.get("couverture_geographique")
            })

        response = requests.post(f"{self.service_url}/api/packs",
json=pack_data, timeout=10)

        if response.status_code == 201:
```

```
            pack = response.json()
            return {
                "response": f"Pack '{pack['nomPack']}' créé avec succès ! ID:
{pack['idPack']}",
                "is_complete": True,
                "data": pack
            }
        else:
            return {
                "response": f"Erreur lors de la création:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _configure_pack(self, data: Dict) -> Dict[str, Any]:
        """Configure un pack existant"""
        pack_id = data["pack_id"]
        config_type = data["configuration_type"]

        if config_type == "statut":
            return self._update_pack_statut(pack_id, data["new_statut"])
        elif config_type == "garanties":
            return self._manage_pack_garanties(pack_id, data)
        elif config_type == "prix":
            return self._update_pack_prix(pack_id, data["new_prix"])
        elif config_type == "conditions_eligibilite":
            return self._update_pack_conditions(pack_id, data)
        elif config_type == "complet":
            return self._update_pack_complet(pack_id, data)
        else:
            return {
                "response": "Type de configuration non valide.",
                "is_complete": False
            }

    def _update_pack_statut(self, pack_id: str, new_statut: str) -> Dict[str,
Any]:
        """Met à jour le statut d'un pack"""
        response =
requests.patch(f"{self.service_url}/api/packs/{pack_id}/desactiver",
timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Statut du pack {pack_id} mis à jour avec succès
vers {new_statut}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour du statut:
{response.status_code} - {response.text}",
                "is_complete": False
```

```
            }

    def _manage_pack_garanties(self, pack_id: str, data: Dict) -> Dict[str,
Any]:
        """Gère les garanties d'un pack"""
        garantie_action = data.get("garantie_action")

        if garantie_action == "ajouter":
            return self._add_garantie_to_pack(pack_id, data)
        elif garantie_action == "supprimer":
            return self._remove_garantie_from_pack(pack_id, data)
        elif garantie_action == "modifier":
            return self._modify_garantie_in_pack(pack_id, data)
        elif garantie_action == "lister":
            return self._list_pack_garanties(pack_id)
        else:
            return {
                "response": "Action sur garantie non valide.",
                "is_complete": False
            }

    def _add_garantie_to_pack(self, pack_id: str, data: Dict) -> Dict[str,
Any]:
        """Ajoute une garantie à un pack"""
        garantie_data = {
            "packId": pack_id,
            "garantieId": data["garantie_id"],
            "optionnelle": False,  # Valeur par défaut
            "prixSupplementaire": 0.0  # Valeur par défaut
        }

        response =
requests.post(f"{self.service_url}/api/pack-configuration/{pack_id}/garanties
/{data['garantie_id']}",
                              json=garantie_data, timeout=10)

        if response.status_code == 201:
            return {
                "response": f"Garantie {data['garantie_id']} ajoutée avec
succès au pack {pack_id}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de l'ajout de garantie:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _remove_garantie_from_pack(self, pack_id: str, data: Dict) ->
Dict[str, Any]:
        """Supprime une garantie d'un pack"""
```

```
        response =
requests.delete(f"{self.service_url}/api/pack-configuration/{pack_id}/garanti
es/{data['pack_garantie_id']}",
                                timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Garantie {data['pack_garantie_id']} supprimée
avec succès du pack {pack_id}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la suppression de garantie:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _modify_garantie_in_pack(self, pack_id: str, data: Dict) -> Dict[str,
Any]:
        """Modifie une garantie dans un pack"""
        garantie_data = {
            "optionnelle": False,  # Valeur par défaut
            "prixSupplementaire": 0.0  # Valeur par défaut
        }

        response =
requests.put(f"{self.service_url}/api/pack-configuration/{pack_id}/garanties/
{data['pack_garantie_id']}",
                              json=garantie_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Garantie {data['pack_garantie_id']} modifiée
avec succès dans le pack {pack_id}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la modification de garantie:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _list_pack_garanties(self, pack_id: str) -> Dict[str, Any]:
        """Liste les garanties d'un pack"""
        response =
requests.get(f"{self.service_url}/api/pack-configuration/{pack_id}/garanties"
, timeout=10)

        if response.status_code == 200:
            garanties = response.json()
            return {
```

```
                "response": f"Le pack {pack_id} contient {len(garanties)}
garantie(s): {', '.join([g['idGarantie'] for g in garanties])}",
                "is_complete": True,
                "data": garanties
            }
        else:
            return {
                "response": f"Erreur lors de la liste des garanties:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_pack_prix(self, pack_id: str, new_prix: float) -> Dict[str,
Any]:
        """Met à jour le prix d'un pack"""
        update_data = {"prixMensuel": new_prix}

        response = requests.put(f"{self.service_url}/api/packs/{pack_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Prix du pack {pack_id} mis à jour avec succès à
{new_prix} DT/mois",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour du prix:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_pack_conditions(self, pack_id: str, data: Dict) -> Dict[str,
Any]:
        """Met à jour les conditions d'éligibilité d'un pack"""
        if not data.get("add_condition_eligibilite"):
            return {
                "response": "Aucune condition d'éligibilité à mettre à jour.",
                "is_complete": True
            }

        update_data = {
            "ageMinimum": data.get("age_minimum"),
            "ageMaximum": data.get("age_maximum"),
            "typeClients": data.get("type_clients", []),
            "anciennetеContratMois": data.get("anciennete_contrat_mois"),
            "couvertureGeographique": data.get("couverture_geographique")
        }

        response = requests.put(f"{self.service_url}/api/packs/{pack_id}",
json=update_data, timeout=10)
```

```
        if response.status_code == 200:
            return {
                "response": f"Conditions d'éligibilité du pack {pack_id} mises
à jour avec succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour des conditions:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_pack_complet(self, pack_id: str, data: Dict) -> Dict[str,
Any]:
        """Met à jour complètement un pack"""
        update_data = {}

        # Ajouter le prix si présent
        if data.get("new_prix"):
            update_data["prixMensuel"] = data["new_prix"]

        # Ajouter les conditions d'éligibilité si présentes
        if data.get("add_condition_eligibilite"):
            update_data.update({
                "ageMinimum": data.get("age_minimum"),
                "ageMaximum": data.get("age_maximum"),
                "typeClients": data.get("type_clients", []),
                "anciennetеContratMois": data.get("anciennete_contrat_mois"),
                "couvertureGeographique": data.get("couverture_geographique")
            })

        if not update_data:
            return {
                "response": "Aucune donnée à mettre à jour.",
                "is_complete": True
            }

        response = requests.put(f"{self.service_url}/api/packs/{pack_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Pack {pack_id} mis à jour complètement avec
succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour complète:
{response.status_code} - {response.text}",
                "is_complete": False
            }
```

```python
import os
import requests
from typing import Dict, List, Any
from .base_chatbot import BaseChatbot

class UnifiedGarantieChatbot(BaseChatbot):
    """
    Chatbot unifié pour la création ET configuration de garanties
d'assurance.
    Permet de créer une garantie et de la configurer en une seule
conversation.
    """

    def __init__(self):
        # Définir les champs pour la création et la configuration
        fields = [
            {
                "key": "action_type",
                "question": "Que souhaitez-vous faire ? (creer / configurer)",
                "type": "choice",
                "choices": ["creer", "configurer"],
                "required": True,
                "validation": lambda x: x in ["creer", "configurer"],
                "error_message": "Veuillez choisir entre 'creer' ou
'configurer'."
            },
            {
                "key": "garantie_id",
                "question": "Quel est l'ID de la garantie ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID de la garantie doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "nom_garantie",
                "question": "Quel est le nom de la garantie ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "Le nom doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "description",
                "question": "Quelle est la description de la garantie ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 10,
```

```
                "error_message": "La description doit contenir au moins 10
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "type_garantie",
                "question": "Quel est le type de garantie ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) > 0,
                "error_message": "Le type de garantie ne peut pas être vide.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "statut",
                "question": "Quel est le statut de la garantie ? (ACTIF /
INACTIF / EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE."
            },
            {
                "key": "configuration_type",
                "question": "Quel type de configuration ? (statut /
parametres_financiers / parametres_contractuels / complet)",
                "type": "choice",
                "choices": ["statut", "parametres_financiers",
"parametres_contractuels", "complet"],
                "required": True,
                "validation": lambda x: x in ["statut",
"parametres_financiers", "parametres_contractuels", "complet"],
                "error_message": "Veuillez choisir parmi: statut,
parametres_financiers, parametres_contractuels, complet.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "new_statut",
                "question": "Quel est le nouveau statut ? (ACTIF / INACTIF /
EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE.",
```

```
                "condition": lambda data: data.get("configuration_type") ==
"statut"
            },
            {
                "key": "niveau_couverture",
                "question": "Quel est le niveau de couverture ? (basic /
premium / gold)",
                "type": "choice",
                "choices": ["basic", "premium", "gold"],
                "required": True,
                "validation": lambda x: x in ["basic", "premium", "gold"],
                "error_message": "Le niveau doit être: basic, premium ou
gold.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "taux_remboursement",
                "question": "Quel est le taux de remboursement (0-100%) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: 0 <= x <= 100,
                "error_message": "Le taux de remboursement doit être entre 0
et 100.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "type_montant",
                "question": "Quel est le type de montant ? (FORFAIT /
FRAIS_REELS / TARIF_CONVENTIONNE)",
                "type": "choice",
                "choices": ["FORFAIT", "FRAIS_REELS", "TARIF_CONVENTIONNE"],
                "required": True,
                "validation": lambda x: x in ["FORFAIT", "FRAIS_REELS",
"TARIF_CONVENTIONNE"],
                "error_message": "Veuillez choisir parmi: FORFAIT,
FRAIS_REELS, TARIF_CONVENTIONNE.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "plafond_annuel",
                "question": "Quel est le plafond annuel (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x > 0,
                "error_message": "Le plafond annuel doit être un nombre
positif.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
```

```
                "key": "plafond_mensuel",
                "question": "Quel est le plafond mensuel (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x > 0,
                "error_message": "Le plafond mensuel doit être un nombre
positif.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "plafond_par_acte",
                "question": "Quel est le plafond par acte (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x > 0,
                "error_message": "Le plafond par acte doit être un nombre
positif.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "franchise",
                "question": "Quelle est la franchise (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x >= 0,
                "error_message": "La franchise doit être un nombre positif ou
nul.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "cout_moyen_par_sinistre",
                "question": "Quel est le coût moyen par sinistre (en DT) ?",
                "type": "float",
                "required": True,
                "validation": lambda x: x >= 0,
                "error_message": "Le coût moyen par sinistre doit être un
nombre positif ou nul.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_financiers", "complet"]
            },
            {
                "key": "duree_min_contrat",
                "question": "Quelle est la durée minimale du contrat (en mois)
?",
                "type": "int",
                "required": True,
                "validation": lambda x: x >= 1,
                "error_message": "La durée minimale doit être d'au moins 1
mois.",
```

```
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_contractuels", "complet"]
            },
            {
                "key": "duree_max_contrat",
                "question": "Quelle est la durée maximale du contrat (en mois)
?",
                "type": "int",
                "required": True,
                "validation": lambda x, data: x >
data.get("duree_min_contrat", 0),
                "error_message": "La durée maximale doit être supérieure à la
durée minimale.",
                "depends_on": "duree_min_contrat",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_contractuels", "complet"]
            },
            {
                "key": "resiliable_annuellement",
                "question": "Le contrat est-il résiliable annuellement ?
(Oui/Non)",
                "type": "boolean",
                "required": True,
                "validation": lambda x: isinstance(x, bool),
                "error_message": "Veuillez répondre par 'Oui' ou 'Non'.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["parametres_contractuels", "complet"]
            },
            {
                "key": "cree_par",
                "question": "Qui a créé cette garantie ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) > 0,
                "error_message": "Le créateur ne peut pas être vide.",
                "condition": lambda data: data.get("action_type") == "creer"
            }
        ]

        service_url = os.getenv("PRODUCT_SERVICE_URL",
"http://localhost:9093")
        super().__init__(fields, service_url, "Garantie")

    def _should_ask_field(self, field_key: str, data: Dict) -> bool:
        """Détermine si un champ doit être demandé selon les conditions"""
        field = next((f for f in self.fields if f["key"] == field_key), None)
        if not field:
            return False

        # Si le champ a une condition, l'évaluer
        if "condition" in field:
            return field["condition"](data)
```

```python
        return True

    def _get_next_field(self) -> Dict:
        """Obtient le prochain champ à demander selon les conditions"""
        data = self.collected_data

        # Parcourir les champs dans l'ordre
        for i, field in enumerate(self.fields):
            if i <= self.current_field_index:
                continue

            # Vérifier si ce champ doit être demandé
            if self._should_ask_field(field["key"], data):
                self.current_field_index = i
                return field

        return None

    def _handle_collecting(self, message: str) -> Dict[str, Any]:
        """Gère la collecte des données avec champs conditionnels"""
        data = self.collected_data
        current_field = self._get_current_field()

        if not current_field:
            return self._handle_welcome(message)

        field_key = current_field["key"]

        # Vérifier si ce champ doit être traité
        if not self._should_ask_field(field_key, data):
            # Passer au champ suivant
            self.current_field_index += 1
            return self._handle_collecting(message)

        # Parser et valider le message
        parsed_value = self._smart_parse(message, current_field)

        if parsed_value is None:
            return {
                "response": current_field["error_message"],
                "is_complete": False,
                "field": field_key
            }

        # Valider la valeur
        if not self._validate_field(parsed_value, current_field):
            return {
                "response": current_field["error_message"],
                "is_complete": False,
                "field": field_key
            }

        # Stocker la valeur
```

```python
        data[field_key] = parsed_value

        # Passer au champ suivant
        self.current_field_index += 1

        # Vérifier si la collecte est terminée
        next_field = self._get_next_field()
        if not next_field:
            return self._handle_validation("")

        return {
            "response": next_field["question"],
            "is_complete": False,
            "field": next_field["key"]
        }

    def _handle_validation(self, message: str) -> Dict[str, Any]:
        """Gère la validation et la création/configuration"""
        data = self.collected_data
        action_type = data.get("action_type")

        try:
            if action_type == "creer":
                return self._create_garantie(data)
            elif action_type == "configurer":
                return self._configure_garantie(data)
            else:
                return {
                    "response": "Type d'action non valide.",
                    "is_complete": False
                }
        except Exception as e:
            return {
                "response": f"Erreur lors de l'opération: {str(e)}",
                "is_complete": False
            }

    def _create_garantie(self, data: Dict) -> Dict[str, Any]:
        """Crée une nouvelle garantie"""
        garantie_data = {
            "nomGarantie": data["nom_garantie"],
            "description": data["description"],
            "typeGarantie": data["type_garantie"],
            "statut": data["statut"],
            "niveauCouverture": data["niveau_couverture"],
            "tauxRemboursement": data["taux_remboursement"],
            "typeMontant": data["type_montant"],
            "plafondAnnuel": data["plafond_annuel"],
            "plafondMensuel": data["plafond_mensuel"],
            "plafondParActe": data["plafond_par_acte"],
            "franchise": data["franchise"],
            "coutMoyenParSinistre": data["cout_moyen_par_sinistre"],
            "dureeMinContrat": data["duree_min_contrat"],
```

```
            "dureeMaxContrat": data["duree_max_contrat"],
            "resiliableAnnuellement": data["resiliable_annuellement"],
            "creePar": data["cree_par"]
        }

        response = requests.post(f"{self.service_url}/api/garanties",
json=garantie_data, timeout=10)

        if response.status_code == 201:
            garantie = response.json()
            return {
                "response": f"Garantie '{garantie['nomGarantie']}' créée avec
succès ! ID: {garantie['idGarantie']}",
                "is_complete": True,
                "data": garantie
            }
        else:
            return {
                "response": f"Erreur lors de la création:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _configure_garantie(self, data: Dict) -> Dict[str, Any]:
        """Configure une garantie existante"""
        garantie_id = data["garantie_id"]
        config_type = data["configuration_type"]

        if config_type == "statut":
            return self._update_garantie_statut(garantie_id,
data["new_statut"])
        elif config_type == "parametres_financiers":
            return self._update_garantie_financiers(garantie_id, data)
        elif config_type == "parametres_contractuels":
            return self._update_garantie_contractuels(garantie_id, data)
        elif config_type == "complet":
            return self._update_garantie_complet(garantie_id, data)
        else:
            return {
                "response": "Type de configuration non valide.",
                "is_complete": False
            }

    def _update_garantie_statut(self, garantie_id: str, new_statut: str) ->
Dict[str, Any]:
        """Met à jour le statut d'une garantie"""
        response =
requests.patch(f"{self.service_url}/api/garanties/{garantie_id}/desactiver",
timeout=10)

        if response.status_code == 200:
            return {
```

```
                "response": f"Statut de la garantie {garantie_id} mis à jour
avec succès vers {new_statut}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour du statut:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_garantie_financiers(self, garantie_id: str, data: Dict) ->
Dict[str, Any]:
        """Met à jour les paramètres financiers d'une garantie"""
        update_data = {
            "niveauCouverture": data.get("niveau_couverture"),
            "tauxRemboursement": data.get("taux_remboursement"),
            "typeMontant": data.get("type_montant"),
            "plafondAnnuel": data.get("plafond_annuel"),
            "plafondMensuel": data.get("plafond_mensuel"),
            "plafondParActe": data.get("plafond_par_acte"),
            "franchise": data.get("franchise"),
            "coutMoyenParSinistre": data.get("cout_moyen_par_sinistre")
        }

        response =
requests.put(f"{self.service_url}/api/garanties/{garantie_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Paramètres financiers de la garantie
{garantie_id} mis à jour avec succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour des paramètres
financiers: {response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_garantie_contractuels(self, garantie_id: str, data: Dict) ->
Dict[str, Any]:
        """Met à jour les paramètres contractuels d'une garantie"""
        update_data = {
            "dureeMinContrat": data.get("duree_min_contrat"),
            "dureeMaxContrat": data.get("duree_max_contrat"),
            "resiliableAnnuellement": data.get("resiliable_annuellement")
        }
```

```
        response =
requests.put(f"{self.service_url}/api/garanties/{garantie_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Paramètres contractuels de la garantie
{garantie_id} mis à jour avec succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour des paramètres
contractuels: {response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_garantie_complet(self, garantie_id: str, data: Dict) ->
Dict[str, Any]:
        """Met à jour tous les paramètres d'une garantie"""
        update_data = {
            "niveauCouverture": data.get("niveau_couverture"),
            "tauxRemboursement": data.get("taux_remboursement"),
            "typeMontant": data.get("type_montant"),
            "plafondAnnuel": data.get("plafond_annuel"),
            "plafondMensuel": data.get("plafond_mensuel"),
            "plafondParActe": data.get("plafond_par_acte"),
            "franchise": data.get("franchise"),
            "coutMoyenParSinistre": data.get("cout_moyen_par_sinistre"),
            "dureeMinContrat": data.get("duree_min_contrat"),
            "dureeMaxContrat": data.get("duree_max_contrat"),
            "resiliableAnnuellement": data.get("resiliable_annuellement")
        }

        response =
requests.put(f"{self.service_url}/api/garanties/{garantie_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Tous les paramètres de la garantie {garantie_id}
ont été mis à jour avec succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour complète:
{response.status_code} - {response.text}",
                "is_complete": False
            }
import os
import requests
from typing import Dict, List, Any
```

```python
from .base_chatbot import BaseChatbot

class UnifiedProductChatbot(BaseChatbot):
    """
    Chatbot unifié pour la création ET configuration de produits
d'assurance.
    Permet de créer un produit et de le configurer en une seule
conversation.
    """

    def __init__(self):
        # Définir les champs pour la création et la configuration
        fields = [
            {
                "key": "action_type",
                "question": "Que souhaitez-vous faire ? (creer / configurer)",
                "type": "choice",
                "choices": ["creer", "configurer"],
                "required": True,
                "validation": lambda x: x in ["creer", "configurer"],
                "error_message": "Veuillez choisir entre 'creer' ou
'configurer'."
            },
            {
                "key": "product_id",
                "question": "Quel est l'ID du produit ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) >= 3,
                "error_message": "L'ID du produit doit contenir au moins 3
caractères.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "nom_produit",
                "question": "Quel est le nom du produit ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) > 0,
                "error_message": "Le nom du produit ne peut pas être vide.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "description",
                "question": "Quelle est la description du produit ?",
                "type": "text",
                "required": True,
                "validation": lambda x: len(x.strip()) > 10,
                "error_message": "La description doit contenir au moins 10
caractères.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
```

```
            {
                "key": "type_produit",
                "question": "Quel est le type de produit ? (Sante / Habitation
/ Auto / Epargne / vie)",
                "type": "choice",
                "choices": ["Sante", "Habitation", "Auto", "Epargne", "vie"],
                "required": True,
                "validation": lambda x: x in ["Sante", "Habitation", "Auto",
"Epargne", "vie"],
                "error_message": "Veuillez choisir parmi: Sante, Habitation,
Auto, Epargne, vie.",
                "condition": lambda data: data.get("action_type") == "creer"
            },
            {
                "key": "statut",
                "question": "Quel est le statut du produit ? (ACTIF / INACTIF
/ EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE."
            },
            {
                "key": "configuration_type",
                "question": "Quel type de configuration ? (statut /
conditions_eligibilite / complet)",
                "type": "choice",
                "choices": ["statut", "conditions_eligibilite", "complet"],
                "required": True,
                "validation": lambda x: x in ["statut",
"conditions_eligibilite", "complet"],
                "error_message": "Veuillez choisir parmi: statut,
conditions_eligibilite, complet.",
                "condition": lambda data: data.get("action_type") ==
"configurer"
            },
            {
                "key": "new_statut",
                "question": "Quel est le nouveau statut ? (ACTIF / INACTIF /
EN_ATTENTE / SUSPENDU / EXPIRE / RESLIE)",
                "type": "choice",
                "choices": ["ACTIF", "INACTIF", "EN_ATTENTE", "SUSPENDU",
"EXPIRE", "RESLIE"],
                "required": True,
                "validation": lambda x: x in ["ACTIF", "INACTIF",
"EN_ATTENTE", "SUSPENDU", "EXPIRE", "RESLIE"],
                "error_message": "Veuillez choisir parmi: ACTIF, INACTIF,
EN_ATTENTE, SUSPENDU, EXPIRE, RESLIE.",
```

```
                "condition": lambda data: data.get("configuration_type") ==
"statut"
            },
            {
                "key": "add_condition_eligibilite",
                "question": "Voulez-vous ajouter une condition d'éligibilité ?
(Oui/Non)",
                "type": "boolean",
                "required": True,
                "validation": lambda x: isinstance(x, bool),
                "error_message": "Veuillez répondre par 'Oui' ou 'Non'.",
                "condition": lambda data: data.get("action_type") == "creer"
or data.get("configuration_type") in ["conditions_eligibilite", "complet"]
            }
        ]

        # Champs pour les conditions d'éligibilité
        condition_fields = [
            {
                "key": "age_minimum",
                "question": "Quel est l'âge minimum autorisé ?",
                "type": "int",
                "required": True,
                "validation": lambda x: 0 <= x <= 120,
                "error_message": "L'âge minimum doit être entre 0 et 120."
            },
            {
                "key": "age_maximum",
                "question": "Quel est l'âge maximum autorisé ?",
                "type": "int",
                "required": True,
                "validation": lambda x, data: x > data.get("age_minimum", 0),
                "error_message": "L'âge maximum doit être supérieur à l'âge
minimum.",
                "depends_on": "age_minimum"
            },
            {
                "key": "type_clients",
                "question": "Quels types de clients sont acceptés ?
(INDIVIDUEL / FAMILLE / ENFANT / SENIOR / ENTREPRISE / ETUDIANT) - Séparez
par des virgules",
                "type": "list",
                "required": True,
                "validation": lambda x: all(client in ["INDIVIDUEL",
"FAMILLE", "ENFANT", "SENIOR", "ENTREPRISE", "ETUDIANT"] for client in x),
                "error_message": "Types valides: INDIVIDUEL, FAMILLE, ENFANT,
SENIOR, ENTREPRISE, ETUDIANT."
            },
            {
                "key": "anciennete_contrat_mois",
                "question": "Quelle est l'ancienneté de contrat minimale
requise (en mois) ?",
                "type": "int",
```

```
                "required": True,
                "validation": lambda x: x >= 0,
                "error_message": "L'ancienneté doit être un nombre positif."
            },
            {
                "key": "couverture_geographique",
                "question": "Quelle est la couverture géographique ? (LOCAL /
NATIONAL / INTERNATIONAL / UE / MAGHREB)",
                "type": "choice",
                "choices": ["LOCAL", "NATIONAL", "INTERNATIONAL", "UE",
"MAGHREB"],
                "required": True,
                "validation": lambda x: x in ["LOCAL", "NATIONAL",
"INTERNATIONAL", "UE", "MAGHREB"],
                "error_message": "Veuillez choisir parmi: LOCAL, NATIONAL,
INTERNATIONAL, UE, MAGHREB."
            }
        ]

        # Combiner tous les champs
        all_fields = fields + condition_fields

        service_url = os.getenv("PRODUCT_SERVICE_URL",
"http://localhost:9093")
        super().__init__(all_fields, service_url, "Produit")

        # Stocker les champs conditionnels séparément
        self.condition_fields = condition_fields

    def _should_ask_field(self, field_key: str, data: Dict) -> bool:
        """Détermine si un champ doit être demandé selon les conditions"""
        field = next((f for f in self.fields if f["key"] == field_key), None)
        if not field:
            return False

        # Si le champ a une condition, l'évaluer
        if "condition" in field:
            return field["condition"](data)

        return True

    def _get_next_field(self) -> Dict:
        """Obtient le prochain champ à demander selon les conditions"""
        data = self.collected_data

        # Parcourir les champs dans l'ordre
        for i, field in enumerate(self.fields):
            if i <= self.current_field_index:
                continue

            # Vérifier si ce champ doit être demandé
            if self._should_ask_field(field["key"], data):
                self.current_field_index = i
```

```
            return field

    return None

def _handle_collecting(self, message: str) -> Dict[str, Any]:
    """ Gère la collecte des données avec champs conditionnels """
    data = self.collected_data
    current_field = self._get_current_field()

    if not current_field:
        return self._handle_welcome(message)

    field_key = current_field["key"]

    # Vérifier si ce champ doit être traité
    if not self._should_ask_field(field_key, data):
        # Passer au champ suivant
        self.current_field_index += 1
        return self._handle_collecting(message)

    # Parser et valider le message
    parsed_value = self._smart_parse(message, current_field)

    if parsed_value is None:
        return {
            "response": current_field["error_message"],
            "is_complete": False,
            "field": field_key
        }

    # Valider la valeur
    if not self._validate_field(parsed_value, current_field):
        return {
            "response": current_field["error_message"],
            "is_complete": False,
            "field": field_key
        }

    # Stocker la valeur
    data[field_key] = parsed_value

    # Passer au champ suivant
    self.current_field_index += 1

    # Vérifier si la collecte est terminée
    next_field = self._get_next_field()
    if not next_field:
        return self._handle_validation("")

    return {
        "response": next_field["question"],
        "is_complete": False,
        "field": next_field["key"]
```

```
        }

    def _handle_validation(self, message: str) -> Dict[str, Any]:
        """Gère la validation et la création/configuration"""
        data = self.collected_data
        action_type = data.get("action_type")

        try:
            if action_type == "creer":
                return self._create_product(data)
            elif action_type == "configurer":
                return self._configure_product(data)
            else:
                return {
                    "response": "Type d'action non valide.",
                    "is_complete": False
                }
        except Exception as e:
            return {
                "response": f"Erreur lors de l'opération: {str(e)}",
                "is_complete": False
            }

    def _create_product(self, data: Dict) -> Dict[str, Any]:
        """Crée un nouveau produit"""
        product_data = {
            "nomProduit": data["nom_produit"],
            "description": data["description"],
            "typeProduit": data["type_produit"],
            "statut": data["statut"]
        }

        # Ajouter les conditions d'éligibilité si présentes
        if data.get("add_condition_eligibilite"):
            product_data.update({
                "ageMinimum": data.get("age_minimum"),
                "ageMaximum": data.get("age_maximum"),
                "typeClients": data.get("type_clients", []),
                "anciennetеContratMois": data.get("anciennete_contrat_mois"),
                "couvertureGeographique": data.get("couverture_geographique")
            })

        response = requests.post(f"{self.service_url}/api/produits",
json=product_data, timeout=10)

        if response.status_code == 201:
            product = response.json()
            return {
                "response": f"Produit '{product['nomProduit']}' créé avec
succès ! ID: {product['idProduit']}",
                "is_complete": True,
                "data": product
            }
```

```
        else:
            return {
                "response": f"Erreur lors de la création:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _configure_product(self, data: Dict) -> Dict[str, Any]:
        """Configure un produit existant"""
        product_id = data["product_id"]
        config_type = data["configuration_type"]

        if config_type == "statut":
            return self._update_product_statut(product_id, data["new_statut"])
        elif config_type in ["conditions_eligibilite", "complet"]:
            return self._update_product_conditions(product_id, data)
        else:
            return {
                "response": "Type de configuration non valide.",
                "is_complete": False
            }

    def _update_product_statut(self, product_id: str, new_statut: str) ->
Dict[str, Any]:
        """Met à jour le statut d'un produit"""
        response =
requests.patch(f"{self.service_url}/api/produits/{product_id}/desactiver",
timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Statut du produit {product_id} mis à jour avec
succès vers {new_statut}",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour du statut:
{response.status_code} - {response.text}",
                "is_complete": False
            }

    def _update_product_conditions(self, product_id: str, data: Dict) ->
Dict[str, Any]:
        """Met à jour les conditions d'éligibilité d'un produit"""
        if not data.get("add_condition_eligibilite"):
            return {
                "response": "Aucune condition d'éligibilité à mettre à jour.",
                "is_complete": True
            }

        update_data = {
            "ageMinimum": data.get("age_minimum"),
```

```
            "ageMaximum": data.get("age_maximum"),
            "typeClients": data.get("type_clients", []),
            "anciennetеContratMois": data.get("anciennete_contrat_mois"),
            "couvertureGeographique": data.get("couverture_geographique")
        }

        response =
requests.put(f"{self.service_url}/api/produits/{product_id}",
json=update_data, timeout=10)

        if response.status_code == 200:
            return {
                "response": f"Conditions d'éligibilité du produit {product_id}
mises à jour avec succès",
                "is_complete": True
            }
        else:
            return {
                "response": f"Erreur lors de la mise à jour des conditions:
{response.status_code} - {response.text}",
                "is_complete": False
            }
import re
import requests
from typing import Dict, List, Optional, Any, Union
from enum import Enum

class ChatState(Enum):
    """États de la conversation partagés par tous les chatbots
d'administration"""
    WELCOME = "welcome"
    COLLECTING = "collecting"
    VALIDATION = "validation"
    CONFIRMATION = "confirmation"
    CORRECTION = "correction"
    COMPLETED = "completed"

class BaseChatbot:
    """
    Classe de base pour les chatbots de création d'entités.
    Gère la state machine, le parsing, la correction et les erreurs API de
manière uniforme.
    """

    def __init__(self, fields: List[Dict], service_url: str, entity_name:
str):
        self.fields = fields
        self.service_url = service_url
        self.entity_name = entity_name
        self.state = ChatState.WELCOME
        self.current_field_index = 0
        self.collected_data = {}
        self.validation_errors = []
```

```
        self.correction_field = None
        self.has_api_error = False

    def reset_state(self):
        """Réinitialise l'état du chatbot"""
        self.state = ChatState.WELCOME
        self.current_field_index = 0
        self.collected_data = {}
        self.validation_errors = []
        self.correction_field = None
        self.has_api_error = False

    def process_message(self, message: str) -> Dict[str, Any]:
        """Dispatche le message selon l'état actuel"""
        message = message.strip()

        if self.state == ChatState.WELCOME:
            return self._handle_welcome(message)
        elif self.state == ChatState.COLLECTING:
            return self._handle_collecting(message)
        elif self.state == ChatState.VALIDATION:
            return self._handle_validation(message)
        elif self.state == ChatState.CONFIRMATION:
            return self._handle_confirmation(message)
        elif self.state == ChatState.CORRECTION:
            return self._handle_correction(message)
        elif self.state == ChatState.COMPLETED:
            return self._handle_completed(message)
        return self._error_response("État inconnu")

    def _handle_welcome(self, message: str) -> Dict[str, Any]:
        self.state = ChatState.COLLECTING
        self.current_field_index = 0
        field = self._get_current_field()
        return {
            "response": f"Bonjour ! Je vais vous aider à créer un(e)
{self.entity_name}.\n\n{field['question']}",
            "state": self.state.value,
            "is_complete": False,
            "current_field": field["key"],
            "progress": 0
        }

    def _handle_collecting(self, message: str) -> Dict[str, Any]:
        field = self._get_current_field()
        if not field:
            return self._move_to_validation()

        parsed_value = self._smart_parse(message, field)
        if parsed_value is None or not self._validate_field(parsed_value,
field):
            return {
```

```
                "response": f"❌ {field.get('error_message', 'Valeur
invalide')}\n\n{field['question']}",
                "state": self.state.value,
                "is_complete": False,
                "current_field": field["key"],
                "progress": self._calculate_progress()
            }

        self.collected_data[field["key"]] = parsed_value
        self.current_field_index += 1

        next_field = self._get_current_field()
        if not next_field:
            return self._move_to_validation()

        return {
            "response": f"✅ Bien enregistré.\n\n{next_field['question']}",
            "state": self.state.value,
            "is_complete": False,
            "current_field": next_field["key"],
            "progress": self._calculate_progress()
        }

    def _move_to_validation(self) -> Dict[str, Any]:
        """Déclenche la phase de validation"""
        return self._handle_validation("")

    def _handle_validation(self, message: str) -> Dict[str, Any]:
        self.state = ChatState.VALIDATION
        errors = self._validate_all_data()
        if errors:
            self.validation_errors = errors
            self.state = ChatState.CONFIRMATION # Pour demander si on veut
corriger
            error_list = "\n".join([f"• {e}" for e in errors])
            return {
                "response": f"⚠️ Des erreurs ont été détectées
:\n{error_list}\n\nVoulez-vous corriger ? (Oui/Non)",
                "state": self.state.value,
                "is_complete": False
            }
        return self._move_to_confirmation()

    def _move_to_confirmation(self) -> Dict[str, Any]:
        self.state = ChatState.CONFIRMATION
        summary = self._generate_summary()
        return {
            "response": f" Récapitulatif :\n{summary}\n\nConfirmez-vous la
création ? (Oui/Non)",
            "state": self.state.value,
            "is_complete": False,
            "collected_data": self.collected_data
        }
```

```
    def _handle_confirmation(self, message: str) -> Dict[str, Any]:
        choice = self._parse_boolean(message)
        if choice is True:
            # Si on a des erreurs de validation OU si on sort d'une erreur
API
            if self.validation_errors or self.has_api_error:
                self.has_api_error = False # Reset après usage
                self.state = ChatState.CORRECTION
                return self._ask_which_field_to_correct()

            return self._perform_create()
        elif choice is False:
            # Si l'utilisateur dit Non alors qu'on propose de corriger
            if self.validation_errors or self.has_api_error:
                self.has_api_error = False # Reset après usage
                return {"response": "Création annulée.", "is_complete":
True}

            # Cas normal: Non à la confirmation -> Correction
            self.state = ChatState.CORRECTION
            return self._ask_which_field_to_correct()
        return {"response": "Veuillez répondre par Oui ou Non.", "state":
self.state.value}

    def _ask_which_field_to_correct(self) -> Dict[str, Any]:
        fields_list = "\n".join([f"• {f['key']}" for f in self.fields])
        return {
            "response": f"Quel champ souhaitez-vous corriger
?\n\n{fields_list}",
            "state": self.state.value
        }

    def _handle_correction(self, message: str) -> Dict[str, Any]:
        if not self.correction_field:
            field_key = message.lower().strip()
            field = next((f for f in self.fields if f["key"].lower() ==
field_key), None)
            if not field:
                return {"response": f"Champ '{field_key}' inconnu.
Réessayez.", "state": self.state.value}
            self.correction_field = field
            return {"response": f"Quelle est la nouvelle valeur pour :
{field['question']}", "state": self.state.value}

        # On a le champ, on parse la valeur
        val = self._smart_parse(message, self.correction_field)
        if val is not None and self._validate_field(val,
self.correction_field):
            self.collected_data[self.correction_field["key"]] = val
            self.correction_field = None
            return self._handle_validation("") # Re-valider tout
```

```
        return {"response": f"❌ Invalide.
{self.correction_field['error_message']}\n{self.correction_field['question']}
", "state": self.state.value}

    def _perform_create(self) -> Dict[str, Any]:
        """À implémenter par les classes filles pour l'appel API
spécifique"""
        raise NotImplementedError

    def _handle_api_response(self, response: requests.Response) -> Dict[str,
Any]:
        """Gère les réponses API de manière intelligente, notamment le
400"""
        if response.status_code in [200, 201]:
            self.state = ChatState.COMPLETED
            return {"response": f"✅ {self.entity_name} créé(e) avec succès
!", "is_complete": True}

        if response.status_code == 400:
            try:
                error_data = response.json()
                msg = error_data.get("message", "Données invalides")
                self.has_api_error = True # Verrouiller l'état d'erreur
                return {
                    "response": f"❌ L'API a rejeté la demande :
{msg}\n\nSouhaitez-vous corriger un champ ? (Oui/Non)",
                    "state": ChatState.CONFIRMATION.value,
                    "is_complete": False,
                    "has_api_error": True
                }
            except:
                pass

        return {"response": f"❌ Erreur serveur (HTTP
{response.status_code}).", "is_complete": False}

    # --- Utils ---
    def _get_current_field(self) -> Optional[Dict]:
        return self.fields[self.current_field_index] if
self.current_field_index < len(self.fields) else None

    def _calculate_progress(self) -> int:
        return int((len(self.collected_data) / len(self.fields)) * 100)

    def _validate_field(self, value: Any, field: Dict) -> bool:
        try:
            if "depends_on" in field:
                return field["validation"](value, self.collected_data)
            return field["validation"](value)
        except:
            return False

    def _validate_all_data(self) -> List[str]:
```

```
        """Validation croisée basique. Peut être surchargée."""
        return []

    def _generate_summary(self) -> str:
        lines = []
        for f in self.fields:
            val = self.collected_data.get(f["key"], "Non saisi")
            lines.append(f"• {f['key']} : {val}")
        return "\n".join(lines)

    def _smart_parse(self, message: str, field: Dict) -> Any:
        t = field["type"]
        msg = message.strip()
        if t == "text": return msg if msg else None
        if t == "int":
            nums = re.findall(r'\d+', msg)
            if not nums: return None
            val = int(nums[0])
            if 'an' in msg.lower(): val *= 12
            return val
        if t == "float":
            nums = re.findall(r'\d+\.?\d*', msg.replace(',', '.'))
            return float(nums[0]) if nums else None
        if t == "boolean": return self._parse_boolean(msg)
        if t == "choice":
            choices = field.get("choices", [])
            for c in choices:
                if c.lower() in msg.lower(): return c
            return None
        if t == "list":
            items = [i.strip() for i in re.split(r'[,\s;]+', msg) if
i.strip()]
            return items if items else None
        return None

    def _parse_boolean(self, message: str) -> Optional[bool]:
        m = message.lower().strip()
        # Utilisation de regex pour éviter les faux positifs (comme 'non'
contenant 'o')
        if re.search(r'\b(oui|yes|o|vrai|actif|ok)\b', m): return True
        if re.search(r'\b(non|no|n|faux|inactif|stop)\b', m): return False
        return None

    def _handle_completed(self, message: str) -> Dict[str, Any]:
        return {"response": "Opération terminée. Tapez 'Pack', 'Produit' ou
'Garantie' pour recommencer.", "is_complete": True}

    def _error_response(self, msg: str) -> Dict[str, Any]:
        return {"response": f"Erreur technique: {msg}", "is_complete": False}

    def get_welcome_message(self) -> Dict[str, Any]:
        self.reset_state()
        return self._handle_welcome("")
```

```python
"""
Configuration des API externes pour le service AI
"""

import os
from typing import Optional

class APIConfig:
    """Classe de configuration pour les API externes"""

    def __init__(self):
        self.google_api_key = os.getenv("GOOGLE_API_KEY")
        self.product_service_url = os.getenv("PRODUCT_SERVICE_URL",
"http://localhost:9093")
        self.recommendation_service_url =
os.getenv("RECOMMENDATION_SERVICE_URL", "http://localhost:9095")
        self.flask_port = int(os.getenv("FLASK_PORT", "5000"))
        self.flask_debug = os.getenv("FLASK_DEBUG", "True").lower() == "true"

    def get_google_api_key(self) -> Optional[str]:
        """Récupère la clé API Google"""
        return self.google_api_key

    def is_google_api_configured(self) -> bool:
        """Vérifie si la clé API Google est configurée"""
        return bool(self.google_api_key and len(self.google_api_key.strip())
> 0)

    def get_service_urls(self) -> dict:
        """Retourne les URLs des services"""
        return {
            "product_service": self.product_service_url,
            "recommendation_service": self.recommendation_service_url
        }

    def get_flask_config(self) -> dict:
        """Retourne la configuration Flask"""
        return {
            "port": self.flask_port,
            "debug": self.flask_debug
        }

    def validate_config(self) -> dict:
        """Valide la configuration et retourne les erreurs"""
        errors = []

        if not self.google_api_key:
            errors.append("GOOGLE_API_KEY n'est pas configurée")
        elif len(self.google_api_key.strip()) < 10:
            errors.append("GOOGLE_API_KEY semble invalide (trop courte)")

        if not self.product_service_url:
            errors.append("PRODUCT_SERVICE_URL n'est pas configurée")
```

```
        return {
            "is_valid": len(errors) == 0,
            "errors": errors
        }

# Instance globale de configuration
api_config = APIConfig()
""" Scoring Engine - Moteur de scoring ML
Utilise un modèle Random Forest pour prédire le score de pertinence
de chaque pack d'assurance par rapport au profil client.
Le modèle est entraîné sur des données simulées et peut être
ré-entraîné avec des données historiques réelles. """
import os
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib
import requests
from builtins import print

class ScoringEngine:
    """ Moteur de scoring basé sur du Machine Learning (Random Forest).
    Prédit la pertinence d'un pack d'assurance pour un profil client donné.
"""

    MODEL_PATH = os.path.join(os.path.dirname(__file__), "model.joblib")
    ENCODERS_PATH = os.path.join(os.path.dirname(__file__), "encoders.joblib")

    def __init__(self):
        self.model = None
        self.encoders = {}
        self.product_service_url = os.getenv("PRODUCT_SERVICE_URL",
"http://localhost:9093")

        # Charger le modèle existant si disponible
        if os.path.exists(self.MODEL_PATH) and
os.path.exists(self.ENCODERS_PATH):
            try:
                self.model = joblib.load(self.MODEL_PATH)
                self.encoders = joblib.load(self.ENCODERS_PATH)
                print("[ScoringEngine] Modèle chargé avec succès.")
            except (EOFError, Exception) as e:
                print(f"[ScoringEngine] Fichier modèle corrompu, suppression :
{e}")
                self.model = None
                self.encoders = {}
                # Supprimer les fichiers corrompus
                if os.path.exists(self.MODEL_PATH):
                    os.remove(self.MODEL_PATH)
```

```
                if os.path.exists(self.ENCODERS_PATH):
                    os.remove(self.ENCODERS_PATH)

    def train_model(self) -> dict:
        """ Entraîner le modèle ML sur des données simulées.
        En production, ces données viendraient de l'historique des
souscriptions. """
        # Générer des données simulées
        data = self._generate_training_data()
        df = pd.DataFrame(data)

        # Encoder les variables catégorielles
        self.encoders = {}
        categorical_cols = ["sexe", "profession", "situation_familiale",
"niveau_couverture"]

        for col in categorical_cols:
            le = LabelEncoder()
            # On s'assure qu'une catégorie "Autre" existe pour la robustesse
            unique_values = df[col].unique().tolist()
            if "Autre" not in unique_values and col in ["profession",
"situation_familiale"]:
                unique_values.append("Autre")

            le.fit(unique_values)
            df[col + "_encoded"] = le.transform(df[col].astype(str))
            self.encoders[col] = le

        # Features
        feature_cols = [
            "age", "sexe_encoded", "profession_encoded",
"situation_familiale_encoded",
            "maladie_chronique", "diabetique", "tension", "maladies_legeres",
            "nombre_beneficiaires", "budget_mensuel", "pack_prix",
"niveau_couverture_encoded",
            "has_garantie_chronique", "has_garantie_legere",
            "has_garantie_ophtalmo", "has_garantie_dentaire",
            "has_garantie_hospitalisation", "has_garantie_maternite"  ]

        X = df[feature_cols].values
        y = df["score_category"].values  # 0=mauvais, 1=moyen, 2=bon

        # Split
        X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

        # Entraîner le Random Forest
        self.model = RandomForestClassifier(
            n_estimators=100,
            max_depth=10,
            random_state=42,
            n_jobs=-1
        )
```

```
        self.model.fit(X_train, y_train)

        # Évaluer
        y_test_pred = self.model.predict(X_test)
        accuracy = accuracy_score(y_test, y_test_pred)

        # Sauvegarder
        joblib.dump(self.model, self.MODEL_PATH)
        joblib.dump(self.encoders, self.ENCODERS_PATH)

        return {
            "status": "success",
            "message": f"Modèle entraîné avec succès",
            "accuracy": round(accuracy * 100, 2),
            "training_samples": len(X_train),
            "test_samples": len(X_test),
            "features": feature_cols
        }

    def get_model_status(self) -> dict:
        """Retourne les informations sur l'état du modèle."""
        if self.model is None:
            return {"status": "NOT_TRAINED", "message": "Le modèle n'a pas
encore été initialisé."}

        return {
            "status": "READY",
            "model_type": "RandomForestClassifier",
            "n_estimators": self.model.n_estimators,
            "features": [
                "age", "sexe", "profession", "situation_familiale",
                "maladie_chronique", "diabetique", "tension",
"maladies_legeres",
                "nb_beneficiaires", "budget", "prix_pack", "niveau_couverture"
            ],
            "categories_supported": {col: list(le.classes_) for col, le in
self.encoders.items()}
        }

    def score_packs(self, client_profile: dict) -> dict:
        """  Scorer tous les packs disponibles pour un profil client donné.
        Args:
            client_profile: dict avec age, sexe, profession, etc.
        Returns:
            dict avec scored_packs trié par score décroissant   """
        # Si pas de modèle, entraîner d'abord
        if self.model is None:
            self.train_model()

        # Récupérer les packs depuis le Product Service
        try:
            packs = requests.get(f"{self.product_service_url}/api/packs",
timeout=5).json()
```

```
            produits =
requests.get(f"{self.product_service_url}/api/produits", timeout=5).json()
            garanties =
requests.get(f"{self.product_service_url}/api/garanties", timeout=5).json()
        except requests.exceptions.ConnectionError:
            return {
                "status": "error",
                "message": "Service produit non disponible",
                "scored_packs": []
            }

        # Maps
        garanties_map = {g["idGarantie"]: g for g in garanties}

        scored_packs = []
        for pack in packs:
            if not pack.get("actif", False):
                continue

            # Déterminer les garanties du pack directement depuis ses champs
            pack_garantie_types = set()
            all_garantie_ids = list(pack.get("garantiesInclusesIds") or []) +
\
                                list(pack.get("garantiesOptionnellesIds") or
[])
            for gid in all_garantie_ids:
                g = garanties_map.get(gid)
                if g:
                    pack_garantie_types.add(g.get("typeGarantie", ""))

            # Construire le feature vector
            features = self._build_features(client_profile, pack,
pack_garantie_types)

            # Prédire avec le modèle
            prediction = self.model.predict([features])[0]
            probabilities = self.model.predict_proba([features])[0]

            # Convertir en score 0-100
            score = round(probabilities[2] * 100 if len(probabilities) > 2
else probabilities[-1] * 100, 1)

            # Générer les raisons
            raisons = self._generate_reasons(client_profile,
pack_garantie_types, score, pack.get("prixMensuel", 0))

            scored_packs.append({
                "packId": pack.get("idPack"),
                "nomPack": pack.get("nomPack"),
                "score": score,
                "category": ["Non recommandé", "Acceptable",
"Recommandé"][int(prediction)],
                "raisons": raisons,
```

```
                "prixMensuel": pack.get("prixMensuel", 0),
                "niveauCouverture": pack.get("niveauCouverture", "")
            })

        # Trier par score décroissant
        scored_packs.sort(key=lambda x: x["score"], reverse=True)

        return {
            "status": "success",
            "client_profile": client_profile,
            "scored_packs": scored_packs,
            "total_packs_evaluated": len(scored_packs)
        }

    def _build_features(self, profile: dict, pack: dict, garantie_types: set)
-> list:
        """Construire le vecteur de features pour la prédiction."""
        # Encoder le sexe
        sexe_encoded = self._safe_encode("sexe", profile.get("sexe", "M"))
        profession_encoded = self._safe_encode("profession",
profile.get("profession", "Autre"))
        situation_encoded = self._safe_encode("situation_familiale",

profile.get("situation_familiale", "Célibataire"))
        niveau_encoded = self._safe_encode("niveau_couverture",
                                           pack.get("niveauCouverture",
"basic"))

        return [
            profile.get("age", 30),
            sexe_encoded,
            profession_encoded,
            situation_encoded,
            int(profile.get("maladie_chronique", False)),
            int(profile.get("diabetique", False)),
            int(profile.get("tension", False)),
            int(profile.get("maladies_legeres", False)),
            profile.get("nombre_beneficiaires", 1),
            profile.get("budget_mensuel", 0),
            pack.get("prixMensuel", 0),
            niveau_encoded,
            int("MALADIE_CHRONIQUE" in garantie_types),
            int("MALADIE_LEGERE" in garantie_types),
            int("OPHTALMOLOGIE" in garantie_types),
            int("DENTAIRE" in garantie_types),
            int("HOSPITALISATION" in garantie_types),
            int("MATERNITE" in garantie_types)
        ]

    def _safe_encode(self, col_name: str, value) -> int:
        """Encoder une valeur de manière safe (mapping vers 'Autre' si
inconnu)."""
        if col_name in self.encoders:
```

```
            le = self.encoders[col_name]
            val_str = str(value)
            if val_str in le.classes_:
                return le.transform([val_str])[0]

            # Si inconnu, essayer 'Autre' ou la première classe par défaut
            if "Autre" in le.classes_:
                return le.transform(["Autre"])[0]
            if "Autre" in val_str: # Parfois l'input contient déjà Autre
                return le.transform([le.classes_[0]])[0]

            return le.transform([le.classes_[0]])[0]
        return 0

    def _generate_reasons(self, profile: dict, garantie_types: set, score:
float, pack_prix: float = 0) -> list:
        """Générer des raisons explicatives plus précises."""
        raisons = []

        # Analyse des besoins critiques (Maladies chroniques)
        if profile.get("maladie_chronique"):
            if "MALADIE_CHRONIQUE" in garantie_types:
                raisons.append("✅ Couvre spécifiquement vos besoins
pathologiques")
            else:
                raisons.append("❌ Attention: Ne couvre pas les maladies
chroniques")

        # Analyse du budget
        budget = profile.get("budget_mensuel", 0)
        if budget > 0:
            if pack_prix <= budget:
                raisons.append("💰 Pile dans votre budget mensuel")
            elif pack_prix <= budget * 1.2:
                raisons.append("⚖️ Légèrement au-dessus du budget mais
équilibré")

        # Garanties premium
        confort_count = sum(1 for g in ["OPHTALMOLOGIE", "DENTAIRE",
"MATERNITE"] if g in garantie_types)
        if confort_count >= 2:
            raisons.append("💎 Excellent niveau de confort
(Optique/Dentaire/Maternité)")

        # Synthèse globale
        if score >= 80:
            raisons.insert(0, "⭐ Recommandation prioritaire pour votre
profil")
        elif score >= 50:
            raisons.insert(0, "👍 Option solide et adaptée")
        else:
            raisons.insert(0, "ℹ️ Option de base avec une couverture
limitée")
```

```
        return raisons

    def _generate_training_data(self) -> list:
        """
        Génère des données d'entraînement simulées.
        Simule 1000 interactions client/pack avec des scores réalistes.
        """
        np.random.seed(42)
        data = []

        sexes = ["M", "F"]
        professions = ["Ingénieur", "Médecin", "Enseignant", "Commerçant",
"Étudiant",
                       "Ouvrier", "Cadre", "Retraité", "Agriculteur",
"Autre"]
        situations = ["Célibataire", "Marié", "Divorcé", "Veuf"]
        niveaux = ["basic", "premium", "gold"]

        for _ in range(1000):
            age = np.random.randint(18, 70)
            sexe = np.random.choice(sexes)
            profession = np.random.choice(professions)
            situation = np.random.choice(situations)
            maladie_chronique = np.random.choice([0, 1], p=[0.7, 0.3])
            diabetique = np.random.choice([0, 1], p=[0.8, 0.2]) if
maladie_chronique else 0
            tension_val = np.random.choice([0, 1], p=[0.75, 0.25]) if
maladie_chronique else 0
            maladies_legeres = np.random.choice([0, 1], p=[0.4, 0.6])
            nombre_benef = np.random.randint(1, 6)
            budget = np.random.choice([50, 100, 150, 200, 300, 500])

            # Simuler un pack
            niveau = np.random.choice(niveaux)
            pack_prix = {"basic": np.random.uniform(30, 100),
                         "premium": np.random.uniform(100, 250),
                         "gold": np.random.uniform(250, 500)}[niveau]

            has_chronique = np.random.choice([0, 1], p=[0.4, 0.6])
            has_legere = np.random.choice([0, 1], p=[0.3, 0.7])
            has_ophtalmo = np.random.choice([0, 1], p=[0.5, 0.5])
            has_dentaire = np.random.choice([0, 1], p=[0.5, 0.5])
            has_hospi = np.random.choice([0, 1], p=[0.4, 0.6])
            has_maternite = np.random.choice([0, 1], p=[0.6, 0.4])

            # Calculer un score réaliste
            score = 0
            if maladie_chronique and has_chronique:
                score += 30
            if maladie_chronique and not has_chronique:
                score -= 20
            if diabetique and has_chronique:
```

```
        score += 15
    if tension_val and has_chronique:
        score += 10
    if maladies_legeres and has_legere:
        score += 15
    if has_ophtalmo:
        score += 5
    if has_hospi:
        score += 10
    if has_dentaire:
        score += 5
    if situation in ["Marié"] and has_maternite:
        score += 10
    if pack_prix <= budget:
        score += 10
    elif pack_prix > budget * 1.5:
        score -= 15
    if niveau == "gold":
        score += 5
    if age > 50 and has_hospi:
        score += 10

    # Normaliser et catégoriser
    score = max(0, min(100, score))
    if score >= 60:
        category = 2  # bon
    elif score >= 30:
        category = 1  # moyen
    else:
        category = 0  # mauvais

    data.append({
        "age": age,
        "sexe": sexe,
        "profession": profession,
        "situation_familiale": situation,
        "maladie_chronique": maladie_chronique,
        "diabetique": diabetique,
        "tension": tension_val,
        "maladies_legeres": maladies_legeres,
        "nombre_beneficiaires": nombre_benef,
        "budget_mensuel": budget,
        "pack_prix": round(pack_prix, 2),
        "niveau_couverture": niveau,
        "has_garantie_chronique": has_chronique,
        "has_garantie_legere": has_legere,
        "has_garantie_ophtalmo": has_ophtalmo,
        "has_garantie_dentaire": has_dentaire,
        "has_garantie_hospitalisation": has_hospi,
        "has_garantie_maternite": has_maternite,
        "score_category": category
    })
```

```python
        return data
"""
Service d'intégration avec l'API Google
Utilise la clé API configurée pour les fonctionnalités Google
"""
import os
import requests
import json
from typing import Dict, Any, Optional, List
from config.api_config import api_config

class GoogleAPIService:
    """Service pour interagir avec les API Google"""

    def __init__(self):
        self.api_key = api_config.get_google_api_key()
        self.base_url = "https://www.googleapis.com"

    def is_configured(self) -> bool:
        """Vérifie si le service est correctement configuré"""
        return api_config.is_google_api_configured()

    def get_headers(self) -> Dict[str, str]:
        """Retourne les en-têtes pour les requêtes API"""
        return {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {self.api_key}"
        }

    # API Google Places (pour les adresses, localisations)
    def search_places(self, query: str, location: Optional[str] = None,
radius: Optional[int] = None) -> Dict[str, Any]:
        """
        Recherche des lieux via Google Places API
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/place/textsearch/json"
        params = {
            "query": query,
            "key": self.api_key
        }

        if location:
            params["location"] = location
        if radius:
            params["radius"] = radius

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
```

```
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    def get_place_details(self, place_id: str) -> Dict[str, Any]:
        """
        Récupère les détails d'un lieu spécifique
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/place/details/json"
        params = {
            "place_id": place_id,
            "key": self.api_key,
            "fields":
"name,formatted_address,geometry,opening_hours,rating,reviews"
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    # API Google Geocoding
    def geocode_address(self, address: str) -> Dict[str, Any]:
        """
        Convertit une adresse en coordonnées GPS
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/geocode/json"
        params = {
            "address": address,
            "key": self.api_key
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
```

```
            return {"error": f"Erreur de connexion: {str(e)}"}

    def reverse_geocode(self, lat: float, lng: float) -> Dict[str, Any]:
        """
        Convertit des coordonnées GPS en adresse
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/geocode/json"
        params = {
            "latlng": f"{lat},{lng}",
            "key": self.api_key
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    # API Google Distance Matrix
    def calculate_distance_matrix(self, origins: List[str], destinations:
List[str], mode: str = "driving") -> Dict[str, Any]:
        """
        Calcule une matrice de distances entre plusieurs points
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/distancematrix/json"
        params = {
            "origins": "|".join(origins),
            "destinations": "|".join(destinations),
            "mode": mode,
            "key": self.api_key
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    # API Google Directions
```

```
    def get_directions(self, origin: str, destination: str, mode: str =
"driving", alternatives: bool = False) -> Dict[str, Any]:
        """
        Récupère les itinéraires entre deux points
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/maps/api/directions/json"
        params = {
            "origin": origin,
            "destination": destination,
            "mode": mode,
            "alternatives": "true" if alternatives else "false",
            "key": self.api_key
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    # API Google Translate (si nécessaire)
    def translate_text(self, text: str, target_language: str, source_language:
Optional[str] = None) -> Dict[str, Any]:
        """
        Traduit du texte via Google Translate API
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        url = f"{self.base_url}/language/translate/v2"
        params = {
            "key": self.api_key,
            "q": text,
            "target": target_language
        }

        if source_language:
            params["source"] = source_language

        try:
            response = requests.post(url, data=params, timeout=10)
            if response.status_code == 200:
                return response.json()
            else:
                return {"error": f"Erreur API: {response.status_code}",
"details": response.text}
```

```python
        except Exception as e:
            return {"error": f"Erreur de connexion: {str(e)}"}

    def validate_api_key(self) -> Dict[str, Any]:
        """
        Valide si la clé API fonctionne correctement
        """
        if not self.is_configured():
            return {"valid": False, "error": "Clé API non configurée"}

        # Test simple avec geocoding
        test_result = self.geocode_address("Paris, France")

        if "error" in test_result:
            return {"valid": False, "error": test_result["error"]}
        elif "status" in test_result and test_result["status"] == "OK":
            return {"valid": True, "message": "Clé API valide"}
        else:
            return {"valid": False, "error": "Réponse inattendue de l'API"}

    def get_usage_info(self) -> Dict[str, Any]:
        """
        Récupère les informations d'utilisation de l'API (si disponible)
        """
        if not self.is_configured():
            return {"error": "API Google non configurée"}

        # Note: Ceci nécessiterait l'API Google Cloud Platform billing
        # Pour l'instant, retourne les informations de base
        return {
            "api_key_configured": True,
            "api_key_length": len(self.api_key) if self.api_key else 0,
            "available_apis": [
                "Places API",
                "Geocoding API",
                "Distance Matrix API",
                "Directions API",
                "Translate API"
            ]
        }

# Instance globale du service
google_api_service = GoogleAPIService()

"""
Moteur de recommandation hybride intelligent pour PFE Vermeg
Combine prompt unique et conversation interactive
"""

import os
import requests
import json
from typing import Dict, List, Any, Optional, Tuple
```

```python
from datetime import datetime
import sys

sys.path.append(os.path.join(os.path.dirname(__file__), '..', 'config'))
sys.path.append(os.path.join(os.path.dirname(__file__), '..', 'services'))

from config.api_config import api_config
from services.enhanced_recommendation_service import
enhanced_recommendation_service
from services.google_api_service import google_api_service

class HybridRecommendationEngine:
    """Moteur hybride intelligent pour recommandations d'assurance"""

    def __init__(self):
        self.enhanced_service = enhanced_recommendation_service
        self.google_api = google_api_service

        # Schéma de données client complet
        self.client_schema = {
            "profil_personnel": {
                "age": {"type": "int", "required": True, "min": 18, "max":
100},
                "sexe": {"type": "choice", "required": True, "options":
["Homme", "Femme"]},
                "profession": {"type": "text", "required": True},
                "situation_familiale": {"type": "choice", "required": True,
                                        "options": ["Célibataire", "Marié",
"Divorcé", "Veuf"]},
                "nombre_enfants": {"type": "int", "required": False, "min": 0,
"max": 10},
                "revenu_mensuel": {"type": "float", "required": False, "min":
0}
            },
            "contexte_geographique": {
                "adresse": {"type": "text", "required": False},
                "code_postal": {"type": "text", "required": False},
                "ville": {"type": "text", "required": False},
                "rayon_deplacement": {"type": "int", "required": False, "min":
1, "max": 100}
            },
            "besoins_assurance": {
                "type_assurance": {"type": "choice", "required": True,
                                   "options": ["Santé", "Habitation", "Auto",
"Vie", "Multirisque"]},
                "budget_mensuel": {"type": "float", "required": True, "min":
10},
                "priorites": {"type": "multi_choice", "required": False,
                              "options": ["Prix", "Couverture", "Rapidité",
"Conseil", "Digital"]},
                "duree_contrat": {"type": "choice", "required": False,
                                  "options": ["1 an", "2 ans", "3 ans", "5 ans",
"Illimité"]}
```

```
            },
            "profil_sante": {
                "antecedents_medicaux": {"type": "boolean", "required":
False},
                "maladies_chroniques": {"type": "boolean", "required": False},
                "fumeur": {"type": "boolean", "required": False},
                "sports_pratiques": {"type": "multi_choice", "required":
False,
                                     "options": ["Aucun", "Course", "Natation",
"Cyclisme", "Fitness", "Autres"]},
                "visites_annuelles": {"type": "int", "required": False, "min":
0, "max": 20}
            },
            "contraintes_specifiques": {
                "exclusions": {"type": "multi_choice", "required": False,
                               "options": ["Dentaire", "Optique",
"Hospitalisation", "Maternité", "Sports"]},
                "preferences_assureur": {"type": "text", "required": False},
                "mode_paiement": {"type": "choice", "required": False,
                                  "options": ["Mensuel", "Trimestriel",
"Annuel"]},
                "date_effet_souhaitee": {"type": "date", "required": False}
            }
        }

    def analyze_input_completeness(self, client_data: Dict[str, Any]) ->
Dict[str, Any]:
        """Analyse la complétude des données client"""
        analysis = {
            "completion_rate": 0,
            "missing_fields": [],
            "missing_categories": [],
            "can_proceed_with_single_prompt": False,
            "needs_conversation": False,
            "priority_questions": []
        }

        total_fields = 0
        completed_fields = 0

        for category, fields in self.client_schema.items():
            category_complete = True
            category_missing = []

            for field_name, field_config in fields.items():
                total_fields += 1

                if field_config["required"] and field_name not in client_data:
                    category_complete = False
                    category_missing.append(field_name)

analysis["missing_fields"].append(f"{category}.{field_name}")
                elif field_name in client_data and client_data[field_name]:
```

```
                    completed_fields += 1

            if not category_complete:
                analysis["missing_categories"].append(category)
                analysis["needs_conversation"] = True

        analysis["completion_rate"] = completed_fields / total_fields if
total_fields > 0 else 0

        # Logique de décision
        analysis["can_proceed_with_single_prompt"] = (
            analysis["completion_rate"] >= 0.7 and  # 70% minimum
            "profil_personnel.age" not in analysis["missing_fields"] and
            "profil_personnel.sexe" not in analysis["missing_fields"] and
            "besoins_assurance.type_assurance" not in
analysis["missing_fields"] and
            "besoins_assurance.budget_mensuel" not in
analysis["missing_fields"]
        )

        # Générer les questions prioritaires
        analysis["priority_questions"] = self._generate_priority_questions(
            analysis["missing_categories"], analysis["missing_fields"]
        )

        return analysis

    def _generate_priority_questions(self, missing_categories: List[str],
missing_fields: List[str]) -> List[Dict[str, Any]]:
        """Génère les questions prioritaires basées sur les champs
manquants"""
        questions = []

        # Questions par ordre de priorité métier
        priority_mapping = {
            "profil_personnel.age": {
                "question": "Quel est votre âge ?",
                "type": "int",
                "validation": "between 18 and 100",
                "importance": "critique"
            },
            "profil_personnel.sexe": {
                "question": "Quel est votre sexe ?",
                "type": "choice",
                "options": ["Homme", "Femme"],
                "importance": "critique"
            },
            "besoins_assurance.type_assurance": {
                "question": "Quel type d'assurance recherchez-vous ?",
                "type": "choice",
                "options": ["Santé", "Habitation", "Auto", "Vie",
"Multirisque"],
                "importance": "critique"
```

```
            },
            "besoins_assurance.budget_mensuel": {
                "question": "Quel est votre budget mensuel pour l'assurance
?",
                "type": "float",
                "validation": ">= 10",
                "importance": "critique"
            },
            "profil_personnel.profession": {
                "question": "Quelle est votre profession ?",
                "type": "text",
                "importance": "haute"
            },
            "profil_personnel.situation_familiale": {
                "question": "Quelle est votre situation familiale ?",
                "type": "choice",
                "options": ["Célibataire", "Marié", "Divorcé", "Veuf"],
                "importance": "haute"
            },
            "contexte_geographique.adresse": {
                "question": "Quelle est votre adresse complète ?",
                "type": "text",
                "importance": "moyenne"
            },
            "profil_sante.antecedents_medicaux": {
                "question": "Avez-vous des antécédents médicaux importants ?",
                "type": "boolean",
                "importance": "moyenne"
            }
        }

        for field in missing_fields:
            if field in priority_mapping:
                questions.append({
                    "field": field,
                    **priority_mapping[field]
                })

        # Trier par importance
        importance_order = {"critique": 0, "haute": 1, "moyenne": 2, "basse":
3}
        questions.sort(key=lambda x: importance_order.get(x.get("importance",
"basse"), 3))

        return questions[:5]  # Limiter à 5 questions maximum

    def process_single_prompt(self, prompt: str) -> Dict[str, Any]:
        """Traite un prompt unique complet"""
        try:
            # Parser le prompt (JSON ou texte naturel)
            parsed_data = self._parse_comprehensive_prompt(prompt)

            # Enrichir avec contexte géographique si adresse fournie
```

```
            if "adresse" in parsed_data:
                geo_context =
self.google_api.geocode_address(parsed_data["adresse"])
                if "results" in geo_context and len(geo_context["results"]) >
0:
                    location =
geo_context["results"][0]["geometry"]["location"]
                    parsed_data["coordinates"] = {
                        "lat": location["lat"],
                        "lng": location["lng"]
                    }
                    parsed_data["formatted_address"] =
geo_context["results"][0]["formatted_address"]

            # Générer les recommandations
            recommendations =
self.enhanced_service.generate_recommendations(parsed_data)

            return {
                "success": True,
                "mode": "single_prompt",
                "parsed_data": parsed_data,
                "recommendations": recommendations,
                "processing_time": datetime.now().isoformat()
            }

        except Exception as e:
            return {
                "success": False,
                "error": f"Erreur lors du traitement du prompt: {str(e)}",
                "mode": "single_prompt"
            }

    def start_conversation(self, initial_data: Dict[str, Any]) -> Dict[str,
Any]:
        """Démarre une conversation pour collecter les informations
manquantes"""
        completeness = self.analyze_input_completeness(initial_data)

        if completeness["can_proceed_with_single_prompt"]:
            # Données suffisantes, traiter directement
            return self.process_single_prompt(initial_data)

        # Démarrer la conversation
        return {
            "success": True,
            "mode": "conversation",
            "session_id":
f"session_{datetime.now().strftime('%Y%m%d_%H%M%S')}",
            "current_data": initial_data,
            "completeness_analysis": completeness,
            "next_question": completeness["priority_questions"][0] if
completeness["priority_questions"] else None,
```

```
            "estimated_questions_remaining":
len(completeness["priority_questions"]),
            "message": self._generate_welcome_message(completeness)
        }

    def process_conversation_response(self, session_data: Dict[str, Any],
user_response: str) -> Dict[str, Any]:
        """Traite une réponse dans la conversation"""
        try:
            current_data = session_data["current_data"]
            current_question = session_data["next_question"]

            # Parser et valider la réponse
            parsed_response = self._parse_user_response(user_response,
current_question)

            if parsed_response["valid"]:
                # Ajouter la réponse aux données
                field_name = current_question["field"]
                current_data[field_name.split(".")[-1]] =
parsed_response["value"]

                # Ré-analyser la complétude
                new_completeness =
self.analyze_input_completeness(current_data)

                if new_completeness["can_proceed_with_single_prompt"]:
                    # Données complètes, générer les recommandations
                    recommendations =
self.enhanced_service.generate_recommendations(current_data)

                    return {
                        "success": True,
                        "mode": "conversation_completed",
                        "session_data": session_data,
                        "updated_data": current_data,
                        "recommendations": recommendations,
                        "message": "Parfait ! J'ai maintenant toutes les
informations nécessaires pour vous recommander les meilleures offres."
                    }
                else:
                    # Continuer la conversation
                    next_question = new_completeness["priority_questions"][0]
if new_completeness["priority_questions"] else None

                    return {
                        "success": True,
                        "mode": "conversation_continue",
                        "session_data": session_data,
                        "updated_data": current_data,
                        "completeness_analysis": new_completeness,
                        "next_question": next_question,
```

```
                        "completion_rate":
new_completeness["completion_rate"],
                        "message":
self._generate_acknowledgment_message(current_question,
parsed_response["value"])
                    }
            else:
                # Réponse invalide, poser la question différemment
                return {
                    "success": False,
                    "mode": "conversation_retry",
                    "session_data": session_data,
                    "next_question":
self._rephrase_question(current_question),
                    "error": parsed_response["error"],
                    "message": f"Je n'ai pas bien compris.
{self._rephrase_question(current_question)['question']}"
                }

        except Exception as e:
            return {
                "success": False,
                "error": f"Erreur lors du traitement de la réponse: {str(e)}",
                "mode": "conversation_error"
            }

    def _parse_comprehensive_prompt(self, prompt: str) -> Dict[str, Any]:
        """Parse un prompt complet (JSON ou texte naturel)"""
        try:
            # Essayer JSON d'abord
            if prompt.strip().startswith('{'):
                return json.loads(prompt)

            # Parser texte naturel
            return self._parse_natural_language_prompt(prompt)

        except Exception as e:
            raise ValueError(f"Erreur de parsing du prompt: {str(e)}")

    def _parse_natural_language_prompt(self, prompt: str) -> Dict[str, Any]:
        """Parse un prompt en langage naturel"""
        import re

        data = {}

        # Extraire l'âge
        age_match = re.search(r'(\d+)\s*ans?', prompt.lower())
        if age_match:
            data["age"] = int(age_match.group(1))

        # Extraire le sexe
        if any(word in prompt.lower() for word in ["homme", "masculin"]):
            data["sexe"] = "Homme"
```

```
        elif any(word in prompt.lower() for word in ["femme", "féminin"]):
            data["sexe"] = "Femme"

        # Extraire le type d'assurance
        insurance_types = ["santé", "habitation", "auto", "vie",
"multirisque"]
        for ins_type in insurance_types:
            if ins_type in prompt.lower():
                data["type_assurance"] = ins_type.title()
                break

        # Extraire le budget
        budget_match =
re.search(r'budget\s*[:=]?\s*(\d+(?:\.\d+)?)\s*?e?ur?', prompt.lower())
        if budget_match:
            data["budget_mensuel"] = float(budget_match.group(1))

        # Extraire l'adresse
        address_patterns = [
            r'adresse\s*[:=]?\s*([^,\.]+)',
            r'à\s*([^,\.]+)',
            r'(\d+\s+[^,\.]+\s+(?:rue|avenue|boulevard|place)[^,\.]*)'
        ]

        for pattern in address_patterns:
            match = re.search(pattern, prompt.lower())
            if match:
                data["adresse"] = match.group(1).strip()
                break

        return data

    def _parse_user_response(self, response: str, question: Dict[str, Any]) ->
Dict[str, Any]:
        """Parse et valide la réponse utilisateur"""
        question_type = question["type"]

        try:
            if question_type == "int":
                value = int(response.strip())
                if "validation" in question:
                    if "between" in question["validation"]:
                        min_val, max_val = map(int,
question["validation"].split("between")[1].strip().split("and"))
                        if not (min_val <= value <= max_val):
                            return {"valid": False, "error": f"Veuillez entrer
une valeur entre {min_val} et {max_val}"}
                return {"valid": True, "value": value}

            elif question_type == "float":
                value = float(response.strip().replace(',', '.'))
                if "validation" in question and ">=" in
question["validation"]:
```

```
                    min_val =
float(question["validation"].split(">=")[1].strip())
                    if value < min_val:
                        return {"valid": False, "error": f"Veuillez entrer une
valeur supérieure ou égale à {min_val}"}
                return {"valid": True, "value": value}

            elif question_type == "choice":
                response_clean = response.strip().title()
                if response_clean in question["options"]:
                    return {"valid": True, "value": response_clean}
                else:
                    return {"valid": False, "error": f"Veuillez choisir parmi:
{', '.join(question['options'])}"}

            elif question_type == "text":
                if len(response.strip()) >= 2:
                    return {"valid": True, "value": response.strip()}
                else:
                    return {"valid": False, "error": "Veuillez entrer au moins
2 caractères"}

            elif question_type == "boolean":
                response_lower = response.lower().strip()
                if response_lower in ["oui", "yes", "true", "1"]:
                    return {"valid": True, "value": True}
                elif response_lower in ["non", "no", "false", "0"]:
                    return {"valid": True, "value": False}
                else:
                    return {"valid": False, "error": "Veuillez répondre par
oui ou non"}

        except ValueError:
            return {"valid": False, "error": "Format invalide"}

        return {"valid": False, "error": "Type de question non géré"}

    def _generate_welcome_message(self, completeness: Dict[str, Any]) -> str:
        """Génère le message de bienvenue pour la conversation"""
        completion_rate = completeness["completion_rate"]

        if completion_rate == 0:
            return ("Bonjour ! Je suis votre conseiller assurance intelligent.
"
                    "Pour vous recommander les meilleures offres, j'ai besoin
de quelques informations. "
                    "Commençons par les plus importantes.")
        elif completion_rate < 0.3:
            return ("Merci pour ces informations. J'ai encore quelques
questions importantes "
                    "pour vous proposer les recommandations les plus
pertinentes.")
        else:
```

```
            return ("Excellent ! J'ai déjà une bonne partie des informations.
"
                    "Quelques questions supplémentaires pour affiner mes
recommandations.")

    def _generate_acknowledgment_message(self, question: Dict[str, Any],
value: Any) -> str:
        """Génère un message d'acquittement"""
        field_name = question["field"].split(".")[-1]

        acknowledgments = {
            "age": f"Merci ! {value} ans, noté.",
            "sexe": f"Parfait, bien noté.",
            "profession": f"Intéressant, {value}.",
            "budget_mensuel": f"Budget de {value}EUR/mois, compris.",
            "type_assurance": f"Assurance {value}, excellente choix."
        }

        return acknowledgments.get(field_name, "Parfait, bien noté.")

    def _rephrase_question(self, question: Dict[str, Any]) -> Dict[str, Any]:
        """Reformule une question pour plus de clarté"""
        rephrased = question.copy()

        if question["field"] == "profil_personnel.age":
            rephrased["question"] = "Pouvez-vous me donner votre âge en
chiffres (ex: 35) ?"
        elif question["field"] == "besoins_assurance.budget_mensuel":
            rephrased["question"] = "Quel budget mensuel pouvez-vous allouer à
l'assurance (en euros) ?"
        elif question["field"] == "contexte_geographique.adresse":
            rephrased["question"] = "Quelle est votre adresse complète pour
analyser les offres locales ?"

        return rephrased

# Instance globale du moteur hybride
hybrid_recommendation_engine = HybridRecommendationEngine()
"""
Service de recommandation amélioré avec intégration Google API
Utilise des données réelles et la géolocalisation pour des recommandations
pertinentes
"""

import os
import requests
import json
import math
from typing import Dict, List, Any, Optional, Tuple
from datetime import datetime, date
import sys

# Ajouter les répertoires au chemin Python
```

```python
sys.path.append(os.path.join(os.path.dirname(__file__), '..', 'config'))
sys.path.append(os.path.join(os.path.dirname(__file__), '..', 'chatbot'))

from config.api_config import api_config
from services.google_api_service import google_api_service

class EnhancedRecommendationService:
    """Service de recommandation avec données réelles et géolocalisation"""

    def __init__(self):
        self.product_service_url = api_config.product_service_url
        self.google_api = google_api_service

        # Poids pour le scoring des recommandations
        self.scoring_weights = {
            'age_match': 0.25,
            'budget_match': 0.20,
            'location_proximity': 0.20,
            'coverage_match': 0.15,
            'family_situation': 0.10,
            'health_profile': 0.10
        }

        # Facteurs de risque et tarification
        self.risk_factors = {
            'age_high_risk': 65,  # Âge à risque élevé
            'chronic_diseases_multiplier': 1.3,
            'smoker_multiplier': 1.2,
            'high_blood_pressure_multiplier': 1.15,
            'family_history_multiplier': 1.1
        }

    def get_user_location_context(self, user_data: Dict[str, Any]) ->
Dict[str, Any]:
        """Récupère le contexte géographique de l'utilisateur"""
        context = {
            'has_location': False,
            'coordinates': None,
            'address': None,
            'nearby_facilities': []
        }

        # Vérifier si l'utilisateur a fourni une adresse
        if 'address' in user_data and user_data['address']:
            address = user_data['address']

            # Géocoder l'adresse
            geo_result = self.google_api.geocode_address(address)

            if 'results' in geo_result and len(geo_result['results']) > 0:
                location = geo_result['results'][0]['geometry']['location']
                formatted_address =
geo_result['results'][0].get('formatted_address', address)
```

```
                context.update({
                    'has_location': True,
                    'coordinates': f"{location['lat']},{location['lng']}",
                    'address': formatted_address
                })

                # Rechercher des établissements de santé à proximité
                nearby_places = self.google_api.search_places(
                    "hôpital OR clinique OR pharmacie OR médecin",
                    context['coordinates'],
                    5000  # 5km de rayon
                )

                if 'results' in nearby_places:
                    context['nearby_facilities'] =
nearby_places['results'][:5]  # Top 5

        return context

    def calculate_age_risk_score(self, age: int, health_data: Dict[str, Any])
-> float:
        """Calcule un score de risque basé sur l'âge et les données de
santé"""
        base_score = 1.0

        # Facteur âge
        if age >= self.risk_factors['age_high_risk']:
            age_multiplier = 1 + (age - self.risk_factors['age_high_risk']) *
0.02
            base_score *= age_multiplier

        # Facteurs de santé
        if health_data.get('has_chronic_diseases', False):
            base_score *= self.risk_factors['chronic_diseases_multiplier']

        if health_data.get('is_smoker', False):
            base_score *= self.risk_factors['smoker_multiplier']

        if health_data.get('has_high_blood_pressure', False):
            base_score *= self.risk_factors['high_blood_pressure_multiplier']

        if health_data.get('has_family_history', False):
            base_score *= self.risk_factors['family_history_multiplier']

        return min(base_score, 3.0)  # Plafonner à 3x le tarif de base

    def get_real_pricing_data(self) -> Dict[str, Any]:
        """Récupère les données de tarification réelles depuis l'API
produit"""
        try:
            response = requests.get(f"{self.product_service_url}/api/packs",
timeout=10)
```

```
            if response.status_code == 200:
                packs = response.json()

                # Analyser les prix réels
                prices = [pack.get('prixMensuel', 0) for pack in packs if
isinstance(pack, dict)]

                return {
                    'packs': packs,
                    'price_range': {
                        'min': min(prices) if prices else 0,
                        'max': max(prices) if prices else 0,
                        'average': sum(prices) / len(prices) if prices else 0
                    },
                    'coverage_levels': self._analyze_coverage_levels(packs)
                }
        except Exception as e:
            print(f"Erreur lors de la récupération des données de
tarification: {e}")
            return {'packs': [], 'price_range': {'min': 0, 'max': 0,
'average': 0}}

    def _analyze_coverage_levels(self, packs: List[Dict]) -> Dict[str, Any]:
        """Analyse les niveaux de couverture disponibles"""
        coverage_stats = {
            'basic': {'count': 0, 'avg_price': 0},
            'premium': {'count': 0, 'avg_price': 0},
            'gold': {'count': 0, 'avg_price': 0}
        }

        for pack in packs:
            if isinstance(pack, dict):
                level = pack.get('niveauCouverture', 'basic')
                price = pack.get('prixMensuel', 0)

                if level in coverage_stats:
                    coverage_stats[level]['count'] += 1
                    coverage_stats[level]['avg_price'] += price

        # Calculer les moyennes
        for level in coverage_stats:
            if coverage_stats[level]['count'] > 0:
                coverage_stats[level]['avg_price'] /=
coverage_stats[level]['count']

        return coverage_stats

    def calculate_budget_match_score(self, user_budget: float, pack_price:
float) -> float:
        """Calcule le score de compatibilité budget"""
        if not user_budget or user_budget <= 0:
            return 0.5  # Score neutre si pas de budget
```

```
        if pack_price <= user_budget:
            # Bonus si le pack est significativement moins cher
            savings_ratio = (user_budget - pack_price) / user_budget
            return min(1.0, 0.7 + savings_ratio * 0.6)
        else:
            # Pénalité si le pack dépasse le budget
            over_budget_ratio = (pack_price - user_budget) / user_budget
            return max(0.0, 1.0 - over_budget_ratio * 2)

    def calculate_age_appropriateness(self, user_age: int, pack_data:
Dict[str, Any]) -> float:
        """Calcule le score d'adéquation âge-pack"""
        if not user_age:
            return 0.5

        # Analyser les conditions d'éligibilité du pack
        conditions = pack_data.get('conditionsEligibilite', {})
        age_min = conditions.get('ageMinimum', 0)
        age_max = conditions.get('ageMaximum', 120)

        if age_min <= user_age <= age_max:
            # Score optimal si dans la plage
            return 1.0
        elif user_age < age_min:
            # Pénalité si trop jeune
            diff = age_min - user_age
            return max(0.0, 1.0 - diff * 0.1)
        else:
            # Pénalité si trop âgé
            diff = user_age - age_max
            return max(0.0, 1.0 - diff * 0.05)

    def calculate_coverage_match(self, user_health: Dict[str, Any],
pack_guarantees: List[str]) -> float:
        """Calcule le score de correspondance de couverture"""
        if not user_health or not pack_guarantees:
            return 0.5

        # Besoins basés sur le profil de santé
        needs = []

        if user_health.get('has_chronic_diseases', False):
            needs.extend(['MALADIE_CHRONIQUE', 'SOINS_CHRONIQUES'])

        if user_health.get('has_dental_issues', False):
            needs.extend(['DENTAIRE', 'SOINS_DENTAIRES'])

        if user_health.get('has_vision_issues', False):
            needs.extend(['OPHTALMOLOGIE', 'LUNETTES'])

        if user_health.get('needs_hospitalization', False):
            needs.extend(['HOSPITALISATION', 'CHIRURGIE'])
```

```
        if not needs:
            return 0.8  # Bon score si pas de besoins spécifiques

        # Vérifier la correspondance
        matching_needs = sum(1 for need in needs if any(need in garantie for
garantie in pack_guarantees))
        total_needs = len(needs)

        return matching_needs / total_needs if total_needs > 0 else 0.5

    def calculate_family_situation_score(self, user_family: Dict[str, Any],
pack_data: Dict[str, Any]) -> float:
        """Calcule le score basé sur la situation familiale"""
        if not user_family:
            return 0.5

        situation = user_family.get('situation', 'célibataire')
        nombre_beneficiaires = user_family.get('nombre_beneficiaires', 1)

        # Analyser les garanties du pack pour la famille
        pack_guarantees = pack_data.get('garanties', [])
        family_coverage = any('FAMILLE' in str(g) for g in pack_guarantees)
        children_coverage = any('ENFANT' in str(g) for g in pack_guarantees)

        score = 0.5  # Score de base

        # Bonus pour couverture familiale
        if situation in ['marié', 'mariée'] and family_coverage:
            score += 0.3

        # Bonus pour couverture enfants
        if nombre_beneficiaires > 1 and children_coverage:
            score += 0.2

        return min(1.0, score)

    def calculate_location_proximity_score(self, user_location: Dict[str,
Any], pack_data: Dict[str, Any]) -> float:
        """Calcule le score basé sur la proximité géographique"""
        if not user_location.get('has_location', False):
            return 0.5  # Score neutre si pas de localisation

        # Vérifier si le pack a une couverture géographique spécifique
        conditions = pack_data.get('conditionsEligibilite', {})
        pack_coverage = conditions.get('couvertureGeographique', 'NATIONAL')

        # Score basé sur la proximité des établissements de santé
        nearby_facilities = user_location.get('nearby_facilities', [])

        if not nearby_facilities:
            return 0.6  # Score moyen si pas d'infos sur les établissements

        # Bonus pour bonne accessibilité aux soins
```

```
        facility_score = min(1.0, len(nearby_facilities) / 10)  # Normaliser
sur 10 établissements

        # Ajuster selon la couverture géographique
        if pack_coverage == 'LOCAL':
            return facility_score * 0.8
        elif pack_coverage == 'INTERNATIONAL':
            return 0.9  # Score élevé pour couverture internationale
        else:
            return facility_score

    def calculate_comprehensive_score(self, user_data: Dict[str, Any],
pack_data: Dict[str, Any],
                                      location_context: Dict[str, Any],
pricing_data: Dict[str, Any]) -> Dict[str, Any]:
        """Calcule un score de recommandation complet"""
        scores = {}

        # Score d'âge
        user_age = user_data.get('age', 30)
        age_score = self.calculate_age_appropriateness(user_age, pack_data)
        scores['age_match'] = age_score

        # Score de budget
        user_budget = user_data.get('budget_mensuel', 0)
        pack_price = pack_data.get('prixMensuel', 0)
        budget_score = self.calculate_budget_match_score(user_budget,
pack_price)
        scores['budget_match'] = budget_score

        # Score de localisation
        location_score =
self.calculate_location_proximity_score(location_context, pack_data)
        scores['location_proximity'] = location_score

        # Score de couverture santé
        user_health = user_data.get('sante', {})
        pack_guarantees = [g.get('nomGarantie', '') for g in
pack_data.get('garanties', [])]
        coverage_score = self.calculate_coverage_match(user_health,
pack_guarantees)
        scores['coverage_match'] = coverage_score

        # Score situation familiale
        user_family = user_data.get('famille', {})
        family_score = self.calculate_family_situation_score(user_family,
pack_data)
        scores['family_situation'] = family_score

        # Score profil de santé (risque)
        health_profile_score = 1.0 - (self.calculate_age_risk_score(user_age,
user_health) - 1.0) / 2.0
        scores['health_profile'] = health_profile_score
```

```
        # Score total pondéré
        total_score = sum(
            scores[key] * self.scoring_weights[key]
            for key in self.scoring_weights
        )
        scores['total_score'] = total_score

        return scores

    def generate_recommendations(self, user_data: Dict[str, Any]) -> Dict[str,
Any]:
        """Génère des recommandations personnalisées"""
        try:
            # Récupérer le contexte géographique
            location_context = self.get_user_location_context(user_data)

            # Récupérer les données de tarification réelles
            pricing_data = self.get_real_pricing_data()

            # Récupérer tous les packs disponibles
            all_packs = pricing_data.get('packs', [])

            # Calculer les scores pour chaque pack
            scored_packs = []

            for pack in all_packs:
                if isinstance(pack, dict):
                    scores = self.calculate_comprehensive_score(
                        user_data, pack, location_context, pricing_data
                    )

                    pack_with_score = pack.copy()
                    pack_with_score['recommendation_score'] = scores
                    scored_packs.append(pack_with_score)

            # Trier par score de recommandation
            scored_packs.sort(key=lambda x:
x['recommendation_score']['total_score'], reverse=True)

            # Générer les recommandations finales
            recommendations = []

            for i, pack in enumerate(scored_packs[:5]):  # Top 5
recommandations
                score_data = pack['recommendation_score']

                recommendation = {
                    'rank': i + 1,
                    'pack': {
                        'id': pack.get('idPack', ''),
                        'name': pack.get('nomPack', ''),
                        'description': pack.get('description', ''),
```

```
                        'price': pack.get('prixMensuel', 0),
                        'coverage_level': pack.get('niveauCouverture',
'basic'),
                        'product_id': pack.get('produitId', '')
                    },
                    'scores': score_data,
                    'recommendation_reasons':
self._generate_recommendation_reasons(score_data, user_data),
                    'pricing_analysis': self._analyze_pricing_for_user(pack,
user_data, pricing_data),
                    'location_benefits': self._get_location_benefits(pack,
location_context)
                }

                recommendations.append(recommendation)

            return {
                'success': True,
                'user_profile': {
                    'age': user_data.get('age', 'Non spécifié'),
                    'budget': user_data.get('budget_mensuel', 'Non spécifié'),
                    'location': location_context.get('address', 'Non
spécifiée'),
                    'health_profile': user_data.get('sante', {}),
                    'family_situation': user_data.get('famille', {})
                },
                'recommendations': recommendations,
                'analysis_metadata': {
                    'total_packs_analyzed': len(all_packs),
                    'location_enhanced': location_context.get('has_location',
False),
                    'nearby_facilities_count':
len(location_context.get('nearby_facilities', [])),
                    'pricing_data_source': 'real_api_data',
                    'scoring_model': 'enhanced_weighted_algorithm',
                    'generated_at': datetime.now().isoformat()
                }
            }

        except Exception as e:
            return {
                'success': False,
                'error': f"Erreur lors de la génération des recommandations:
{str(e)}",
                'fallback_recommendations':
self._get_fallback_recommendations(user_data)
            }

    def _generate_recommendation_reasons(self, scores: Dict[str, Any],
user_data: Dict[str, Any]) -> List[str]:
        """Génère les raisons des recommandations basées sur les scores"""
        reasons = []
```

```
        if scores.get('age_match', 0) > 0.8:
            reasons.append("Adapté à votre tranche d'âge")

        if scores.get('budget_match', 0) > 0.8:
            reasons.append("Respecte votre budget mensuel")

        if scores.get('location_proximity', 0) > 0.7:
            reasons.append("Bonne couverture dans votre région")

        if scores.get('coverage_match', 0) > 0.7:
            reasons.append("Couverture santé adaptée à vos besoins")

        if scores.get('family_situation', 0) > 0.7:
            reasons.append("Avantages pour votre situation familiale")

        if not reasons:
            reasons.append("Recommandation basée sur votre profil global")

        return reasons

    def _analyze_pricing_for_user(self, pack: Dict[str, Any], user_data:
Dict[str, Any],
                                  pricing_data: Dict[str, Any]) -> Dict[str,
Any]:
        """Analyse la tarification pour l'utilisateur"""
        pack_price = pack.get('prixMensuel', 0)
        user_budget = user_data.get('budget_mensuel', 0)
        price_range = pricing_data.get('price_range', {})

        analysis = {
            'price_position': 'moyenne',
            'budget_fit': 'inconnu',
            'value_score': 0.5
        }

        if price_range['min'] > 0 and price_range['max'] > 0:
            # Position dans la gamme de prix
            price_percentile = (pack_price - price_range['min']) /
(price_range['max'] - price_range['min'])

            if price_percentile < 0.33:
                analysis['price_position'] = 'économique'
                analysis['value_score'] = 0.9
            elif price_percentile < 0.67:
                analysis['price_position'] = 'moyenne'
                analysis['value_score'] = 0.7
            else:
                analysis['price_position'] = 'premium'
                analysis['value_score'] = 0.6

        # Adéquation budget
        if user_budget > 0:
            if pack_price <= user_budget * 0.8:
```

```
                analysis['budget_fit'] = 'très bon'
                analysis['value_score'] += 0.2
            elif pack_price <= user_budget:
                analysis['budget_fit'] = 'bon'
                analysis['value_score'] += 0.1
            else:
                analysis['budget_fit'] = 'dépasse le budget'
                analysis['value_score'] -= 0.2

        return analysis

    def _get_location_benefits(self, pack: Dict[str, Any], location_context:
Dict[str, Any]) -> List[str]:
        """Récupère les bénéfices liés à la localisation"""
        benefits = []

        if not location_context.get('has_location', False):
            return ["Spécifiez votre adresse pour des recommandations
localisées"]

        nearby_facilities = location_context.get('nearby_facilities', [])

        # Analyser les types d'établissements à proximité
        facility_types = [f.get('types', []) for f in nearby_facilities]
        flat_types = [item for sublist in facility_types for item in sublist]

        if 'hospital' in str(flat_types).lower():
            benefits.append("Proximité d'établissements hospitaliers")

        if 'pharmacy' in str(flat_types).lower():
            benefits.append("Accès facile aux pharmacies")

        if 'doctor' in str(flat_types).lower():
            benefits.append("Présence de médecins à proximité")

        # Vérifier la couverture géographique du pack
        conditions = pack.get('conditionsEligibilite', {})
        coverage = conditions.get('couvertureGeographique', 'NATIONAL')

        if coverage == 'LOCAL' and len(nearby_facilities) > 5:
            benefits.append("Couverture locale optimisée pour votre région")
        elif coverage == 'INTERNATIONAL':
            benefits.append("Couverture internationale complète")

        return benefits

    def _get_fallback_recommendations(self, user_data: Dict[str, Any]) ->
List[Dict[str, Any]]:
        """Recommandations de secours si l'API n'est pas disponible"""
        # Recommandations basiques basées sur l'âge et le budget
        age = user_data.get('age', 30)
        budget = user_data.get('budget_mensuel', 200)
```

```python
        fallback_recs = []

        if age < 30:
            if budget < 100:
                fallback_recs.append({
                    'type': 'basic_young',
                    'description': 'Pack essentiel pour jeunes',
                    'estimated_price': 50
                })
            else:
                fallback_recs.append({
                    'type': 'standard_young',
                    'description': 'Pack complet pour jeunes actifs',
                    'estimated_price': 120
                })
        elif age < 50:
            if budget < 150:
                fallback_recs.append({
                    'type': 'family_basic',
                    'description': 'Pack familial économique',
                    'estimated_price': 100
                })
            else:
                fallback_recs.append({
                    'type': 'family_comprehensive',
                    'description': 'Pack familial complet',
                    'estimated_price': 180
                })
        else:
            fallback_recs.append({
                'type': 'senior_comprehensive',
                'description': 'Pack senior avec couverture étendue',
                'estimated_price': 200
            })

        return fallback_recs

# Instance globale du service amélioré
enhanced_recommendation_service = EnhancedRecommendationService()
# Configuration du service IA
FLASK_PORT=5001
FLASK_DEBUG=True

# URL des microservices Spring Boot
PRODUCT_SERVICE_URL=http://localhost:9093
RECOMMENDATION_SERVICE_URL=http://localhost:9095

# Configuration JWT (doit correspondre au backend)
JWT_SECRET=vermeg-secret-key-2024
JWT_EXPIRATION=86400

# Désactiver temporairement l'authentification pour les tests
AUTH_ENABLED=false
```

```
# Clés API externes
GOOGLE_API_KEY=AIzaSyARA_9qf_WLhot5e5LL0Uyhkzc0mzvrHS8
flask==3.1.0
flask-cors==5.0.1
requests==2.32.3
scikit-learn==1.6.1
numpy==2.2.3
pandas==2.2.3
joblib==1.4.2
python-dotenv==1.1.0
```